Chapter 14 — Case Studies: Everyday Objects

Where this fits. Chapters 1–13 gave you the tools: OOP (object-oriented programming: objects that bundle data with the methods acting on it), UML (Unified Modeling Language, the standard way to draw classes and their links), SOLID (five class-design principles), clean code, the 23 GoF patterns (the "Gang of Four" patterns, after the four authors of the 1994 book *Design Patterns*) and concurrency (several pieces of work running at once). Chapters 14–21 apply them to real interview questions, starting here with **physical machines you have used**: a parking lot, a vending machine, a coffee machine, an ATM, an elevator, a traffic light and a parcel locker. You already know how they behave, so all your attention goes to the design.

What you will learn in this chapter

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK: sweep orders for visiting stops, see 14.5) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System (Amazon Locker)	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

Key terms — the pattern names in this table. These are design patterns (proven, named shapes for arranging classes) from the earlier chapters. **Strategy**: a swappable rule (how to price, which car to send) hidden behind an interface. **Factory**: one place that creates objects, so callers need not know how they are built. **Observer**: an object keeps a list of listeners and tells each of them when something changes. **State**: each situation a machine can be in is its own class. **Chain of Responsibility**: a request is passed along a line of handlers, each doing what it can. **Command**: an action wrapped in an object, so it can be run, logged or queued. **Sealed result types**: a Java type whose possible variants form a fixed, known list (for example "served" or "refused"), so the compiler can check that callers handle each one.

What these problems test

These seven questions check three skills:

1. **State machines.** A machine's behaviour depends on what has happened so far: a vending machine must not dispense before it is paid, an ATM must not allow a withdrawal before the PIN is checked, a traffic light must never go from green straight to green on the crossing road. A **state machine** is a

fixed set of situations (states) plus rules for moving between them (transitions); the **State pattern** (Chapter 11) makes each state a class, so every button has a defined answer in every state.
2. **Resource allocation.** A limited set of things (parking spots, notes in a cassette, millilitres of milk, elevator cars, locker doors) handed out correctly: never twice, never below zero, and fairly.
3. **Concurrency.** Many actors at once — several entrance gates, two ATMs on one joint account, ten customers tapping "Latte", a timer thread and an emergency thread (a **thread** is one independent line of execution inside a program; many can run at once and touch the same objects). The design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** The hardest part of each problem is small: one method that picks a spot, makes change, deducts ingredients or picks the next elevator stop. Find it early, name it out loud, and spend your time there.

How each case study is laid out

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → TL;DR. It is also a good order to speak in during an interview.

How to read the course diagrams

The course draws class diagrams in stages: classes from earlier stages in grey, new ones in colour. The legend is the same for all four course case studies in this chapter.

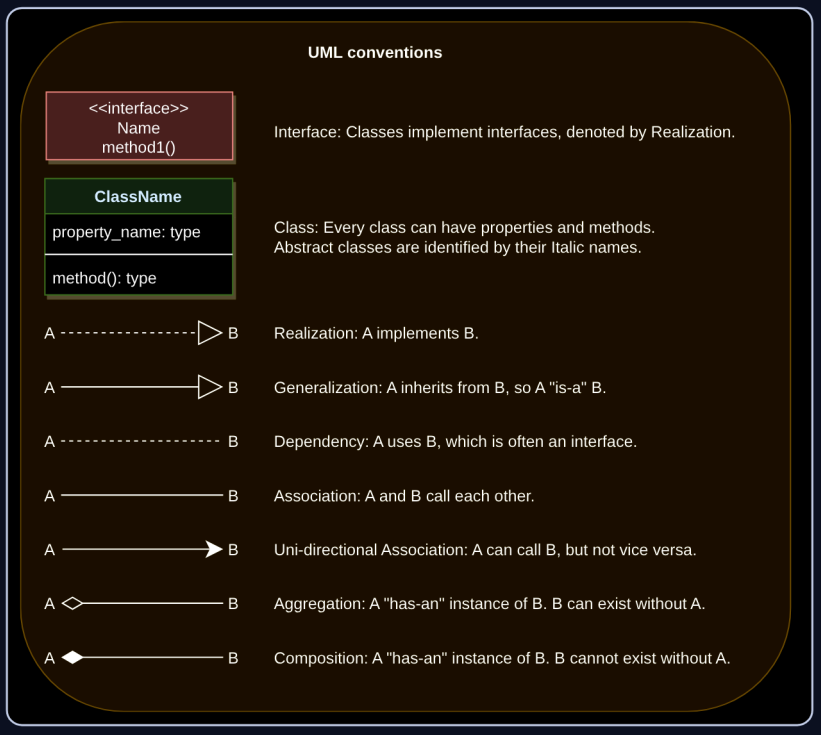


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Hollow triangle = inheritance ("is a"); **dashed line + hollow triangle** = implements an interface (an **interface** lists methods a class promises to provide, without saying how); **filled diamond** = composition (the part dies with the whole); **plain arrow** = association (refers to without owning). Chapter 3 covers these in depth.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator, trafficsignal, locker
```

`javac` compiles `.java` sources to bytecode (`--release 21` targets Java 21, `-d` is the output folder); `java -cp` runs a class from the given **classpath** (where the JVM — the Java Virtual Machine that runs bytecode — looks for classes). All seven `Main` classes compile with Java 21 (standard library only) and run; the output under each implementation is real. Listings below keep the essential classes and methods — the full files (imports, getters, `Main` demos) are in `code/lld/ch14/<case>/`.

> **Code-style note.** A file sometimes holds several small package-private types (declared without `public`, visible only in their package) — e.g. `Vehicles.java` holds `SpotType`, `VehicleType` and `Vehicle`. That is legal Java and compiles with no warnings under `-Xlint:all` (every optional compiler *lint* check) except `auxiliaryclass`, which exists precisely to flag this style. In production each top-level type would get its own file.

14.1 Design a Parking Lot

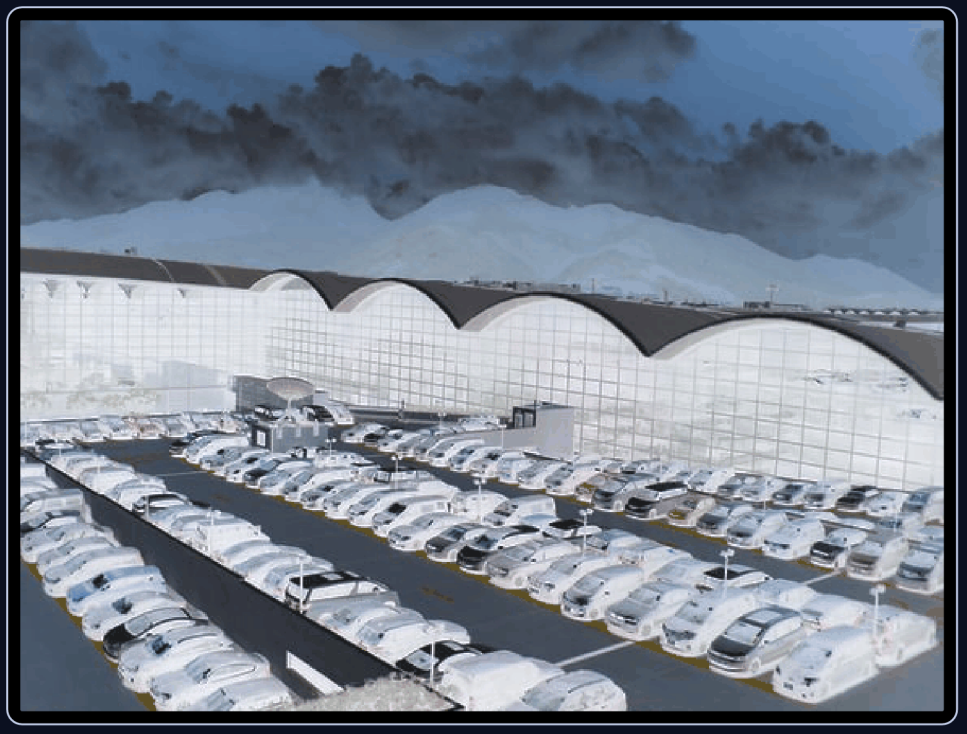


Figure — A large multi-level car park, the kind of building this design models (course diagram)

A **parking lot** (car park) is an area set aside for parking vehicles; malls, stadiums, airports and office towers have large multi-floor lots with gates, ticket machines and boards showing free spaces — many floors, many spot sizes, several ways in and out.

This is the most-asked LLD question (low-level design: the classes, their responsibilities and how they talk to each other). It looks easy, which is why it is useful: the interviewer watches how you **scope** it, how you **model the fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Ask these before drawing anything. The answers in the right column are the ones this design assumes.

Question	Assumed answer
One lot or a chain of lots?	One lot, many floors.
How many entrances and exits?	Several of each, all working at the same time.
Which vehicle types?	Car, truck, van, motorcycle, electric car.
Which spot types?	Compact, large, handicapped (accessible), motorcycle, electric.
How do customers pay?	Cash or card, at the exit panel, to an attendant, or earlier at an info portal on each floor.
How is the fee computed?	Per hour, with tiers: $4 for the first hour, $3.50 for hours 2 and 3, $2.50 for each hour after. A started hour counts as a full hour.
Do EVs charge?	Yes: electric spots have a panel where the driver pays for charging.
Reservations, monthly passes, number-plate recognition?	Out of scope (mentioned as extensions).
What happens when the lot is full?	The entrance panel and the ground-floor board show a "full" message; no more vehicles are admitted.

Requirements

Functional

1. The lot has **multiple floors**.
2. It has **multiple entry and exit points**.
3. A customer **collects a ticket at the entrance** and **pays at the exit**.
4. Payment at the **automated exit panel** or to a **parking attendant**.
5. Payment by **cash or credit card**.
6. Payment is also possible at a **customer info portal on each floor**; a ticket paid there is not charged again at the exit.
7. The lot **never admits more vehicles than its capacity**. When full, a message shows on the entrance panel and on the ground-floor display board.
8. Each floor has many spots of several **types**: compact, large, handicapped, motorcycle (and electric, below).
9. Some spots are for **electric cars**; each has an **electric panel** where the customer can pay for and charge the vehicle.
10. Many **vehicle types**: car, truck, van, motorcycle (and electric car).
11. Each floor has a **display board** showing free spots per spot type.
12. A **per-hour fee model** with tiers ($4 / $3.50 / $3.50 / $2.50 …).

Non-functional

- **Correctness under concurrency:** two entrance gates working at the same instant must never be given the same spot, and a ticket must never be paid twice.
- **Low latency at the gate:** finding a spot must not scan thousands of spots; taking one from a sorted set of free spots is O(log n). (**Latency** is how long one request takes from start to finish; see System Design Fundamentals. **O(log n)** is Big-O notation for cost: the work grows with the logarithm of the number of spots *n*, so even 10,000 spots need only about 14 steps.)
- **Extensibility:** a new pricing rule, a new spot-finding rule or a new vehicle type should be a small, local change.
- **Testability** (how easily automated tests can check the code)**:** time must be injectable (passed in from outside rather than read inside the code, here as a fake clock), so fee tests do not wait for hours.

Use cases

Actors (an actor is anyone or anything outside the system that uses it):

- **Admin** — adds, removes and edits floors, spots, entrance/exit panels; adds and removes attendants; sets the hourly rate.
- **Customer** — takes a ticket, pays, leaves.
- **Parking attendant** — can do everything a customer can, on the customer's behalf, and can accept cash.
- **System** — shows messages on panels and boards, assigns a vehicle to a spot and frees it again.

Top use cases: add/remove/edit a floor (each floor with its own display board); add/remove/edit a spot; add/remove an attendant; **take ticket**; **scan ticket** (to find the total charge); **credit-card payment**; **cash payment**; add/modify the parking rate.

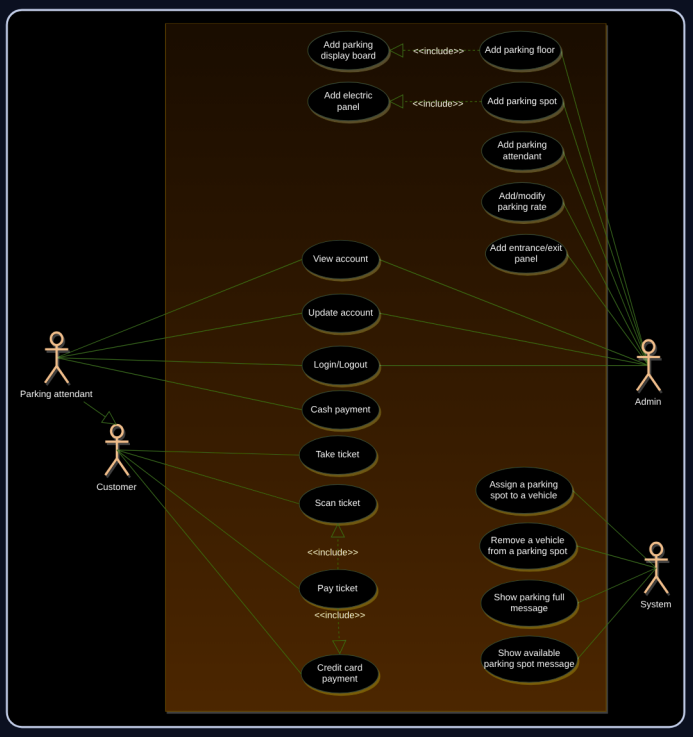


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

In the figure above, notice that the attendant's use cases overlap the customer's: the attendant is a customer "proxy" who can also take cash. The System actor is what assigns spots and updates the boards.

Core entities

Class	Responsibility
`VehicleType` (enum)	The kinds of vehicle and, for each, the spot types it fits in, **best fit first**.
`SpotType` (enum)	The kinds of spot: motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit. Knows its preferred spot types.
`ParkingSpot`	One physical spot: id, floor, type, and who occupies it (atomic).
`ChargingPanel`	The pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Holds the **free** spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Shows free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot (nearest floor first, least busy floor first, …).
`PricingStrategy`	*How much* a stay costs (hourly tiers, flat rate, weekend rate, …).
`Ticket`	Proof of entry: vehicle, the actual spot, entry time, status (ACTIVE → PAID → EXITED).
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	The devices customers touch. They hold **no rules**; they call the lot.

In the table, an **enum** is a Java type with a fixed list of named constants (such as `SpotType.COMPACT`), and a **record** is a short Java class that only carries a few values and cannot be changed after it is created. **Thread-safe** means an object stays correct even when several threads use it at the same moment.

The course also models **`Account`** with **`Admin`** and **`ParkingAttendant`** subclasses (both sign in). They carry no parking logic, so they are omitted from the code below; the attendant simply uses the same `pay` method as an exit panel.

Class diagram

The course grows the design in five steps. Every step adds a handful of classes and leaves you with a feature that runs. Use the same order in an interview: if the clock stops you early, what you have drawn already works, instead of being an unfinished picture.

Stage	What it adds	What works after it
1	Vehicle and spot types	The system can answer "may this vehicle use this spot?"
2	Floors and display boards	A floor can hand out a free spot and report counts
3	Ticket, rate card, payment	A stay can be turned into a price and settled
4	The `ParkingLot`	A full visit works: enter, park, pay, leave
5	Entrance/exit panels, info portal, electric panel	Drivers use the flow through physical devices

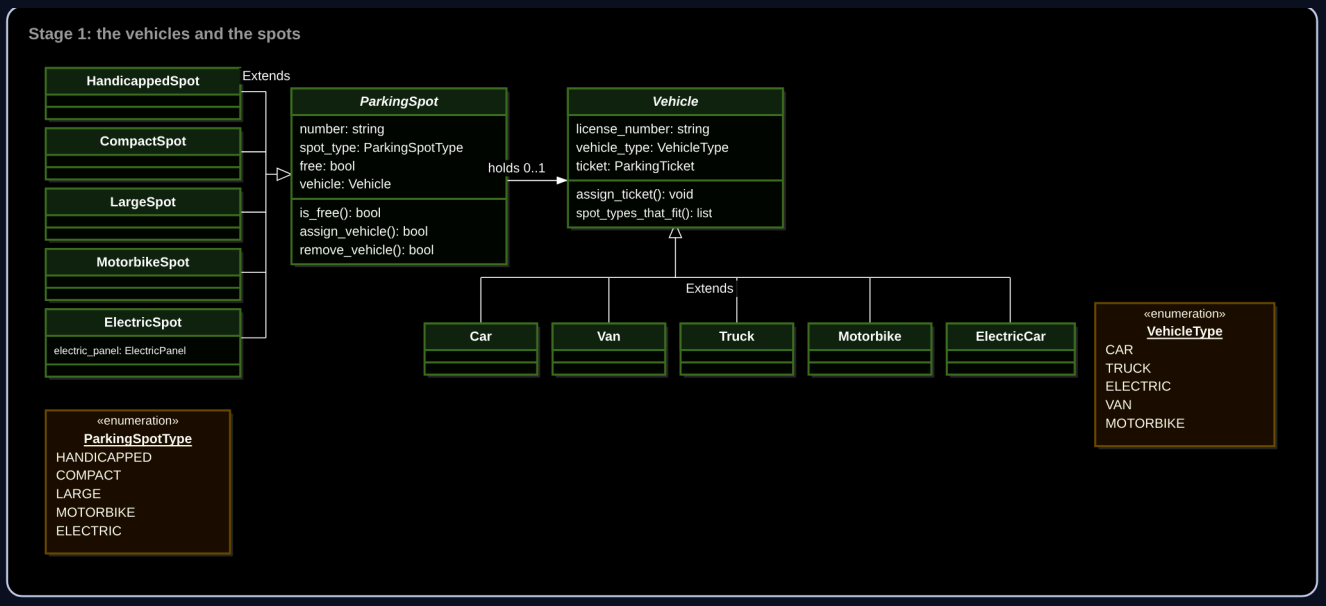


Figure — Stage 1: the vehicle and spot hierarchies, and the "holds 0..1" link from a spot to a vehicle (course diagram)

Stage 1 — vehicles and spots. Requirements 8 and 10: two small families of types, and a spot holds **at most one** vehicle (`0..1`). The most important rule — *which spot types can this vehicle use?* — lives on the **vehicle** side, so a new kind of vehicle changes only the vehicle types, never the lot. Fit rules: a motorcycle uses a motorcycle spot, trucks and vans only large spots, and an electric car tries electric, then compact, then large.

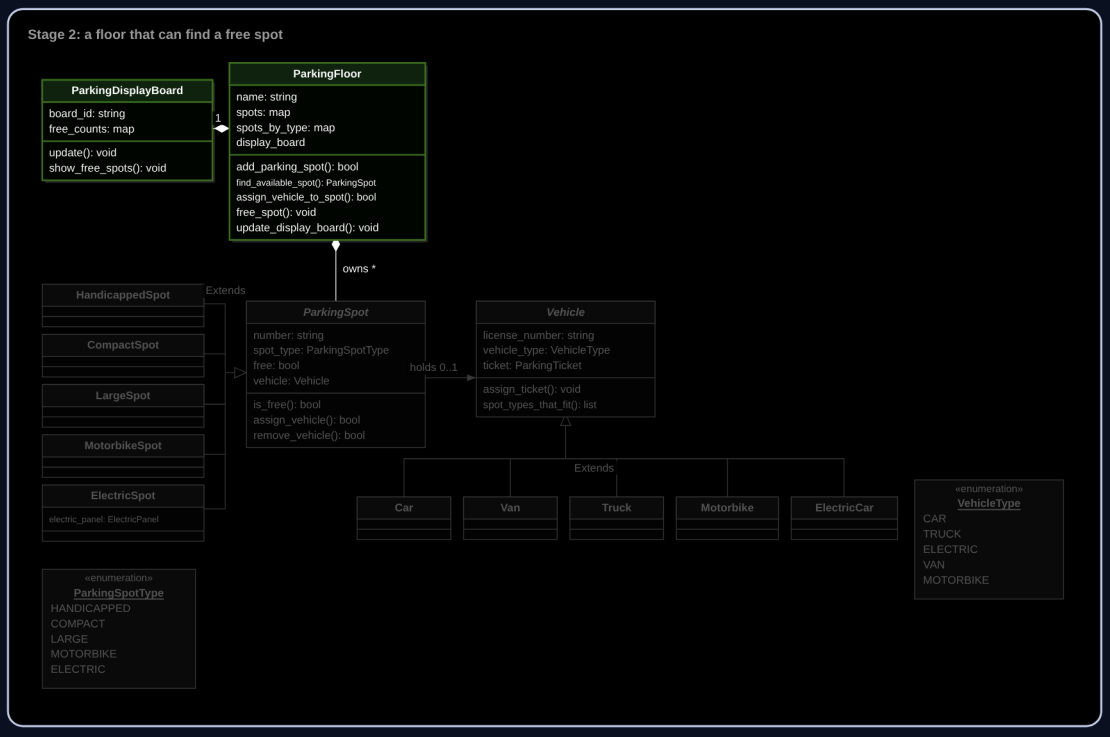


Figure — Stage 2: a floor owns its spots and its display board (composition) (course diagram)

Stage 2 — a floor that can find a free spot. Requirements 1, 8 and 11. Both links are **composition** (filled diamond): deleting a floor deletes its spots and board. The board stores **how many** spots of each type are free, not a pointer to "a sample free spot", which goes stale the moment that spot is taken. The course's floor keeps a map of all spots plus an index by type (our code keeps one map from spot

type to its free spots). Rejected: one separate collection per type — every method would need a five-way `if`, and a sixth type would touch all of them.

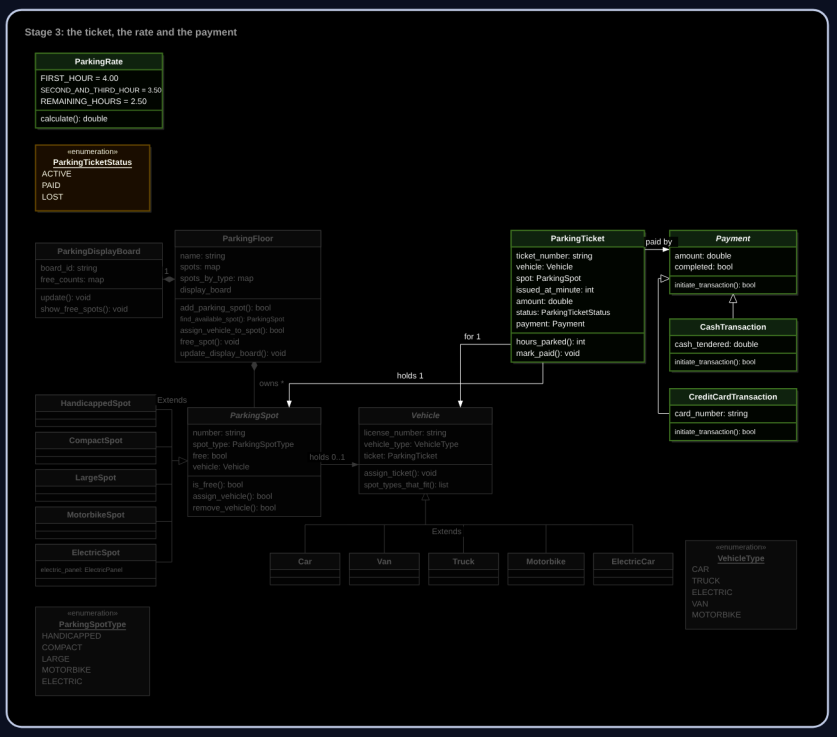


Figure — Stage 3: ticket, parking rate and the abstract payment with cash and card subclasses (course diagram)

Stage 3 — ticket, rate, payment. Requirements 3, 5 and 12. Key decision: **the ticket stores a reference to the exact spot**, not just "one more spot is taken" — at the exit, that is how the system knows which spot to free. The ticket only points at vehicle and spot (plain associations). `Payment` is abstract (cannot be created directly) with cash and card subclasses — justified because either can stand in wherever "a payment" is expected (Liskov Substitution, Chapter 4).

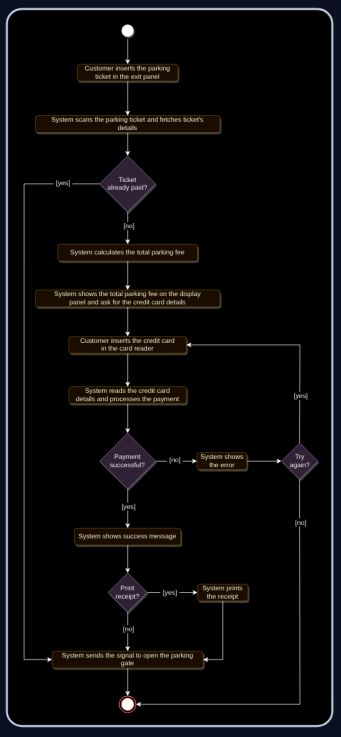


Figure — Activity diagram: the customer inserts the ticket at the exit, the system computes the fee, the customer pays by card or cash, and the barrier opens (course diagram)

The activity diagram above is the exit flow; the ticket, rate and payment classes between them handle each of its steps.

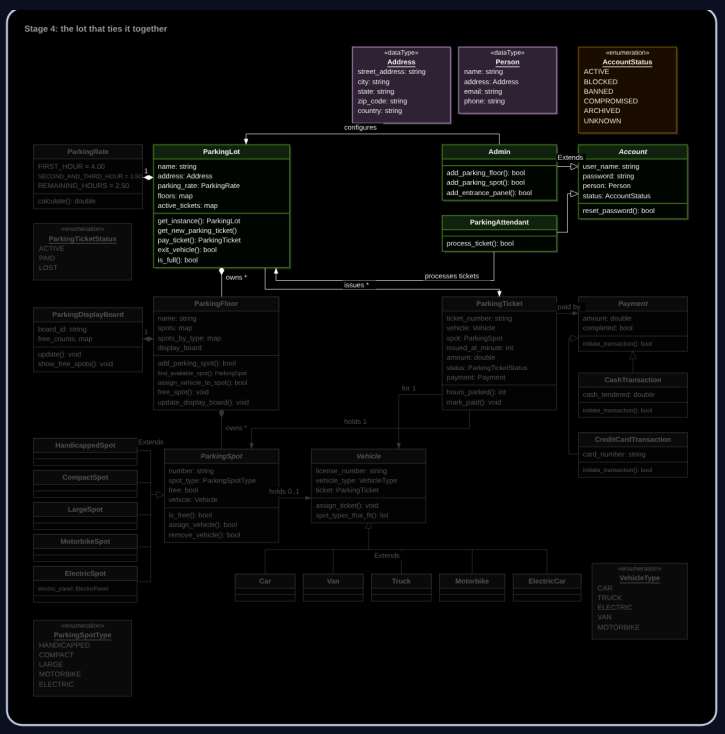


Figure — Stage 4: the parking lot owns floors and the rate; Admin and ParkingAttendant accounts connect to it (course diagram)

Stage 4 — the lot. Requirement 7 and the actors. The lot has a **name** and **address** and owns its floors and rate card (filled diamonds). `Admin` and `ParkingAttendant` are plain arrows: they work *on* the lot without owning any part of it. The course makes the lot a **Singleton** (only one instance may exist) and serialises spot assignment (**one caller at a time**), since two entrances assigning in parallel could pick the same spot.

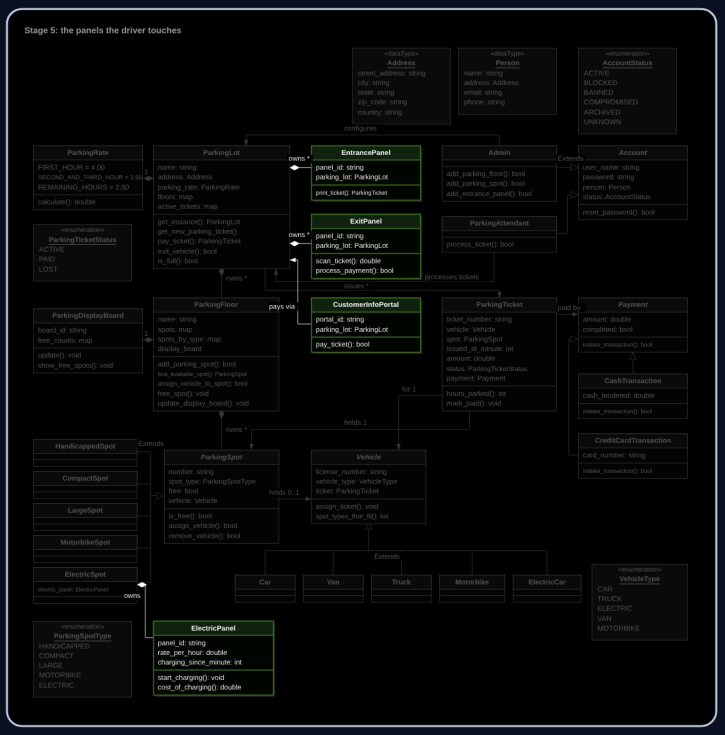


Figure — Stage 5: entrance and exit panels (owned by the lot), the customer info portal (refers to the lot) and the electric panel (owned by an electric spot) (course diagram)

Stage 5 — the panels. Requirements 2, 4, 6 and 9. Panels are thin — they forward every request to the lot, so all rules sit in one class. The entrance panel prints tickets, the exit panel scans and takes payment, the info portal settles early (marking the ticket paid), the electric panel bills

charging. The lot owns its entrance/exit panels (composition) but keeps no list of portals (a portal just refers to the lot); the electric panel is owned by its electric spot.

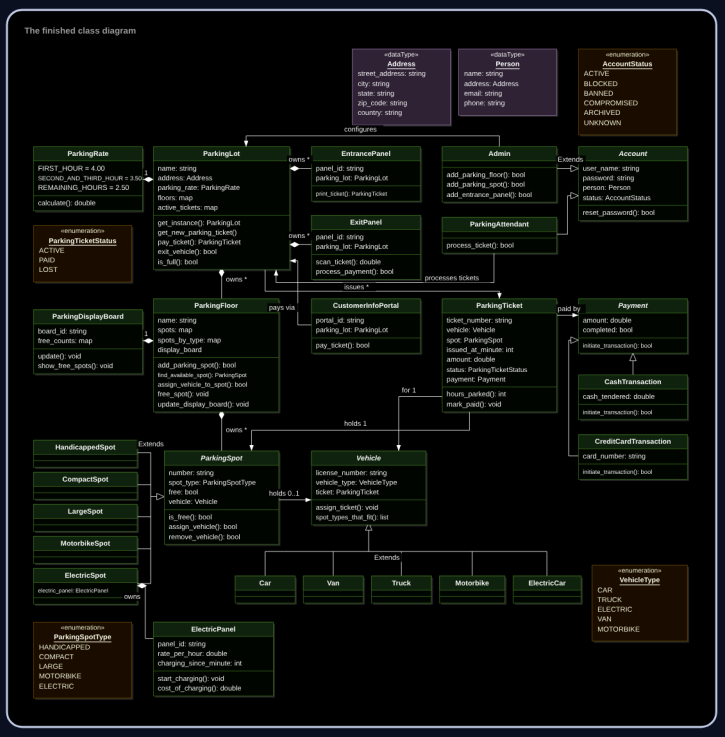
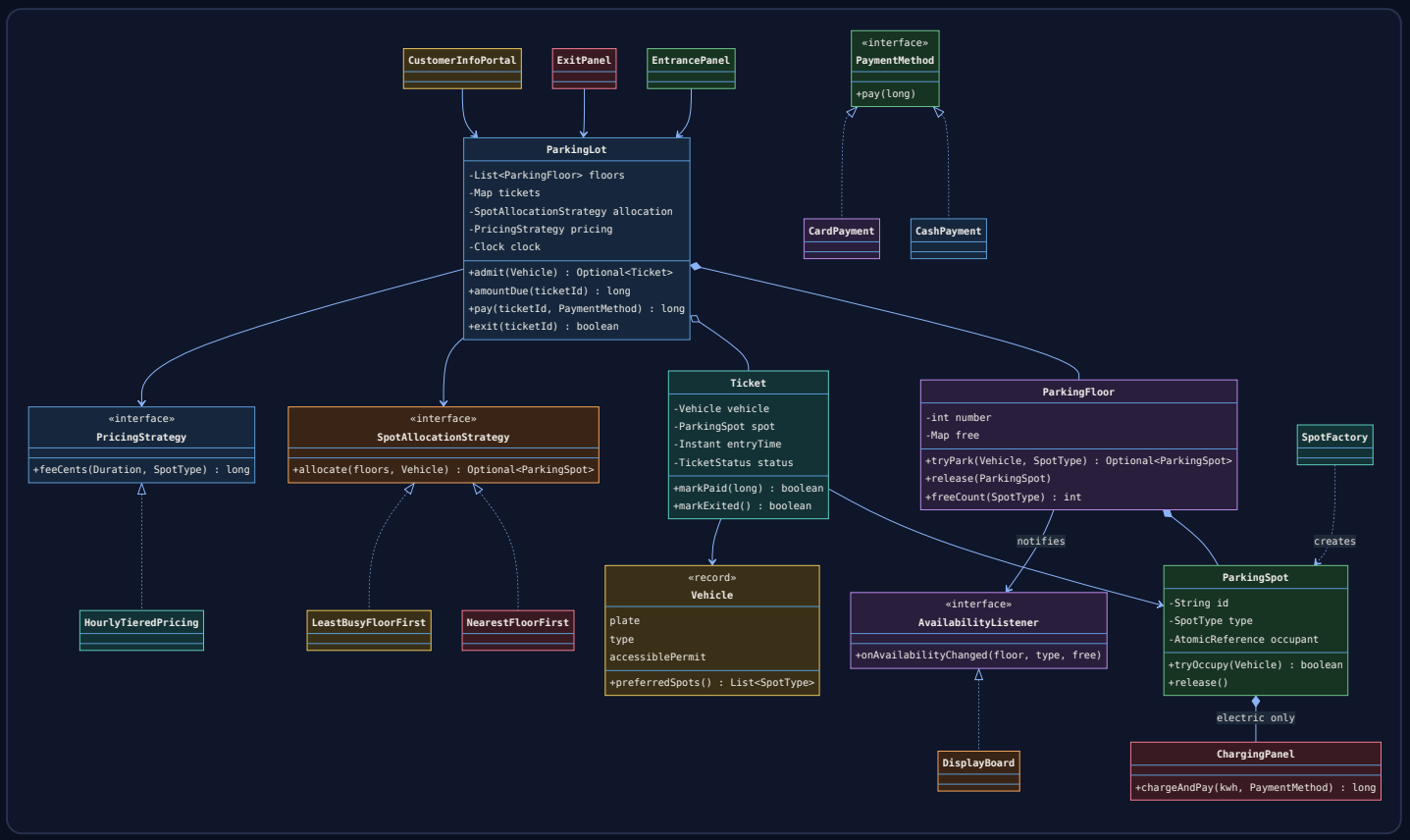


Figure — The finished class diagram of the parking lot (course diagram)

The finished diagram above is simply all five stages in colour; nothing new appears in it. Here is the class diagram of **the Java code in this chapter**. It keeps the same shape but makes three changes that interviewers usually push for: spot types are an enum instead of five empty subclasses, pricing and spot-finding are Strategies, and the display board is an Observer.

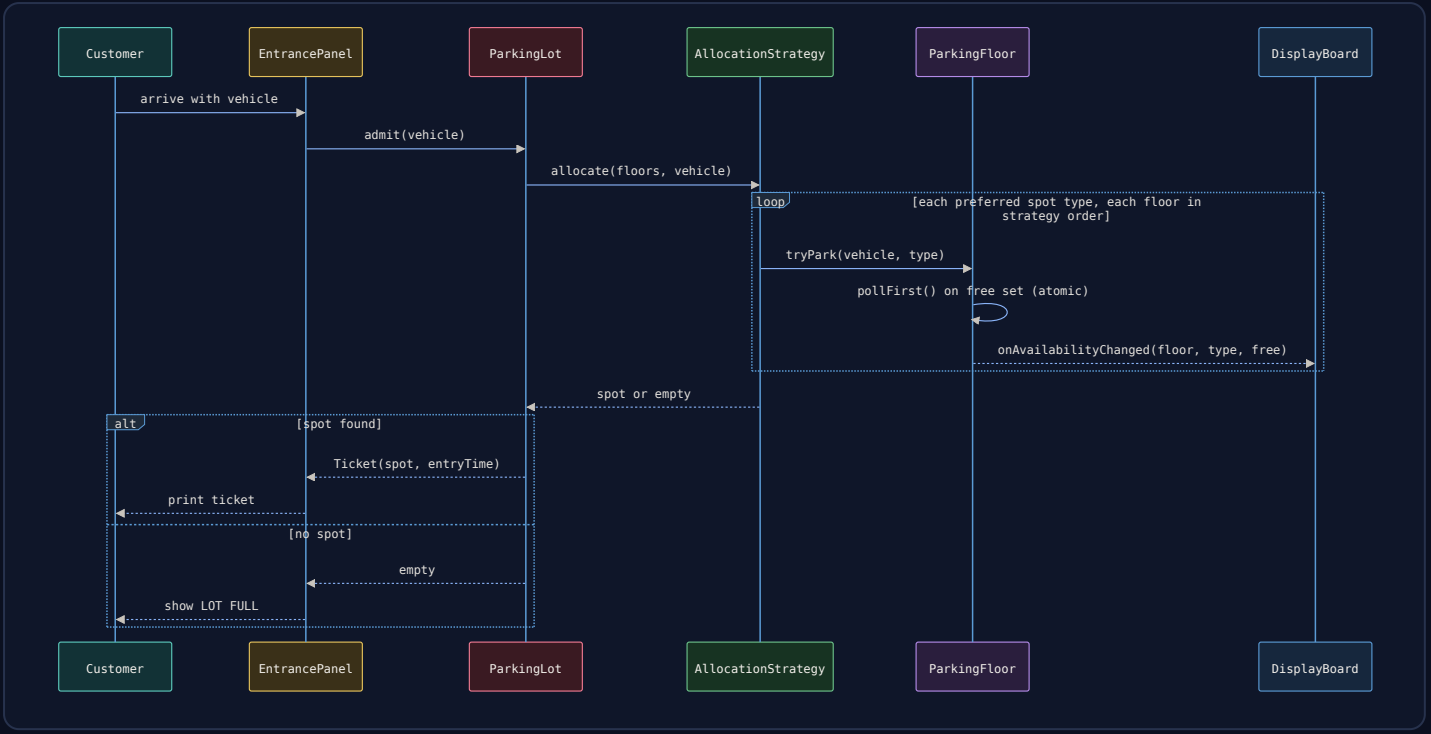


In this Mermaid diagram (Mermaid is a text format that renders as a diagram), `-` marks a private field (hidden inside its class), `+` a public method, `~T~` a generic type such as `List<ParkingFloor>`, `*--`

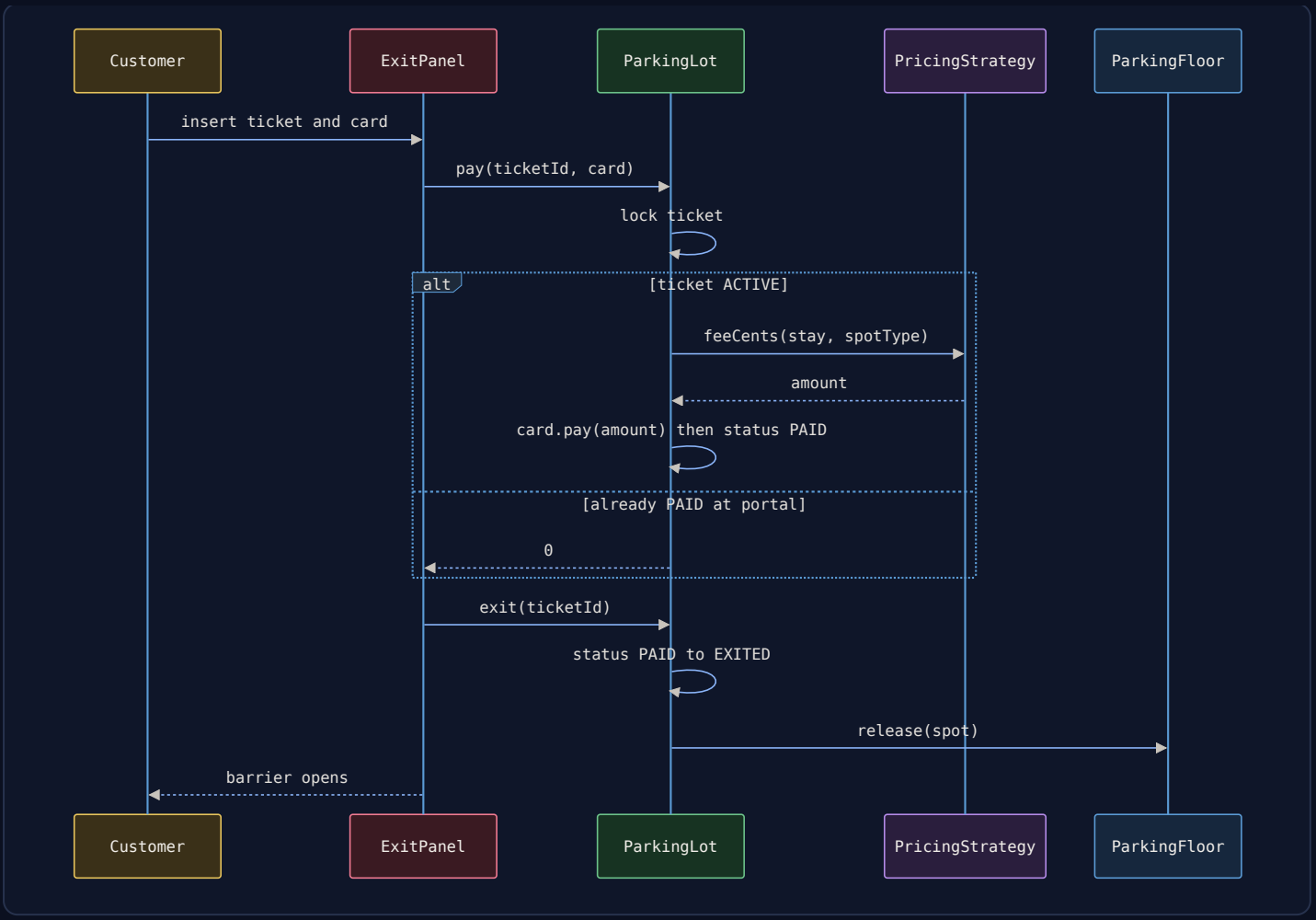
composition, `o--` aggregation (a looser "has a" where the parts can live on their own), `-->` association, `<|..` "implements", and `..>` "uses or creates".

Key flows

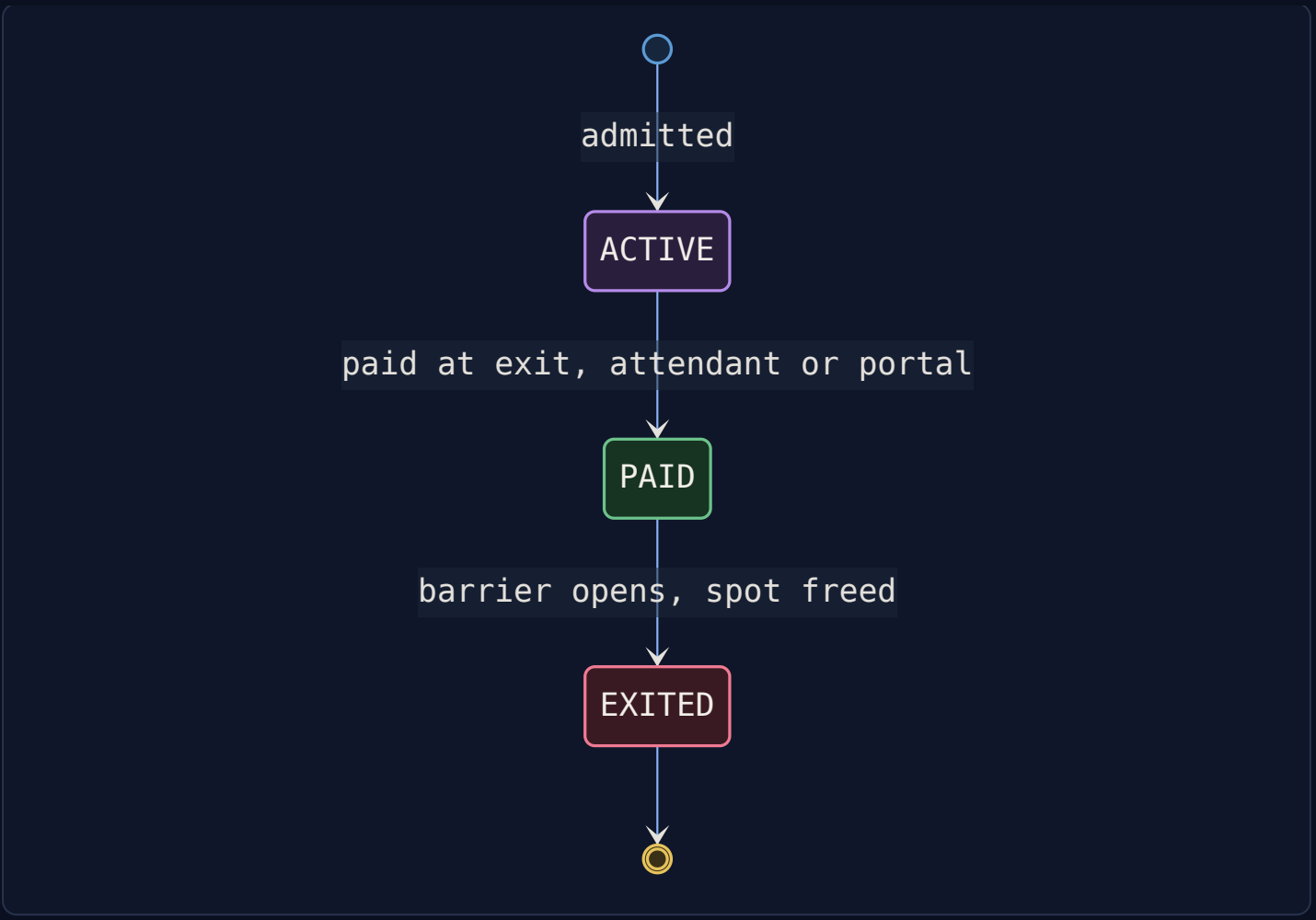
Entry (take ticket). The important step is that the spot is taken **atomically** (as one indivisible step that no other thread can interrupt halfway): `pollFirst()` on a concurrent sorted set removes the spot for exactly one caller. The **sequence diagram** below shows the messages between objects in time order, from top to bottom.



Exit (scan, pay, leave). Payment is guarded per ticket so the info portal and the exit panel cannot both charge.



Ticket life cycle.


admitted
ACTIVE
paid at exit, attendant or portal
PAID
barrier opens, spot freed
EXITED

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
The fit rule lives on `VehicleType`	Information Expert (give a job to the class that owns the data), Open/Closed (extend without editing)	Adding a vehicle type is one enum constant; the lot never changes	A big `switch(vehicleType)` inside the lot
Spot type is an **enum field**, not 5 subclasses	Composition over inheritance (hold a field instead of subclassing)	The 5 course subclasses carried no behaviour; a new type is one constant	`CompactSpot`, `LargeSpot` … (reviewers flag these as empty classes)
How to pick a spot is a `SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy floor, closest to the lift, VIP floors… all swap without touching the lot	`if (mode == NEAREST) … else …` in the lot
How much to charge is a `PricingStrategy`	**Strategy**	The course's "what if pricing changes?" follow-up; admin swaps the rate at run time (the demo swaps to a flat rate)	Hard-coded tiers inside `Ticket`
Electric spots get a `ChargingPanel` in `SpotFactory`	**Factory** (simple static factory)	One place knows the construction rule "electric ⇒ has a panel"	Every caller remembers to attach a panel
Boards subscribe to floors	**Observer**	A floor change updates every board (and could update an app or a sensor light) without the floor knowing who listens	The lot pushing into boards directly
Ticket holds the **spot**, not a count	—	The exit can find and free the exact spot	Counting occupied spots
`ParkingLot` is **not** a hard-coded Singleton	Dependency injection (pass collaborators in from outside)	The course uses a Singleton ("there is one car park"). It is a global in disguise: tests cannot build two lots, and every class that calls `getInstance()` hides a dependency. We create one lot in `main` and pass it to the panels.	`ParkingLot.getInstance()`
`Clock` is injected	Testability	The demo moves time forward by hours instantly	`Instant.now()` inside the code

Honest note on "Strategy". The course points out that a cash/card subclass pair is just an abstract base class, and calling it "Strategy" would be loose use of the word. Strategy means an object **holds** a behaviour and can **swap** it. In our code `ParkingLot` holds a `PricingStrategy` and a `SpotAllocationStrategy` and `setPricing(...)` swaps it at run time — that is genuinely Strategy.

The course also admits that `ParkingLot` does two jobs (finding spots and issuing tickets). Here spot-finding has moved into the strategy and the floors, so the lot is mostly a coordinator; in a longer design you would also move ticket storage into a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are in **cents** (`long`) to avoid floating-point rounding (a `double` stores numbers in binary and cannot hold 0.10 exactly, so sums drift). Essential lines:

```
// Vehicles.java - the fit rule lives on the vehicle type, best fit first
enum SpotType { MOTORCYCLE, COMPACT, LARGE, HANDICAPPED, ELECTRIC }

enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));
    private final List<SpotType> fits;
    VehicleType(List<SpotType> fits) { this.fits = fits; }
    List<SpotType> fits() { return fits; }
}

record Vehicle(String plate, VehicleType type, boolean accessiblePermit) {
    List<SpotType> preferredSpots() {                    // a permit puts HANDICAPPED first; nobody else gets it
        List<SpotType> order = new ArrayList<>();
        if (accessiblePermit && type != VehicleType.MOTORCYCLE && type != VehicleType.TRUCK)
            order.add(SpotType.HANDICAPPED);
        order.addAll(type.fits());
        return order;
    }
}
```

```
// ParkingSpot.java - occupancy is an AtomicReference: CAS succeeds only if the spot was empty
class ParkingSpot implements Comparable<ParkingSpot> {
    private final String id;                             // e.g. "F1-C03"; lowest id = nearest
    private final SpotType type;
    private final AtomicReference<Vehicle> occupant = new AtomicReference<>();
    private final ChargingPanel chargingPanel;           // only electric spots have one (composition)
    boolean tryOccupy(Vehicle v) { return occupant.compareAndSet(null, v); }
    void release() { occupant.set(null); }
    @Override public int compareTo(ParkingSpot o) { return id.compareTo(o.id); }
}

class ChargingPanel {                                    // pay-and-charge at an electric spot
    long chargeAndPay(double kwh, PaymentMethod method) {
        long cost = Math.round(kwh * centsPerKwh);
        method.pay(cost);
        return cost;
    }
}

final class SpotFactory {                                // the only place that knows "electric => has a panel"
    static ParkingSpot create(int floor, SpotType type, int number) {
        String id = "F%d-%s%02d".formatted(floor, type.name().charAt(0), number);
        ChargingPanel panel = type == SpotType.ELECTRIC ? new ChargingPanel(EV_CENTS_PER_KWH) : null;   // 30
c/kWh
        return new ParkingSpot(id, floor, type, panel);
    }
}
```

```java
// ParkingFloor.java - one sorted, thread-safe set of FREE spots per type; boards are observers
interface AvailabilityListener { void onAvailabilityChanged(int floor, SpotType type, int freeNow); }

class ParkingFloor {
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    private final List<AvailabilityListener> listeners = new CopyOnWriteArrayList<>();

    /** pollFirst() is atomic: two threads can never receive the same spot. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }
    void release(ParkingSpot spot) {
        spot.release();
        free.get(spot.type()).add(spot);
        notifyListeners(spot.type());
    }
    int freeCount(SpotType t) { return free.get(t).size(); }
}
// DisplayBoard implements AvailabilityListener: synchronized map type -> free count, renders 0 as "FULL"
```

```java
// Strategies.java
interface SpotAllocationStrategy { Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v); }

class NearestFloorFirst implements SpotAllocationStrategy {      // fill the lowest floor first
    @Override public Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v) {
        for (SpotType type : v.preferredSpots())
            for (ParkingFloor f : floors) {
                Optional<ParkingSpot> s = f.tryPark(v, type);
                if (s.isPresent()) return s;
            }
        return Optional.empty();
    }
}
// LeastBusyFloorFirst: same loop, floors sorted by freeCount(type) descending (losing a race -> next floor)

interface PricingStrategy { long feeCents(Duration stay, SpotType spotType); }

class HourlyTieredPricing implements PricingStrategy {           // $4, $3.50, $3.50, then $2.50/hour
    static HourlyTieredPricing standard() { return new HourlyTieredPricing(List.of(400L, 350L, 350L), 250); }
    @Override public long feeCents(Duration stay, SpotType spotType) {
        long minutes = Math.max(1, stay.toMinutes());
        long hours = (minutes + 59) / 60;                         // a started hour counts in full
        long total = 0;
        for (long h = 0; h < hours; h++) total += h < firstHours.size() ? firstHours.get((int) h) :
laterHourCents;
        return total;
    }
}
```

```
// Ticketing.java - the ticket points at the real spot so the exit can free it
enum TicketStatus { ACTIVE, PAID, EXITED }

final class Ticket {
    private final ParkingSpot spot;
    private final Instant entryTime;
    private TicketStatus status = TicketStatus.ACTIVE;
    synchronized boolean markPaid(long cents) {                  // only one payment can ever succeed
        if (status != TicketStatus.ACTIVE) return false;
        status = TicketStatus.PAID; paidCents = cents; return true;
    }
    synchronized boolean markExited() {
        if (status != TicketStatus.PAID) return false;
        status = TicketStatus.EXITED; return true;
    }
}
interface PaymentMethod { void pay(long cents); String describe(); }
// record CashPayment(long tenderedCents): throws "not enough cash"; record CardPayment(String last4)
```

```
// ParkingLot.java - NOT a Singleton: created once in main and injected into the panels
final class ParkingLot {
    private final Map<String, Ticket> tickets = new ConcurrentHashMap<>();
    private final Map<String, String> ticketByPlate = new ConcurrentHashMap<>();
    private volatile SpotAllocationStrategy allocation;
    private volatile PricingStrategy pricing;

    Optional<Ticket> admit(Vehicle v) {
        if (ticketByPlate.putIfAbsent(v.plate(), "pending") != null)       // same car cannot enter twice
            throw new IllegalStateException(v.plate() + " is already inside");
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) { ticketByPlate.remove(v.plate()); return Optional.empty(); }   // full for this
vehicle
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }
    long amountDue(String ticketId) {
        Ticket t = ticket(ticketId);
        if (t.status() != TicketStatus.ACTIVE) return 0;                // already paid (e.g. at the portal)
        return pricing.feeCents(Duration.between(t.entryTime(), clock.instant()), t.spot().type());
    }
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                      // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);                    // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
        }
    }
    boolean exit(String ticketId) {             // only a PAID ticket opens the barrier
        Ticket t = ticket(ticketId);
        if (!t.markExited()) return false;
        floors.get(t.spot().floor() - 1).release(t.spot());
        tickets.remove(ticketId);
        ticketByPlate.remove(t.vehicle().plate());
        return true;
    }
    void setPricing(PricingStrategy p) { this.pricing = p; }          // admin changes the rate at run time
}
// Panels.java: EntrancePanel.issueTicket, ExitPanel.scanPayAndExit (pay then exit), CustomerInfoPortal.pay -
// no rules, they only call the lot. Main.java: a TestClock (a movable java.time.Clock), then 20 threads
// released by a CountDownLatch race for the last 5 car spots.
```

A `ConcurrentSkipListSet` is a thread-safe set from `java.util.concurrent` that keeps its elements sorted; its `pollFirst()` removes the lowest id **atomically**, so if 20 threads call it at once each spot goes to exactly one of them. The lowest id stands for "nearest to the entrance". `tryOccupy` is a second safety

net: **CAS** (compare-and-set) — "set to *v* only if still *null*", one atomic step without a lock. `putIfAbsent` adds a key only if absent, atomically. `pay` locks **one ticket**, not the lot (a **lock** lets one thread at a time run a piece of code; `synchronized` is the built-in lock every object carries), so different tickets pay in parallel.

Output:

```
== Arrivals ==
[entry-A] CAR-1 -> ticket T1, spot F1-C01(COMPACT)
[entry-A] EV-1 -> ticket T2, spot F1-E01(ELECTRIC)
[entry-A] BIKE-1 -> ticket T3, spot F1-M01(MOTORCYCLE)
[entry-A] CAR-2 -> ticket T4, spot F1-H01(HANDICAPPED)
[entry-A] TRUCK-1 -> ticket T5, spot F1-L01(LARGE)
[entry-A] VAN-1 -> ticket T6, spot F2-L01(LARGE)
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
Floor 1 board: MOTORCYCLE=FULL COMPACT=1 LARGE=FULL HANDICAPPED=FULL ELECTRIC=FULL
Floor 2 board: COMPACT=3 LARGE=FULL

== EV charging at the spot ==
EV-1 charged 20 kWh, paid $6.00

== Departures ==
[exit-A] T1: paid $4.00 by cash $5.00, barrier OPEN
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false

== Admin changes the rate: flat $5/hour ==
[exit-A] T3: paid $25.00 by card ****9999, barrier OPEN

== 20 cars race for the remaining CAR spots ==
free compact before: F1=2 F2=3, large: F1=0 F2=0
cars admitted: 5, distinct spots: 5
full for cars now? true
[entry-A] LATE-CAR -> SORRY, LOT FULL for CAR
Floor 2 board: COMPACT=FULL LARGE=FULL
```

Read the output: the truck is turned away (both large spots taken); T2 stayed 4 h 50 min → 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**, paid at the portal, so the exit charges nothing; an unpaid ticket cannot open the barrier; after the switch to a flat $5/hour the motorcycle pays $25 for 5 hours; 20 racing cars get exactly 5 distinct spots.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Two gates want the last compact spot	Both get it (double booking)	`pollFirst()` is atomic; `tryOccupy` uses CAS as a second check
Portal and exit panel pay the same ticket at once	Customer charged twice	`synchronized (ticket)` around price-charge-mark; a PAID ticket costs 0
Same plate enters twice (cloned ticket, sensor glitch)	Two tickets for one car	`ticketByPlate.putIfAbsent`
Payment fails (card declined)	Ticket marked paid anyway	`method.pay` throws **before** `markPaid` ; ticket stays ACTIVE
Car leaves without paying	Spot never freed or fee lost	`exit` only succeeds from PAID; the barrier stays closed
Customer pays at the portal and then stays 3 more hours	Free parking	Real lots give a **grace period** (e.g. 15 min) after payment; add `paidAt` and re-price after the grace period
Display board shows a slightly old number	Two notifications arrive out of order	Acceptable for a display; if exactness matters, have the board **pull** `freeCount` when it renders
Lost ticket	No entry time	Charge a daily maximum, or look up the plate (the plate map already exists)
Stay of 0 minutes	Fee of $0	Minimum of one hour (`Math.max(1, minutes)`)
Lock scope	One global lock would serialise all gates	Locks are per spot (CAS) and per ticket; no global lock

`ConcurrentSkipListSet.size()` is not constant-time (it walks the whole set instead of reading a stored number) and is only an estimate while other threads change the set; that is fine for a display board, but never use it to decide whether a spot is free — use `pollFirst()` .

Extensions & follow-up questions

1. **"What if pricing changes (weekend rate, per spot type, surge)?"** — Write a new `PricingStrategy` (for example `WeekendPricing` , or a decorator (a wrapper object with the same interface that adds behaviour around another object) that multiplies another strategy for large spots) and call `setPricing` . No other class changes.
2. **"Two cars arrive at the same instant — what happens?"** — Spot handout is atomic (`pollFirst` + CAS). Show the 20-thread race in the output.
3. **"Add reservations."** — Add a `Reservation(plate, spotType, from, to)` and a reserved set per floor; the allocation strategy checks the reservation first. Hold the spot with a timeout (release it if the car does not arrive within, say, 30 minutes).
4. **"Add monthly passes / season tickets."** — A `PricingStrategy` that returns 0 for plates with a valid pass, or a `PassHolderPricing` decorator around the normal pricing.
5. **"Number-plate recognition (ticketless)."** — The entrance camera becomes another kind of entrance panel that calls the same `admit` ; the ticket id is keyed by plate. No core class changes.
6. **"How would you store this in a database?"** — Tables `floor` , `spot(id, floor_id, type, occupied_by)` , `ticket(id, plate, spot_id, entry_time, status, paid_cents)` , `payment(id, ticket_id, method, amount)` . Claim a spot with `UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL` — the database's CAS. Candidates report that interviewers like to see **cardinalities** (how many of one thing link to another; `1..*` = one or more: a floor has 1..* spots, a spot holds 0..1 vehicles, a ticket has 1 spot), then add a new entity to see how you rewire the model.
7. **"Many lots in a city?"** — A `ParkingLot` per site plus a `ParkingNetwork` that routes drivers to the lot with free spots. This is another reason not to make the lot a Singleton.

8. **"EV idle fees?"** — Add a charging session to `ChargingPanel` (start, end, kWh) and charge a per-minute fee after charging completes.

Real-world example

Many modern malls and airports use **parking guidance systems**: a sensor above each bay drives a small green/red light, and the boards at each entrance and floor show the free count per level — exactly our `DisplayBoard` observers fed by spot changes. Airports and many city garages increasingly use **automatic number-plate recognition** instead of paper tickets, and apps such as SpotHero and ParkMobile let drivers pre-book a spot, which is the "reservations" extension. Tesla Superchargers charge **idle fees** when a car stays plugged in after charging finishes — the EV extension above.

In the interview

Plan for 45 minutes

Minutes	What you do
0–5	Clarifying questions; write the 12 requirements and an out-of-scope list
5–10	Actors and use cases (take ticket, pay, exit, admin)
10–20	Stages 1–2: vehicle/spot fit rule, floors, board. Say "the fit rule lives on the vehicle"
20–25	Stage 3–4: ticket holds the spot; lot with admit/pay/exit; Strategy for pricing
25–38	Code `admit`, `tryPark`, `pay`, `exit`, and the tiered pricing
38–45	Concurrency (atomic spot handout, per-ticket lock), extensions

30-second answer: "I'll model vehicles and spot types as enums with the fit rule on the vehicle, floors that keep a concurrent sorted set of free spots per type, and a lot that admits, prices and releases. Tickets point at the real spot. Pricing and spot-finding are Strategies so they can change without touching the lot. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates can never get the same spot, and payment locks only the one ticket."

Follow-ups with answers

- *"Why not a Singleton?"* — There is one lot in production, but a Singleton is a hidden global: it makes testing two lots impossible. I create one instance and inject it. If the interviewer insists, a lazy holder-class Singleton (the single instance lives in a nested class that Java loads only on first use, which the JVM makes thread-safe for free) is thread-safe and simple.
- *"Why is the fit rule on the vehicle?"* — Because it changes when vehicle types change; putting it there keeps the lot closed for modification (OCP).
- *"How do you make finding a spot fast?"* — Keep only the free spots, indexed by type, in a sorted set; taking one is O(log n) and needs no scan.
- *"What if the display board and the real count disagree?"* — The board is informational; the atomic `pollFirst` is the source of truth. The board can pull the count on render if needed.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket** → **spot** reference makes the exit possible; status goes ACTIVE → PAID → EXITED.
- **Strategy** for pricing and spot-finding, **Factory** for electric spots, **Observer** for boards; inject the lot instead of a Singleton.
- Concurrency: atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine



Figure — A vending machine: slots with codes, a keypad, a coin/note slot and a tray at the bottom (course diagram)

A **vending machine** is an unattended shop: key in a slot code, pay, pull your item from the tray. Everyone has used one, so the interviewer skips the domain and looks at one thing: **which actions are legal at which moment**. Taking a coin when nothing is chosen is a bug; two items for one payment is a bug; tidy code elsewhere does not make up for getting that order wrong.

Problem & clarifying questions

Question	Assumed answer
Select first, or pay first?	Select a slot code first, then pay.
Which money?	A fixed set of coins and notes (5¢, 10¢, 25¢, $1, $5). Anything else is returned.
Card or contactless?	Out of scope (an extension).
Can it give change?	Yes. If it cannot make **exact** change, it refuses the sale and returns everything.
Can the customer cancel?	Any time before the product is dispensed, with a full refund.
One customer at a time?	Yes (one physical machine).
Who restocks?	An operator: restocks slots, loads coins for change, collects the takings.

Requirements

Functional (the course's twelve)

1. The machine has many **slots**. A slot has a code (`A1` , `B2` , …), holds items of a **single** product, and tracks its remaining **count**.
2. A product has a **name** and a **price in cents**.
3. Order of use: **choose a slot, then pay**.
4. Only a **known set of coins and notes** is accepted; any other piece is given back.
5. After **every** customer action the **display** says what to do next.
6. When the amount paid covers the price, the machine **releases the item and the change**.
7. When **exact change is impossible**, the sale is **refused** and every cent paid is **returned**.

8. Until the item drops, the customer may **cancel** and receive a **full refund**.
9. A slot whose count is **zero cannot be chosen**.
10. One payment buys **exactly one** item, never two.
11. An **operator** refills slots, adds coins for change, and removes the takings.
12. After every sale, cancel and refusal, **both** the item counts and the cash counts must be **exactly right**.

Non-functional

- **No illegal transitions**: every button must have a defined answer in every situation.
- **Exact money**: integer cents only — no floating-point.
- **Extensible**: a new payment device (card reader) or a new state (maintenance) should not require editing every existing class.

Use cases

Actors: Customer (selects, pays, cancels, takes the product), **Operator** (restocks, loads change, collects money), **System** (decides which actions are allowed now, works out change, shows messages).

Top use cases: select a product · insert money · dispense · collect (take from the tray) · cancel · restock a slot · load change and collect takings.

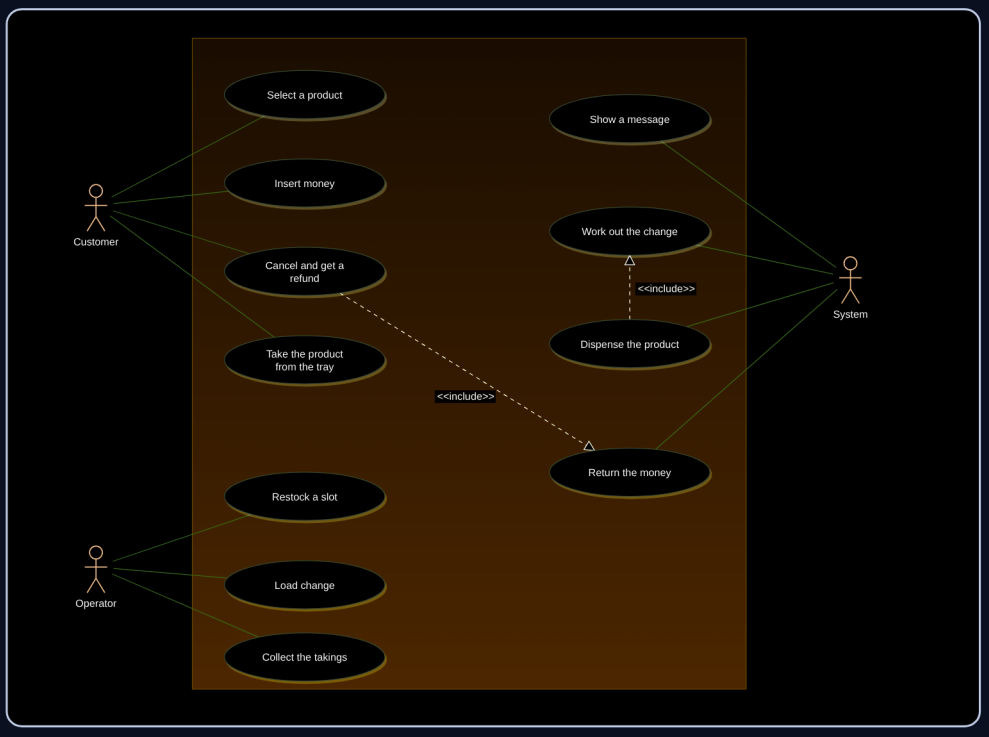


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

In the figure above, "dispense" is a System use case, not a customer one: the customer only *causes* it by paying enough.

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents. Describes *what* is sold.
`Slot`	One physical column: a code, one product, and the **count** left.
`Inventory`	All slots, looked up by code.
`Denomination` (enum)	The accepted coins and notes and their values in cents.
`CashBox`	Counts pieces by value; plans exact change; reports when it cannot.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations the machine can be in (the course has the first four, calling `HasMoneyState` `PaidState`); each answers every button and names the next state.
`Display`, `Keypad`, `CashSlot`, `Dispenser`, `Operator`	The message shown, the hardware the customer touches (calls the machine, holds no rules) and the operator (restocks, loads change, collects takings). Our code folds them into `VendingMachine`: `show(...)`, the public button methods, `restock`, `loadChange`, `collectTakings`.
`VendingMachine`	Holds the parts, the current state, the selection and the balance; forwards each button to the state; exposes operator actions.

Why is the count on the **slot**, not the product? `Product` answers "what is it?"; `Slot` is the physical column. The same product may sit in two columns (the course even allows different prices), and emptying one must leave the other untouched — the first real design decision in this question.

Class diagram

The course builds the machine in five stages:

Stage	What it adds	What works after it
1	`Product`, `Slot`, `Inventory`	Look up a code and learn what it holds and whether it is empty
2	`Coin`, `Note`, `CashBox`	Decide whether a given amount of change can be paid exactly
3	A class for each state	Each button press gets a defined reaction in each situation
4	`Display` and `VendingMachine`	A complete sale, from choosing to picking up the item
5	`Keypad`, `CashSlot`, `Dispenser`, `Operator`	Real hardware drives the same flow

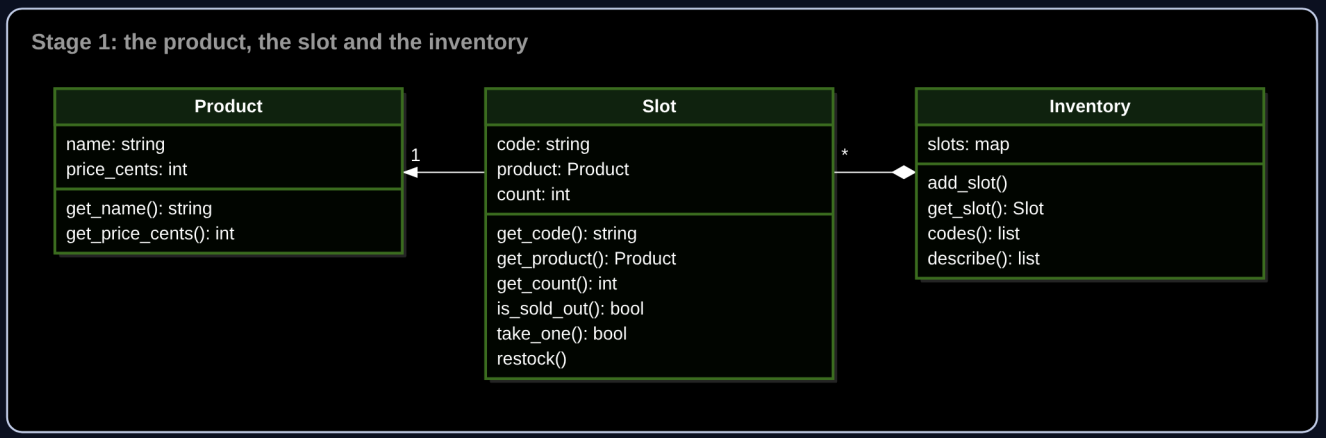


Figure — Stage 1: Product, Slot (association to Product) and Inventory (composition of Slots) (course diagram)

Stage 1. Requirements 1, 2 and 9. `Slot → Product` is an **association**: a product can appear in several slots, so removing a slot must leave the product alone. `Inventory ◆ Slot` is **composition** (filled diamond): slots are parts of the machine and do not outlive it.

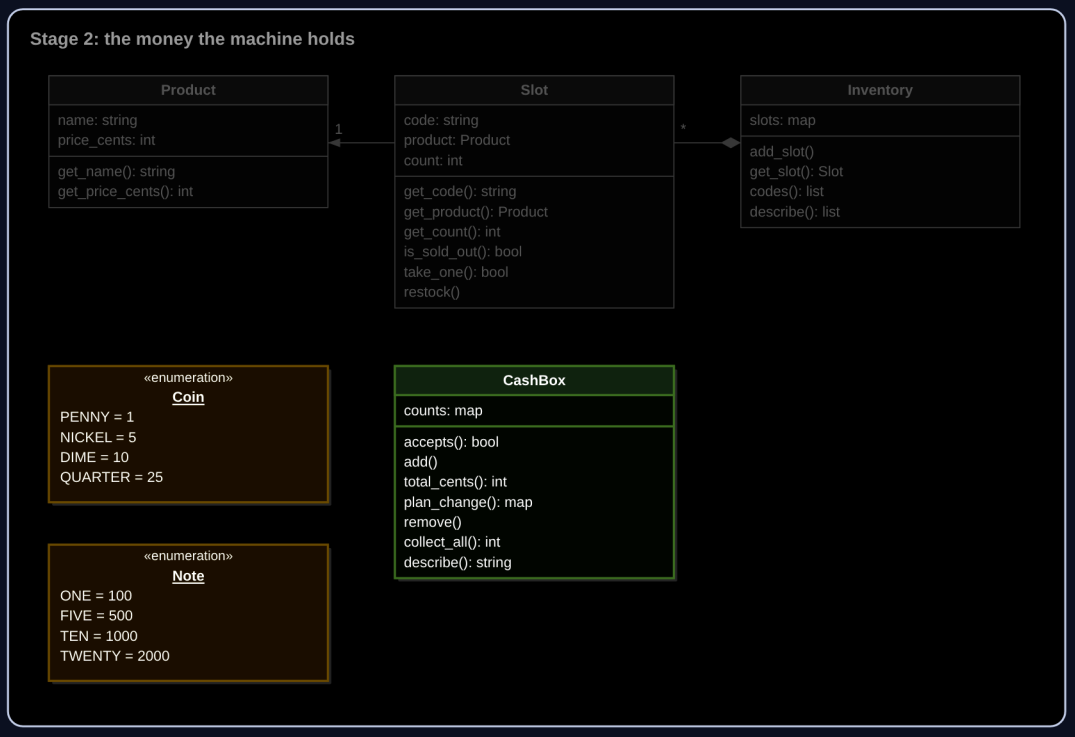


Figure — Stage 2: Coin and Note enumerations and the CashBox that counts pieces by value (course diagram)

Stage 2. Requirements 4, 6 and 7. All amounts are whole cents, so no rounding errors. `Coin` and `Note` are enumerations (fixed sets with no relationship arrows; our code merges them into one `Denomination` enum). The choice to defend: **a piece goes into the cash box the moment it is inserted**, not into a holding area — it is then available for change, and a refund is just "pay out exact change equal to what was inserted". The box stores a **count per value** (which physical quarter comes back doesn't matter); its change planner returns an exact plan or "impossible", never an approximation — that is what makes requirement 7 implementable.

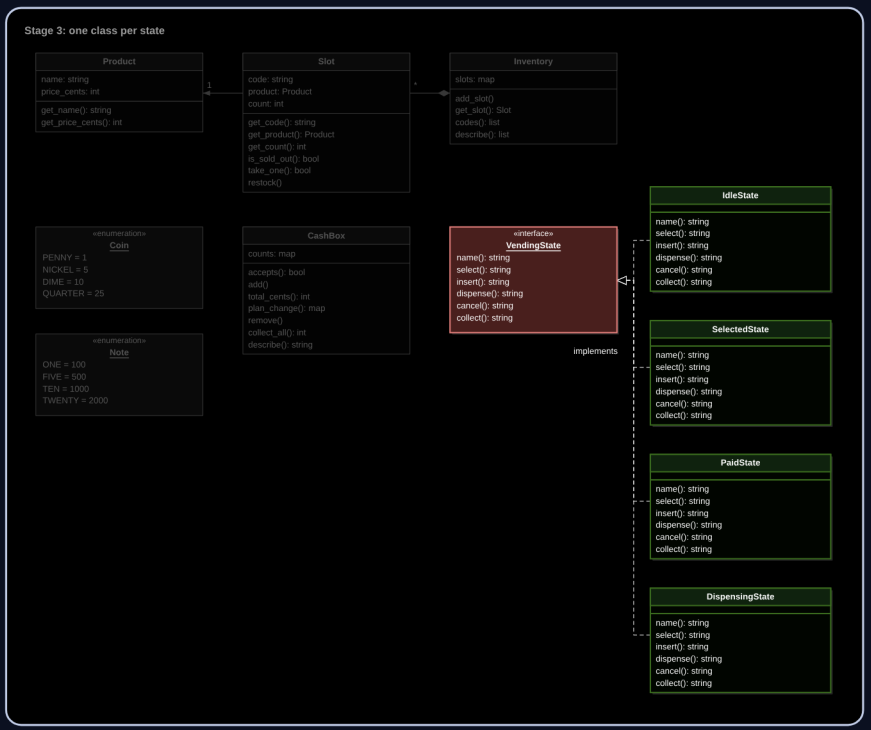


Figure — Stage 3: one class per state, all implementing the VendingState interface (course diagram)

Stage 3 — the heart of the question. Requirements 3, 8 and 10. The obvious first attempt — one class with boolean flags and nested `if`s — is exactly where double dispensing and swallowed coins come from. Use the **State pattern**: each situation is a class implementing one interface on which **each customer button is a method**, so a state that forgets a button does not compile. Picture a **grid** of buttons × situations: the design is finished when every cell has a response. State classes carry no fields and never check which state is active; each handles its buttons and hands over to the next, so a new state needs no edits to existing ones.

The course draws **four** states: `IdleState`, `SelectedState`, `PaidState` and `DispensingState`. Our code keeps those four (its `HasMoneyState` is the course's `PaidState`: some money is in, not yet enough) and adds a fifth, `OutOfStockState`, so that an empty machine refuses every customer button until an operator restocks it.

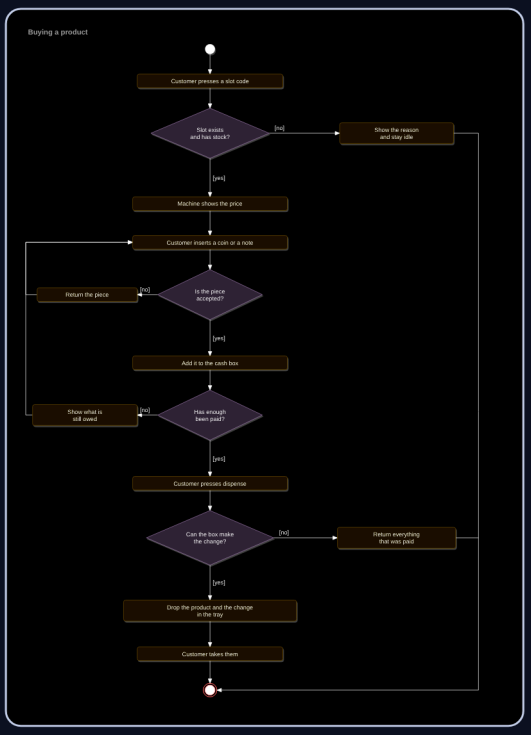


Figure — Activity diagram "Buying a product": select, check stock, insert money until enough, check change, dispense, take (course diagram)

Each decision diamond in the activity diagram above corresponds to one state choosing whether to accept an action.

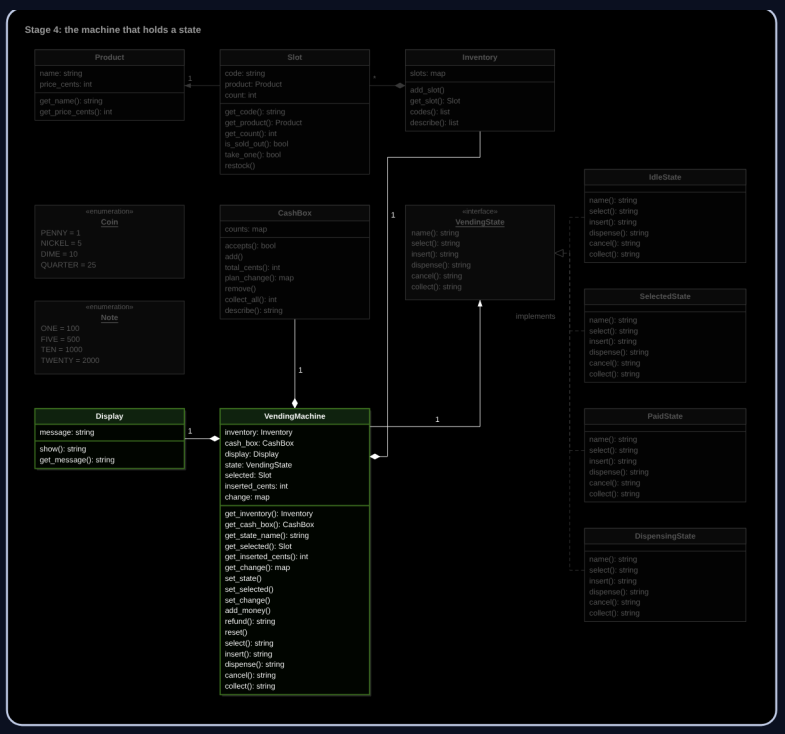


Figure — Stage 4: VendingMachine owns its display, cash box and inventory, and points at the current VendingState (course diagram)

Stage 4. Requirements 5, 6, 8 and 12. `VendingMachine` remembers the chosen slot and the money inserted, and passes every button to its current state. It **owns** its `Display`, `CashBox` and `Inventory` (composition) and merely **refers to** the current state (association). It contains no `if` testing which state is active. Because inserted money is already in the box (stage 2), a refund is just a request for exact change equal to the balance.

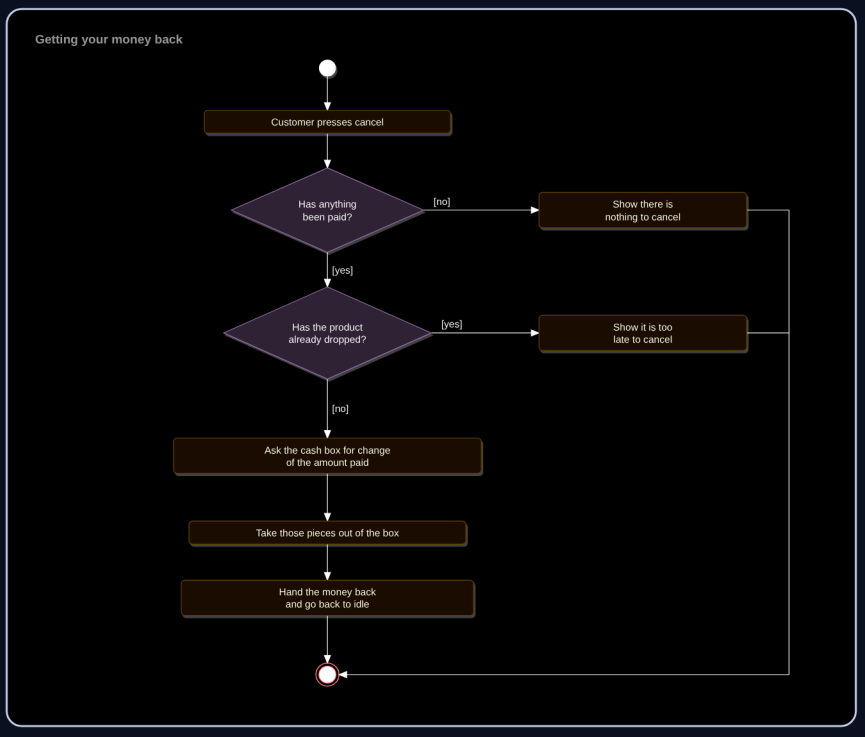


Figure — Activity diagram "Getting your money back": cancel and "cannot make change" end in the same refund path (course diagram)

As the figure above shows, a cancel and a "no exact change" refusal finish identically, so the design needs a **single** refund method.

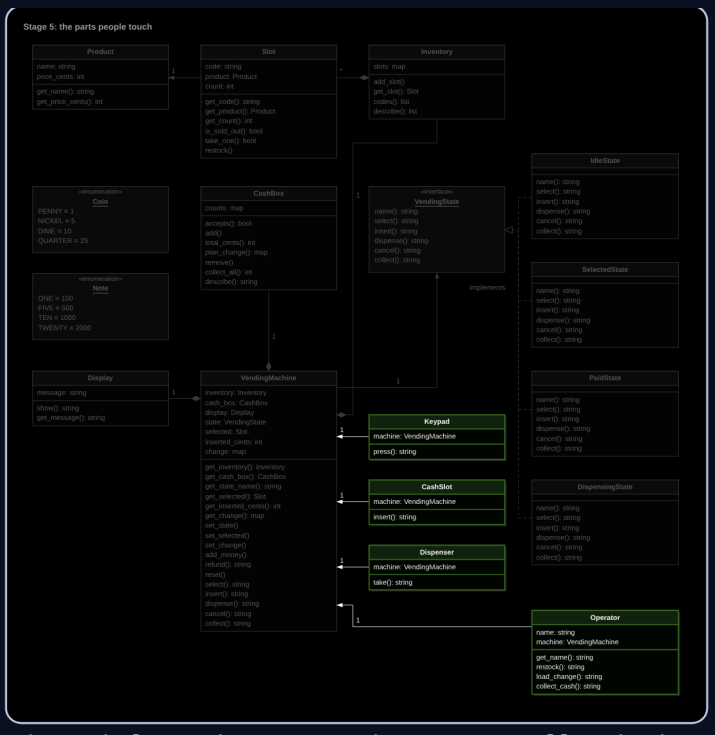


Figure — Stage 5: Keypad, CashSlot, Dispenser and Operator, all pointing at the machine (course diagram)

Stage 5. Requirements 3, 4 and 11. All four new classes hold references **to** the machine, never the reverse, so swapping the keypad for a touch screen or the coin slot for a card reader touches no other class. The `Dispenser` is the tray; pulling the item out finishes a sale, which is why `take` is a customer button. `Operator` stays **off** the state interface: refilling, loading change and emptying the box are not customer actions and are allowed in any state (otherwise every state class would have to answer them).

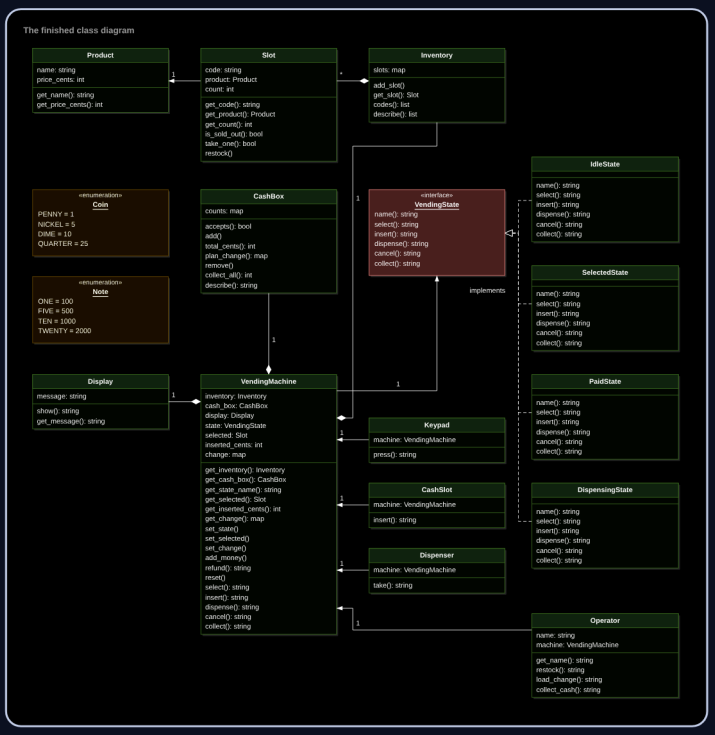
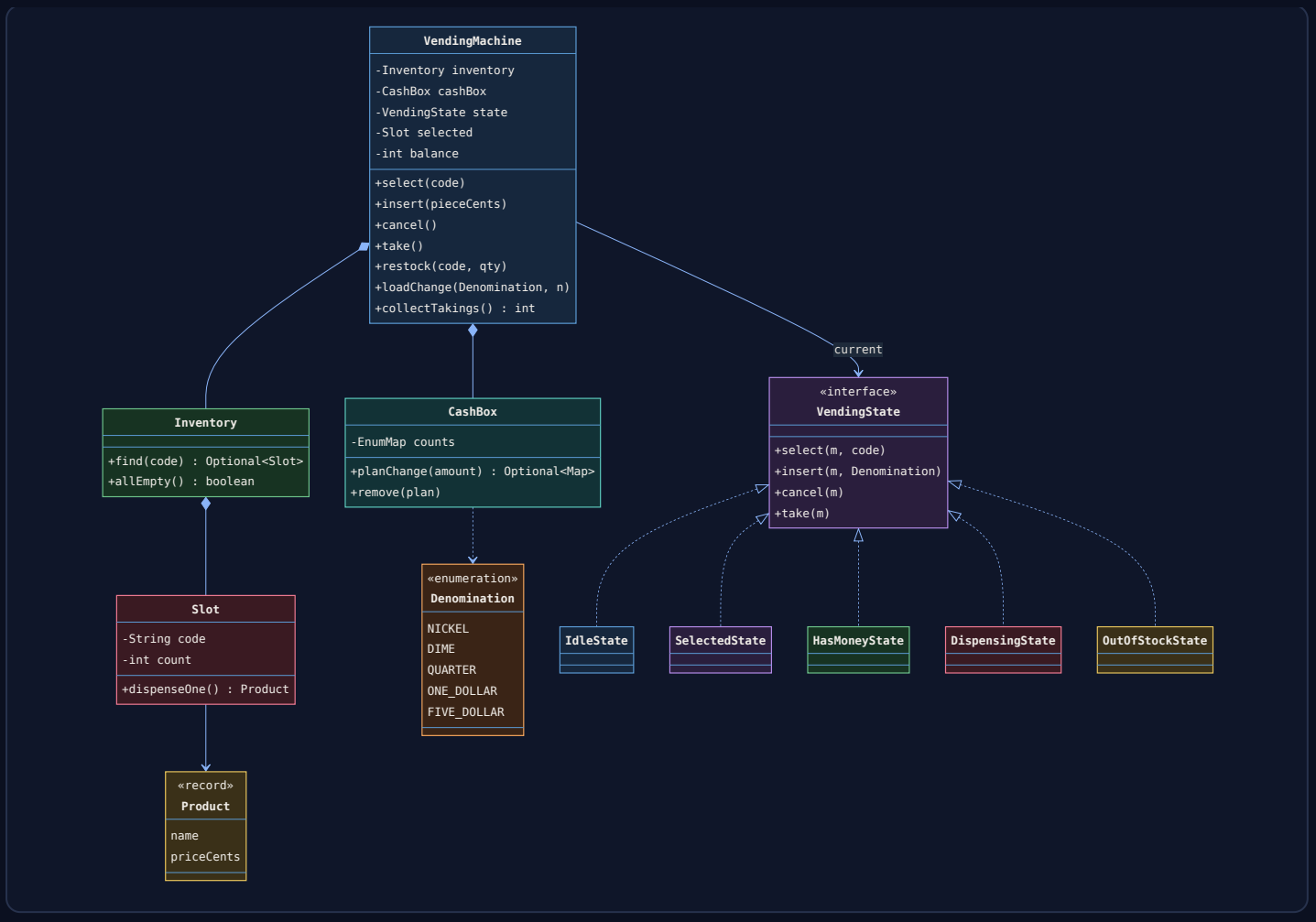


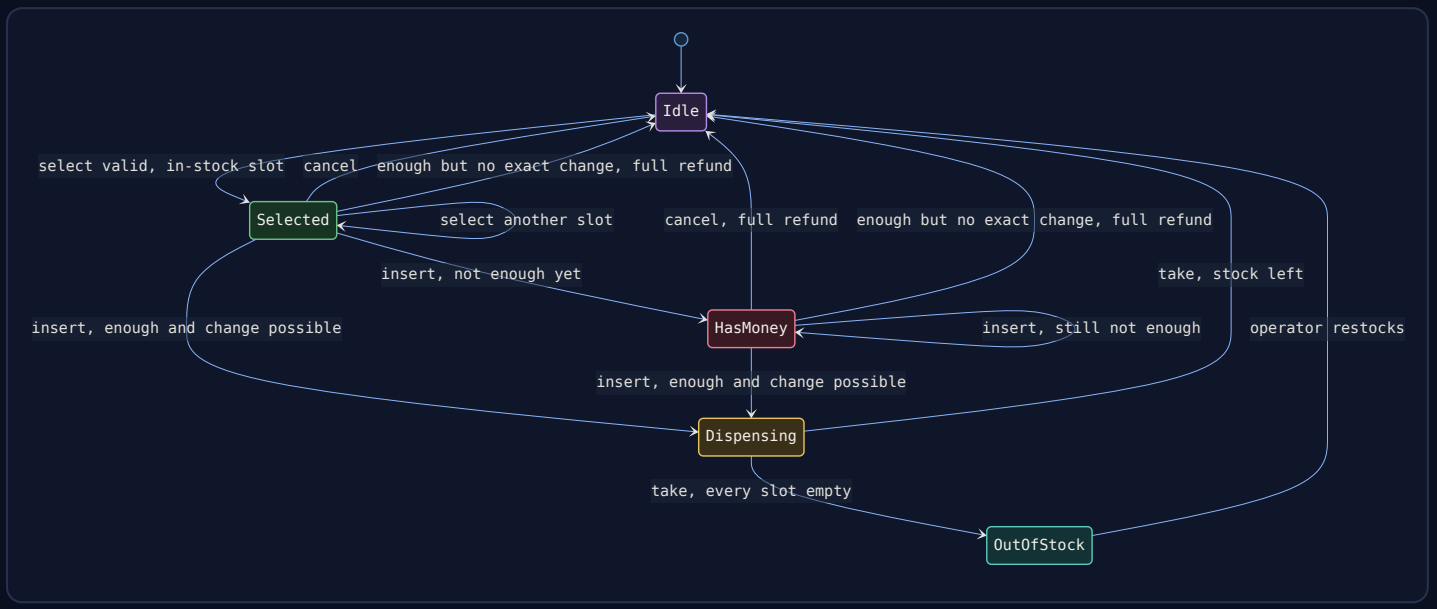
Figure — The finished class diagram of the vending machine (course diagram)

Class diagram of **our Java code** (hardware classes folded into the machine's public methods to keep the code short):



Key flows

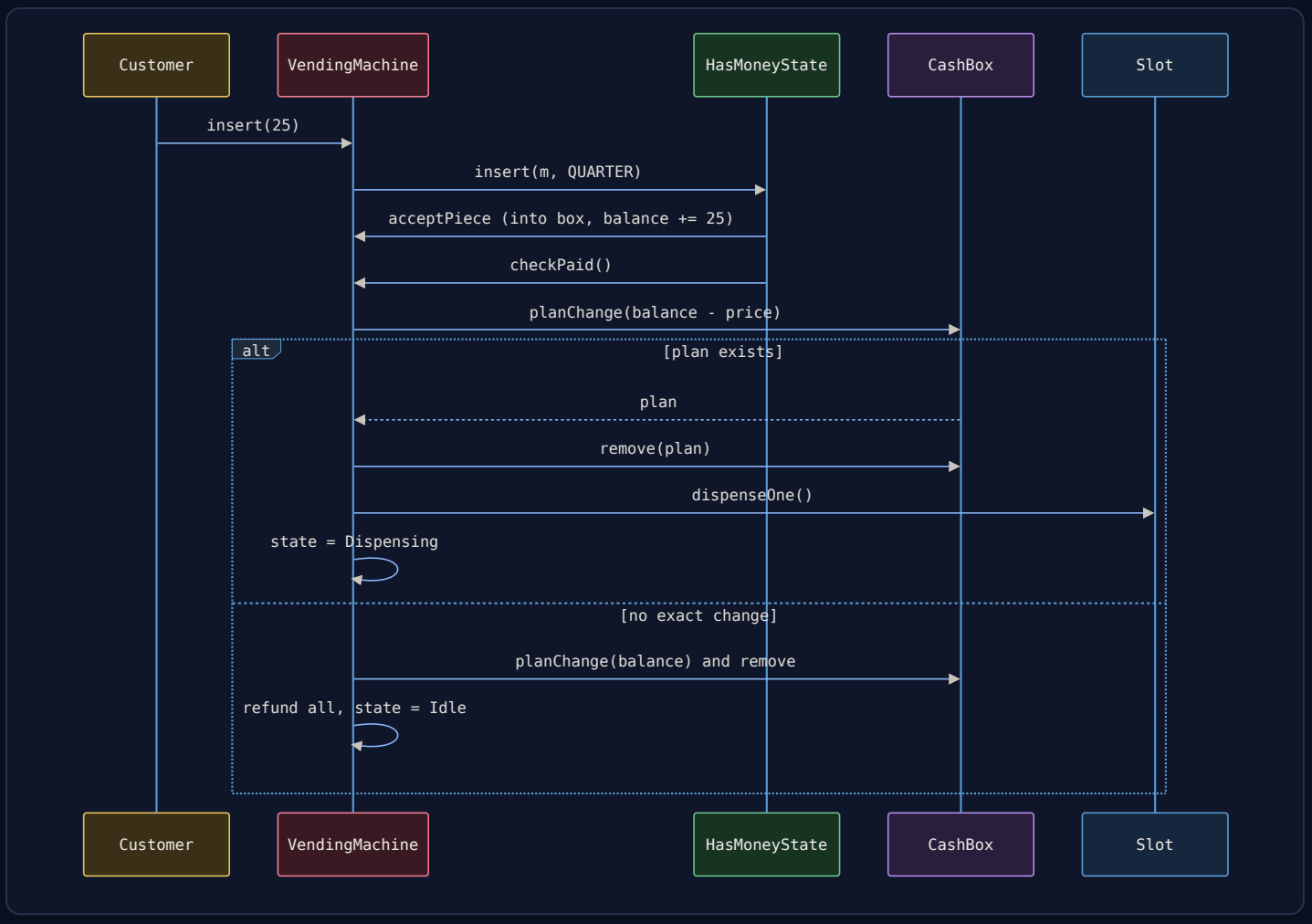
State machine. This diagram *is* the design; draw it before writing code.



The state table (rows = buttons, columns = states). Every cell has an answer:

Button ↓ / State →	Idle	Selected	HasMoney	Dispensing	OutOfStock
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation behind `VendingState`	**State**	Every button × situation cell is answered; the compiler enforces completeness; no `if (state == …)`	Flags + `if/else` chains (the classic source of double-dispense bugs)
States are shared **stateless singletons** (`INSTANCE`; no fields, so one copy serves everyone)	Flyweight-style sharing (one shared object instead of many identical ones)	The course notes that creating a new state object on each transition is readable but wasteful; the states hold no data, so one instance each is enough	`new IdleState()` on every transition
Operator actions are **not** on the state interface	Interface Segregation (no class is forced to implement methods it does not need)	Restocking is allowed in every state; forcing five classes to answer it would be noise	Adding `restock` to `VendingState`
Money counted by value in an `EnumMap` (a fast map keyed by enum constants)	—	The machine never cares which physical quarter it returns	A list of coin objects
Inserted money goes straight into the box	—	Refund = "make change for the balance"; one refund path	A separate escrow pile (a holding area for inserted money)
Exact change with dynamic programming	Correctness	Greedy can fail when the box is short of one coin (see output part 6)	Greedy only
Refuse the sale if exact change is impossible	Requirement 7	Never short-change the customer	Give "close enough" change

About the change algorithm. The course's planner is **greedy** (largest piece first, never looks back) and honestly says so: greedy is correct for standard coin sets when the box is well stocked, but when the box is short of one piece it can fail for an amount that smaller pieces could make. The exact fix is **dynamic programming** — the "coin change with limited coins" problem, where you build the answer for each amount from the answers for smaller amounts. Our `CashBox.planChange` implements that (bounded coin change, fewest pieces), and `greedyPlan` is kept only to show the difference: for 30¢ from {1 quarter, 3 dimes}, greedy takes the quarter and gets stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`. Essential lines:

```
// Inventory.java
record Product(String name, int priceCents) {}

final class Slot {                                  // the COUNT lives on the slot, not on the product
    private final String code;
    private final Product product;
    private int count;
    boolean isEmpty() { return count == 0; }
    Product dispenseOne() {
        if (count == 0) throw new IllegalStateException(code + " is empty");
        count--;
        return product;
    }
}
// Inventory: LinkedHashMap code -> Slot; find(code), allEmpty()
```

```java
// CashBox.java
enum Denomination {                                     // anything else is returned
    NICKEL(5, false), DIME(10, false), QUARTER(25, false), ONE_DOLLAR(100, true), FIVE_DOLLAR(500, true);
    final int cents; final boolean note;
}

final class CashBox {                                   // inserted money goes in at once: a refund is just "make
change"
    private final EnumMap<Denomination, Integer> counts = new EnumMap<>(Denomination.class);

    /** Exact change with the FEWEST pieces, respecting stock (bounded coin change, DP). Empty = impossible. */
    Optional<Map<Denomination, Integer>> planChange(int amount) {
        Denomination[] ds = Denomination.values();
        int[][] best = new int[amount + 1][];           // best[a] = pieces used per denomination to make a
        best[0] = new int[ds.length];
        for (int i = 0; i < ds.length; i++) {
            int v = ds[i].cents;
            int copies = Math.min(counts.get(ds[i]), amount / v);
            for (int c = 0; c < copies; c++)            // each copy may be used once (0/1)
                for (int a = amount; a >= v; a--) {
                    if (best[a - v] == null) continue;
                    int pieces = sum(best[a - v]) + 1;
                    if (best[a] == null || pieces < sum(best[a])) { best[a] = best[a - v].clone(); best[a][i]++; }
                }
        }
        if (best[amount] == null) return Optional.empty();
        // ... convert best[amount] into an EnumMap plan
    }
    // greedyPlan(amount): largest piece first - kept only to compare; remove(plan);
    // collectNotes(): operator empties the notes, coins stay as the float for change
}
```

```java
// States.java - every button is on the interface, so every state must answer every button
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

final class IdleState implements VendingState {
    static final IdleState INSTANCE = new IdleState();   // states hold no data, so share one instance
    @Override public void select(VendingMachine m, String code) {
        m.inventory().find(code).ifPresentOrElse(slot -> {
            if (slot.isEmpty()) { m.show(code + " is sold out. Choose another."); return; }
            m.choose(slot);
            m.show("%s costs %d c. Insert money.".formatted(slot.product().name(), slot.product().priceCents()));
            m.setState(SelectedState.INSTANCE);
        }, () -> m.show("Unknown code " + code));
    }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "select a product first"); }
    @Override public void cancel(VendingMachine m) { m.show("Nothing to cancel."); }
    @Override public void take(VendingMachine m) { m.show("The tray is empty."); }
}

final class HasMoneyState implements VendingState {      // some money in, not enough yet
    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
}
// SelectedState: select = re-select via Idle, insert = acceptPiece + checkPaid, cancel = clear -> Idle, take =
"pay first"
// DispensingState: rejects select/insert/cancel ("too late"); take -> m.emptyTray()
// OutOfStockState: refuses every button; only an operator restock leaves it
```

```java
// VendingMachine.java - forwards every button to the state; no "if (state == ...)" anywhere
final class VendingMachine {
    private VendingState state;
    private Slot selected;
    private int balance;                                  // cents inserted in this sale

    synchronized void select(String code) { state.select(this, code); }
    synchronized void insert(int pieceCents) {            // unknown piece returned before reaching a state
        Denomination.of(pieceCents).ifPresentOrElse(d -> state.insert(this, d),
                () -> show("Unknown piece of " + pieceCents + "c returned."));
    }
    // cancel(), take() likewise

    // operator actions: allowed in every state, so NOT on the state interface
    synchronized void restock(String code, int qty) {
        inventory.find(code).orElseThrow().restock(qty);
        if (state == OutOfStockState.INSTANCE) setState(IdleState.INSTANCE);
    }
    // loadChange(d, n), collectTakings()

    void acceptPiece(Denomination d) { cashBox.add(d, 1); balance += d.cents; }

    void checkPaid() {                                    // after every accepted piece in SELECTED / HAS_MONEY
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
            return;
        }
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) { refundAll("Cannot make exact change."); return; }   // refuse, never short-change
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    void refundAll(String reason) {                       // ONE refund path: cancel and "cannot make change"
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }

    void emptyTray() {                                    // the sale ends when the customer takes the product
        tray.clear();
        setState(inventory.allEmpty() ? OutOfStockState.INSTANCE : IdleState.INSTANCE);
    }
}
// Main.java: A1 Cola 125c x2, B1 Chips 50c x1, C1 Candy 65c x0; float of 2 quarters + 1 dime; six scenarios below.
```

`refundAll` can never fail: the pieces just inserted are in the box, so an exact plan always exists. All public methods are `synchronized`: one physical machine has one customer, but a networked controller (remote price change, telemetry — status data sent automatically to a remote server — or a second input device) must not interleave two actions.

Output:

```
state=IDLE cash=60c stock=[A1:Colax2 B1:Chipsx1 C1:Candyx0]

1) Happy path: Cola with $1 + 25c. Extra coins and a late cancel are refused
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > insert 25c   (DISPENSING)
   [display] QUARTER returned: take your product first.
 > insert 10c   (DISPENSING)
   [display] DIME returned: take your product first.
 > cancel   (DISPENSING)
   [display] Too late to cancel: product already dispensed.
 > take   (DISPENSING)
   [display] Customer took [Cola]

2) Wrong order and bad input
 > insert 25c   (IDLE)
   [display] QUARTER returned: select a product first.
 > select C1   (IDLE)
   [display] C1 is sold out. Choose another.
 > select Z9   (IDLE)
   [display] Unknown code Z9
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 3c   (SELECTED)
   [display] Unknown piece of 3c returned.

3) Cancel with money in: full refund
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > select A1   (HAS_MONEY)
   [display] Cancel first to change your choice.
 > cancel   (HAS_MONEY)
   [display] Cancelled. Refunded 25 c as {QUARTER=1}

4) Cannot make exact change -> sale refused, everything refunded
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 500c   (SELECTED)
   [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
state=IDLE cash=185c stock=[A1:Colax1 B1:Chipsx1 C1:Candyx0]

5) Last items sold -> OUT_OF_STOCK -> operator restocks
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Chips]
 > take   (DISPENSING)
   [display] Customer took [Chips]
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > take   (DISPENSING)
   [display] Customer took [Cola]
state=OUT_OF_STOCK cash=360c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx0]
 > select A1   (OUT_OF_STOCK)
   [display] Machine is out of stock.
   [display] Operator restocked C1 +5
 > select C1   (IDLE)
   [display] Candy costs 65 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Dispensed. Please take: [Candy, change {DIME=1, QUARTER=1}]
```

```
> take   (DISPENSING)
  [display] Customer took [Candy, change {DIME=1, QUARTER=1}]

6) Why the change planner is not greedy
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]

Operator collects notes: 300 c
state=IDLE cash=125c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx4]
```

Read the output: part 1 — coins inserted while the product waits are returned and a late cancel is refused; part 4 — 450¢ change cannot be made, so the $5 note comes straight back; part 5 — `OUT_OF_STOCK` and back to `IDLE` after a restock; part 6 — greedy fails where DP succeeds.

Concurrency & edge cases

Edge case	Handling
Money inserted before selecting	`IdleState.insert` returns the piece
Unknown coin or foreign note	Returned before reaching the state
Selecting an empty slot	`IdleState.select` refuses; the state stays Idle
Changing the choice after paying	`HasMoneyState.select` says "cancel first" (prevents price confusion)
Paying more than the price	Change is planned **before** dispensing; if impossible, the sale is refused
Cancel after dispensing	"Too late" — the product is already in the tray
Two products for one payment	Impossible: after dispensing, the balance is 0 and the state is Dispensing, which accepts no money
Last item sold	`take` moves to `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods; states run under the machine's lock
A bank of machines sharing one stock count	The course notes the design is not safe for that; the shared count on `Slot` would then need a lock or an atomic decrement
Power failure mid-sale	Real machines keep a transaction log (credit held) in non-volatile memory (memory that keeps its contents without power); out of scope

Extensions & follow-up questions

1. **"Where would you add contactless card payment?"** — One new input device beside the cash slot that calls the same machine (for example a `payByCard(amount)` that sets the balance to the price). No state class changes if the states only look at the balance.
2. **"What if it cannot give change?"** — The planner returns empty; the sale is refused and everything is refunded (requirement 7). Mention greedy vs DP.
3. **"Add a maintenance state."** — A new `MaintenanceState` class; the operator switches it on. The existing five states are untouched (Open/Closed).
4. **"Different price per slot for the same product?"** — Move the price from `Product` to `Slot` (the slot already owns the count).
5. **"Report empty slots to headquarters."** — Make `Inventory` notify an observer when a slot reaches 0; a telemetry client sends it over the network.
6. **"What did you leave out on purpose?"** — Card payment, reporting empty slots over a network, refrigeration (chilling) control and per-customer discounts. Say so explicitly: they are scope decisions, not oversights.
7. **"Timeout if the customer walks away with money in?"** — A timer that calls `cancel()` after, say, 60 seconds of inactivity; it goes through the same state machine.

Real-world example

Modern vending machines are often retro-fitted with cashless readers such as **Nayax** or **Cantaloupe (ePort)** units, which plug into the machine's controller as just another payment device — exactly extension 1: a new input beside the coin mechanism, with the dispensing logic unchanged. Coin mechanisms in real machines keep "change tubes" of each coin and show an **"exact change only"** light when a tube runs low — the physical version of our planner reporting that it cannot make change.

In the interview

Plan for 45 minutes: 5 min questions and requirements → 5 min entities (product, slot, inventory, cash box) → **10 min: draw the state diagram and the state table** → 15 min code the interface, two or three states and `checkPaid` / `refundAll` → 10 min edge cases and follow-ups.

> **30-second answer:** "The whole question is the order of events, so I'll use the State pattern: one class per situation — Idle, Selected, HasMoney, Dispensing, OutOfStock — all implementing an interface that lists every button. The machine just forwards to its current state. Money goes into the cash box as it is inserted, so a refund is just 'make exact change for the balance'; if exact change is impossible, the sale is refused and everything is returned. Amounts are integer cents."

Follow-ups with answers

- *"Why not an enum with a switch?"* — It works for tiny machines, but every new state means editing every switch; with the State pattern a new state is a new class, and the compiler forces it to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets (coin systems such as 1/5/10/25 where taking the largest coin first always gives the fewest coins). With limited coins it can fail; use bounded coin-change DP.
- *"Where does the operator fit?"* — Outside the state interface; restocking is allowed in any state and can move OutOfStock → Idle.

TL;DR

- **State pattern is the whole design**: five state classes, one interface listing every button, no `if` on state in the machine.
- Count lives on the **slot**; money in **integer cents**; inserted money goes **into the box immediately**.
- Change: **exact** or refuse; greedy can fail, **DP** (bounded coin change) is the correct planner.
- One refund path for cancel and "no change"; operator actions stay off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee vending machine** makes drinks on demand — espresso, americano, latte, cappuccino, mocha, hot chocolate — from a shared stock of **ingredients** (beans, water, milk, chocolate, sugar). It looks like the previous problem, but the hard part is different. A snack machine hands out a finished item from a slot. A coffee machine **consumes several ingredients at once** from shared containers, and many drinks draw on the same milk. The interviewer wants to see three things:

1. **Recipes as data** — adding a drink should be a data change, not a new class.
2. **Atomic, all-or-nothing deduction** — a latte must take its beans, water *and* milk together, or take nothing; stock must never go negative, even when ten people order at once.
3. **Low-stock alerts** — the operator hears about milk running low *before* it runs out (Observer).

Problem & clarifying questions

Question	Assumed answer
Which drinks?	Espresso, Americano, Latte, Cappuccino, Mocha, Hot Chocolate — loaded from a menu file.
Customisation?	Extra espresso shot (+50¢) and sugar (grams).
Payment?	Amount paid in cents; change returned (coin handling was covered in 14.2).
Several screens / concurrent orders?	Yes: a large machine or a kiosk with several touch screens, so orders can overlap.
Who refills?	An operator, alerted when an ingredient falls to a threshold.
One alert or one per drink?	One alert per crossing of the threshold (re-armed after a refill).

Requirements

Functional

1. Several drink types, each with a **price** and a **recipe** (amount of each ingredient).
2. Show a **menu** with prices and availability.
3. Accept a selection and a **payment**; return **change**; refund in full if the drink cannot be made.
4. Track **ingredient inventory**; deduct exactly the recipe amounts per drink.
5. Raise **low-stock notifications** when an ingredient drops to its threshold.
6. Allow **refills**.
7. Support **customisation** (extra shot, sugar) without changing the menu.

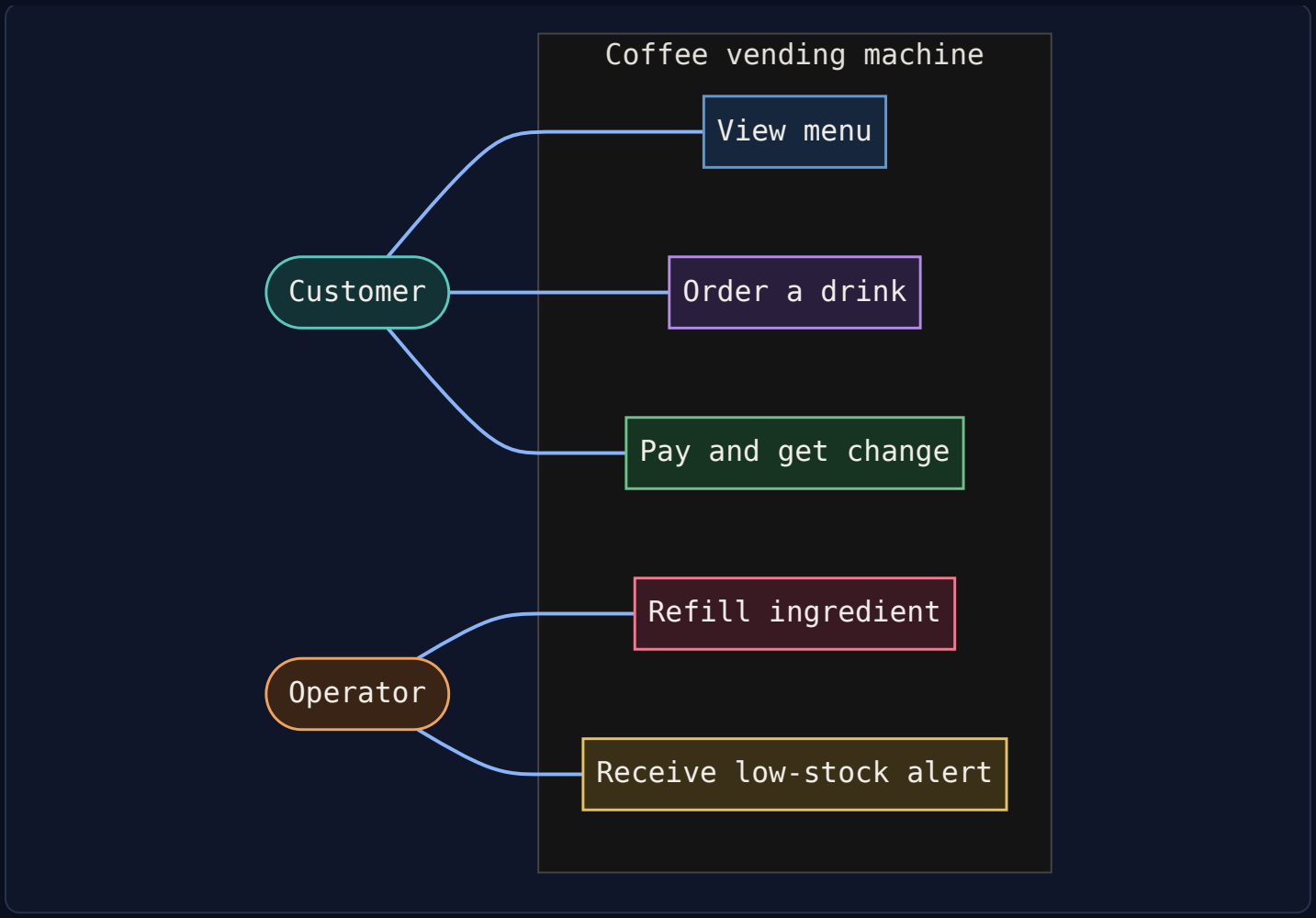
Non-functional

- **Thread-safe**: concurrent orders must never over-draw an ingredient or leave a half-deducted recipe.
- **Data-driven**: new drinks and recipe tweaks without code changes.
- **Observable**: operators, dashboards or a telemetry client can subscribe to alerts without the machine knowing about them.

Use cases

Actors: **Customer** (views menu, orders, pays, takes drink and change), **Operator** (refills, receives alerts), **System** (checks stock, deducts, brews, sends alerts).

Use cases: view menu · order drink (with customisation) · pay / get change / get refund · refill ingredient · receive low-stock alert.



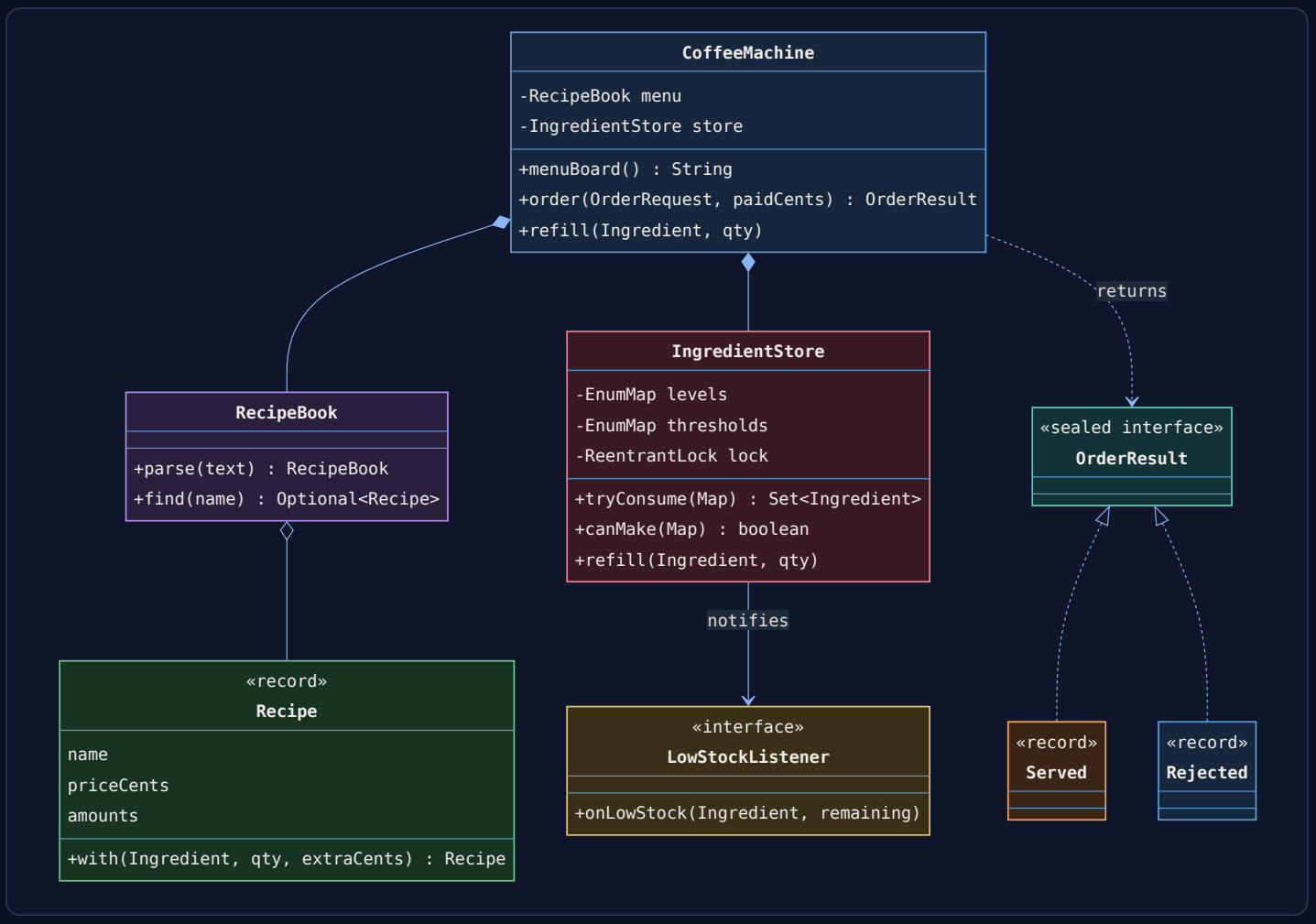
Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with the unit (g or ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient, Integer>` amounts. Immutable (cannot change after creation). `with(...)` returns a customised copy.
`RecipeBook`	Parses the menu text into recipes; looks drinks up by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts listeners.
`LowStockListener` (interface)	Observer called once when an ingredient crosses its threshold.
`OrderRequest` (record)	Drink name + extra shots + sugar grams.
`OrderResult` (sealed)	Either `Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade (one simple front object hiding the parts behind it): menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement. The widely used version (e.g. the open-source *awesome-low-level-design* collection) asks for coffee types with price and recipe, a menu, selection and payment, dispensing with change, ingredient tracking with low-stock notifications, and thread-safe concurrent requests. Its `Coffee` (name, price, recipe) → our `Recipe`; `Ingredient` (name + quantity with `synchronized` updates) → our `Ingredient` enum plus the levels in `IngredientStore`; `Payment` → the `paidCents` argument and change/refund in `OrderResult`; `CoffeeMachine` (a Singleton there) → our injected `CoffeeMachine`; concurrent users via an `ExecutorService` (a pool of worker threads running submitted tasks) → our 10-thread demo. Two deliberate differences: we lock the **whole recipe**, not each ingredient

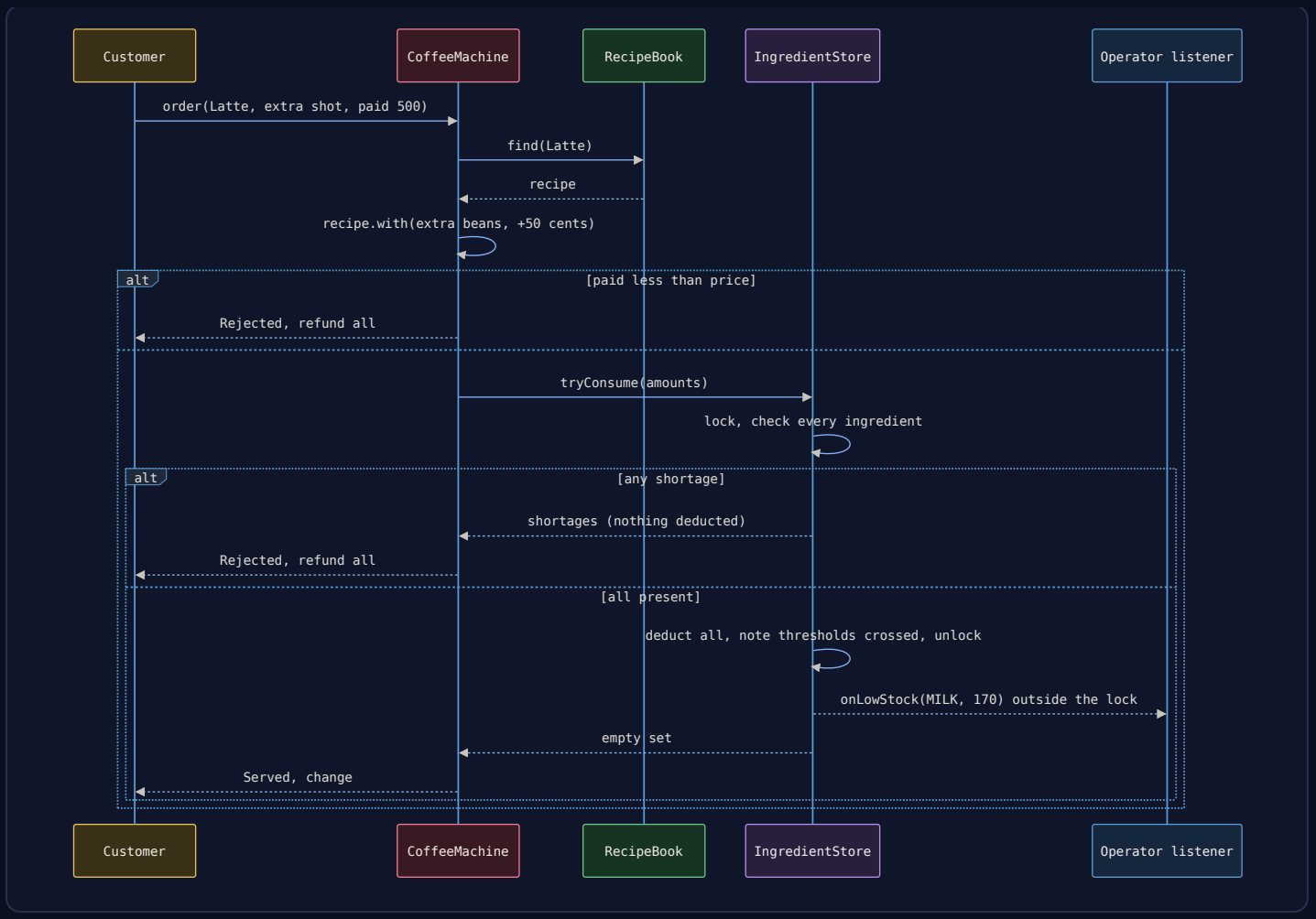
(per-ingredient locking cannot make a multi-ingredient deduction all-or-nothing), and we avoid the Singleton.

Class diagram

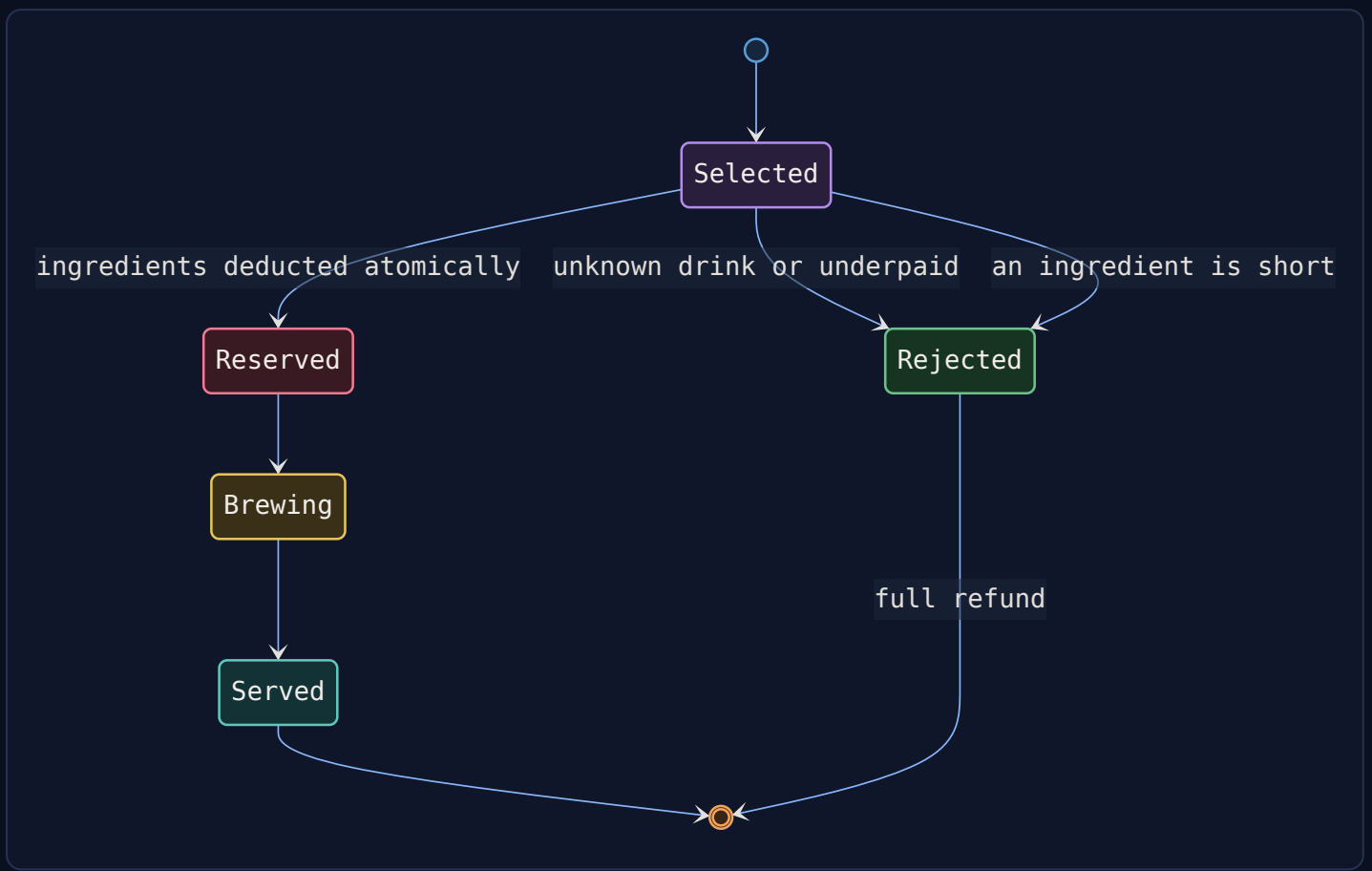


Key flows

Ordering a drink. The key point is that the stock check and the deduction happen **inside one critical section** (a piece of code that only one thread may run at a time), and alerts are sent **after** the lock is released.



Order life cycle.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** parsed from a menu text	Data-driven design, Open/Closed	A new drink is a new line in the menu; no new class, no recompile of logic	One subclass per drink (`Latte extends Coffee`) — class explosion, and recipe tweaks need code changes
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Prevents two orders both seeing "enough milk" and both deducting (a check-then-act **race**: both check, then both act on an answer that is already stale)	Checking in one call and deducting in another; per-ingredient locks (a multi-ingredient recipe would need all of them, in order)
All-or-nothing	Transaction semantics (all steps take effect together, or none do)	Never leave a half-made recipe's ingredients missing	Deduct one by one and "roll back" on failure
Alerts via `LowStockListener`	**Observer**	Operator app, dashboard, telemetry can subscribe independently	The store calling a specific SMS class
Alert **once per crossing**, re-armed by refill	—	Prevents alert storms (one per drink)	Alerting on every order below threshold
Listeners called **outside** the lock	Concurrency hygiene	A slow listener (network call) must not block every order; it also avoids deadlocks (two threads each waiting forever for a lock the other holds) if a listener calls back into the store	Notifying inside the lock
Customisation returns a **new** immutable `Recipe`	Immutability	The menu recipe can never be corrupted by one order	Mutating the menu recipe; a Decorator chain (works, but heavier than needed for "extra X")
`OrderResult` is a **sealed** interface with records	Algebraic data type (a value that is exactly one of a fixed set of shapes; Java 17+ sealed, Java 21 pattern switch)	Callers must handle both Served and Rejected; `switch` is checked for exhaustiveness (the compiler complains if a case is missing)	Returning `null` or throwing for normal refusals
No Singleton	Dependency injection	The problem-set solutions often make `CoffeeMachine` a Singleton; we create one and pass it around, so tests can build fresh machines (the demo builds two)	`CoffeeMachine.getInstance()`

Decorator or data? A classic textbook example (*Head First Design Patterns*) models coffee add-ons with the **Decorator** pattern: `new Whip(new Mocha(new DarkRoast()))` , each wrapper adding cost and description. That is a good fit when each add-on has **behaviour**. Here add-ons only change **quantities and price**, so a data change (`recipe.with(...)`) is simpler. Say both in the interview and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`. Essential lines:

```
// Recipes.java
enum Ingredient { COFFEE_BEANS("g"), WATER("ml"), MILK("ml"), CHOCOLATE("g"), SUGAR("g"); final String unit; }

record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {   // a recipe is DATA
    Recipe { amounts = Collections.unmodifiableMap(new EnumMap<>(amounts)); }   // compact constructor: defensive copy
    Recipe with(Ingredient extra, int qty, int extraCents) {     // customisation returns a NEW recipe
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}
// RecipeBook.parse(text): each non-comment line "name, price_cents, INGREDIENT=qty, ..." -> a Recipe;
// find(name) is case-insensitive. Main.MENU is a text block, e.g.  Latte, 400, COFFEE_BEANS=18, WATER=40, MILK=180
```

```
// IngredientStore.java - the whole check-then-deduct runs under ONE lock: all-or-nothing, never negative
interface LowStockListener { void onLowStock(Ingredient ingredient, int remaining); }

final class IngredientStore {
    private final Map<Ingredient, Integer> levels = new EnumMap<>(Ingredient.class);
    private final Map<Ingredient, Integer> thresholds = new EnumMap<>(Ingredient.class);
    private final Set<Ingredient> alerted = EnumSet.noneOf(Ingredient.class);
    private final List<LowStockListener> listeners = new CopyOnWriteArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();

    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;               // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i))      // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);                                // listeners OUTSIDE the lock
        return shortages;
    }
    void refill(Ingredient i, int qty) {
        lock.lock();
        try {
            levels.merge(i, qty, Integer::sum);
            if (levels.get(i) > thresholds.get(i)) alerted.remove(i); // re-arm the alert
        } finally { lock.unlock(); }
    }
    // canMake(need) and level(i) read under the same lock
}
```

```java
// CoffeeMachine.java
record OrderRequest(String drink, int extraShots, int sugarGrams) {}
sealed interface OrderResult permits Served, Rejected {}            // callers must handle both
record Served(String drink, int pricedCents, int changeCents) implements OrderResult {}
record Rejected(String reason, int refundCents) implements OrderResult {}

final class CoffeeMachine {                       // EXTRA_SHOT_CENTS = 50, SHOT_BEANS_G = 18
    OrderResult order(OrderRequest req, int paidCents) {             // thread-safe: many touch screens
        var base = menu.find(req.drink());
        if (base.isEmpty()) return new Rejected("no such drink: " + req.drink(), paidCents);
        Recipe r = base.get()
                .with(Ingredient.COFFEE_BEANS, req.extraShots() * SHOT_BEANS_G, req.extraShots() *
EXTRA_SHOT_CENTS)
                .with(Ingredient.SUGAR, req.sugarGrams(), 0);
        if (paidCents < r.priceCents()) return new Rejected("insufficient payment, need " + r.priceCents() + " c",
paidCents);
        Set<Ingredient> short_ = store.tryConsume(r.amounts());     // atomic reserve-and-deduct
        if (!short_.isEmpty()) return new Rejected("out of " + short_, paidCents);
        return new Served(r.name(), r.priceCents(), paidCents - r.priceCents());
    }
    // menuBoard(): marks a drink "(unavailable)" when !store.canMake(recipe.amounts()); refill(i, qty)
}
// Main.show uses a Java 21 pattern switch over the sealed type:
//   case Served s -> ...;  case Rejected x -> ...
// Last demo: 10 threads released by a CountDownLatch each order a Latte with milk for exactly 4.
```

A Java text block is a multi-line string between `"""` marks; a compact constructor is a record constructor without a parameter list, used for checks and defensive copies. `ReentrantLock` is an explicit lock with `lock()` / `unlock()` (*re-entrant*: the holder may take it again); `EnumSet` is a compact set of enum constants.

Output:

```
Menu:
  Espresso        250 c
  Americano       300 c
  Latte           400 c
  Cappuccino      380 c
  Mocha           450 c
  Hot Chocolate   300 c

== Orders ==
SERVED   Espresso for 250 c, change 50 c
SERVED   Latte for 450 c, change 50 c
REJECTED no such drink: Flat White, refund 400 c
REJECTED insufficient payment, need 450 c, refund 200 c
SERVED   Mocha for 450 c, change 0 c
   ALERT to operator: MILK low, 170ml left
   ALERT to operator: CHOCOLATE low, 5g left
SERVED   Hot Chocolate for 300 c, change 0 c
SERVED   Cappuccino for 380 c, change 20 c
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
Menu now:
  Espresso        250 c
  Americano       300 c
  Latte           400 c  (unavailable)
  Cappuccino      380 c  (unavailable)
  Mocha           450 c  (unavailable)
  Hot Chocolate   300 c  (unavailable)

== Operator refills milk ==
SERVED   Latte for 400 c, change 0 c

== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Read the output: the rejected latte left the beans at 410 g (nothing deducted when any ingredient is short); milk and chocolate alerts fire once each; ten simultaneous lattes with milk for four → four served, six refused, milk ends at 80 ml (never negative), one alert.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Ten concurrent orders, milk for four	Over-draw (negative stock) — classic check-then-act race	Check and deduct under one lock
Recipe needs several ingredients, one is short	Beans and water taken, milk missing	All-or-nothing: shortages are found before anything is deducted
Listener is slow (sends an SMS)	Every order waits on the network	Listeners run after `unlock()`
Many orders below the threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving ingredients
Brewing fails after deduction (hardware fault)	Customer pays, gets nothing	Extension: refund and optionally `refill` the reserved amounts back (compensation: an action that undoes an earlier step)
Menu displays "available" and then the order fails	A race between display and order	The menu is advisory; `tryConsume` is the source of truth

A **single lock** is right here: the critical section is tiny (five map operations) and the machine brews one drink at a time anyway. If contention (many threads waiting for the same lock) mattered (a cloud-backed inventory shared by many machines), you would move to optimistic concurrency (work without a lock, then check at write time that nobody changed the data, and retry if they did — e.g. version numbers / compare-and-set on a snapshot) or a database transaction (a group of database changes that succeed or fail together).

Extensions & follow-up questions

1. **"Add a new drink."** — Add one line to the menu data. No code change.
2. **"Sizes (small/medium/large)?"** — A `Size` enum with a multiplier; `recipe.scaled(size)` returns a new recipe with scaled amounts and price.
3. **"Several brewing units?"** — A pool of `Brewer` workers (a `BlockingQueue` — a thread-safe queue where a taker waits while it is empty — of orders consumed by N threads). Inventory stays in the one `IngredientStore`.
4. **"Remote monitoring?"** — Another `LowStockListener` that posts to a telemetry endpoint. The store does not change.
5. **"Brewing steps differ per drink (grind, tamp, steam milk)?"** — Template Method (a base class fixes the order of the steps and subclasses fill in some of them): a `BrewProcess` with fixed steps and hooks (optional steps a subclass may override), or a list of `Step` objects in the recipe data (Command).
6. **"Card payments and change?"** — Reuse the `CashBox` from 14.2 for cash; card payment is another payment device.

Real-world example

Professional espresso machines such as the Thermoplan Mastrena used in Starbucks stores store **drink recipes as configuration**, so a chain can roll out a new beverage without new hardware. Starbucks and Microsoft publicly described (in 2019) connecting store equipment, including espresso machines, to the cloud through Microsoft Azure Sphere: each shot's data is collected as telemetry and new recipes are pushed from the cloud instead of carried to stores on USB drives — "recipes as data" plus a remote-

monitoring listener. Many connected self-serve kiosks report stock remotely so a service visit is scheduled before the milk runs out — the low-stock Observer.

In the interview

Plan for 40 minutes: 5 min questions → 5 min entities (ingredient, recipe, store, machine) → 5 min "recipes are data" and the menu format → 15 min code `tryConsume` (lock, all-or-nothing, alert outside the lock) and `order` → 10 min concurrency demo and extensions.

> **30-second answer:** "Recipes are data: a map from ingredient to amount, loaded from a menu, so a new drink is a new row. The ingredient store checks and deducts **every** ingredient of a recipe under one lock, so an order is all-or-nothing and stock never goes negative. When an ingredient crosses its threshold, the store notes it and, after releasing the lock, notifies observers once. Orders return a sealed Served/Rejected result, and a rejected order is refunded in full."

Follow-ups with answers

- *"Why not one class per drink?"* — Drinks differ only in data. Classes would multiply with every size and add-on (class explosion) and every recipe tweak would be a code change.
- *"Why not an `AtomicInteger` per ingredient?"* — Each atomic is safe alone, but a recipe updates several at once; without a common lock, two orders could each take the last milk and beans in different orders, leaving partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

TL;DR

> - **Recipes are data** (`Map<Ingredient,Integer>` + price), parsed from a menu; customisation creates a new immutable recipe.
> - **One lock, check-then-deduct, all-or-nothing**: stock never goes negative under concurrent orders.
> - **Observer** for low-stock alerts, fired **once per crossing** and **outside** the lock.
> - A **sealed** `OrderResult` makes callers handle Served and Rejected; refund in full on rejection.

14.4 Design an ATM



Figure — An automated teller machine with its card reader, keypad, screen, cash dispenser, deposit slot and printer (course diagram)

An **automated teller machine (ATM)** is a cashier-less machine in a public spot where a bank's customers do everyday banking, because branches keep limited hours. The interviewer checks whether you keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate** before anything else, and model the **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Which transactions?	Balance inquiry, cash withdrawal, cash deposit, check deposit, transfer.
Which account types?	Checking and savings. Withdrawal is from checking (and savings within a daily cap).
How is the user identified?	ATM card + PIN (personal identification number, a short secret code). The card is the main security element.
Does the ATM talk to the bank?	Yes, on every transaction (assume the network is available).
Are deposits available at once?	A check is **not**: it waits for verification. We keep a *total* and an *available* balance.
Which notes are loaded?	$100, $50, $20, $10 cassettes.
Wrong PIN?	Three wrong attempts block the card and the ATM keeps it. (The course deliberately leaves out PIN retry limits and card capture, along with receipt-paper levels, an audit journal and other banks' networks; we add the first two and the journal.)
How many customers at a time?	One per machine; the ATM must not shut down mid-session.

Requirements

Parts of the machine (from the course): **card reader** (reads the card), **keypad** (PIN, amounts), **screen** (messages; also used to pick a transaction), **cash dispenser**, **deposit slot** (cash or checks), **printer** (receipts), and a **network link** to the bank.

Functional

1. The customer inserts a card and enters a PIN; the ATM asks the **bank** to authenticate. No transaction is possible without authentication.
2. Five transactions: **balance inquiry** (each account), **deposit cash**, **deposit check**, **withdraw cash** (from checking), **transfer funds** to another account.
3. The ATM **keeps the card** for the whole session; the customer can **cancel** at any time and the card is ejected.
4. The ATM keeps an **internal log (journal)** of transactions, including hardware failures, for the operator to resolve issues.
5. A deposited **check is not added at once**: it waits for manual verification and bank approval. (The course assumes deposits in general are verified first; our code credits counted cash immediately and holds checks — see the trade-off below.)
6. The **bank manager** can see the ATM's system information (reports).

Actors' operations

- **Operator:** turn the ATM on/off with the key switch; refill cash; refill receipt paper and ink; take out deposited cash and checks.
- **Customer:** balance inquiry, withdraw, deposit (cash or check), transfer.
- **Bank manager:** reports of total deposits and total withdrawals; print them; check the cash remaining in the ATM.

Non-functional

- **Security:** PINs never stored or compared in clear on the ATM; limit PIN attempts.
- **Consistency:** debit and dispense happen together or not at all (compensate on failure).
- **Concurrency:** many ATMs act on the same account at once without overdrawing it or deadlocking.
- **Auditability** (every action can be traced afterwards)**:** every action is journaled.

Use cases

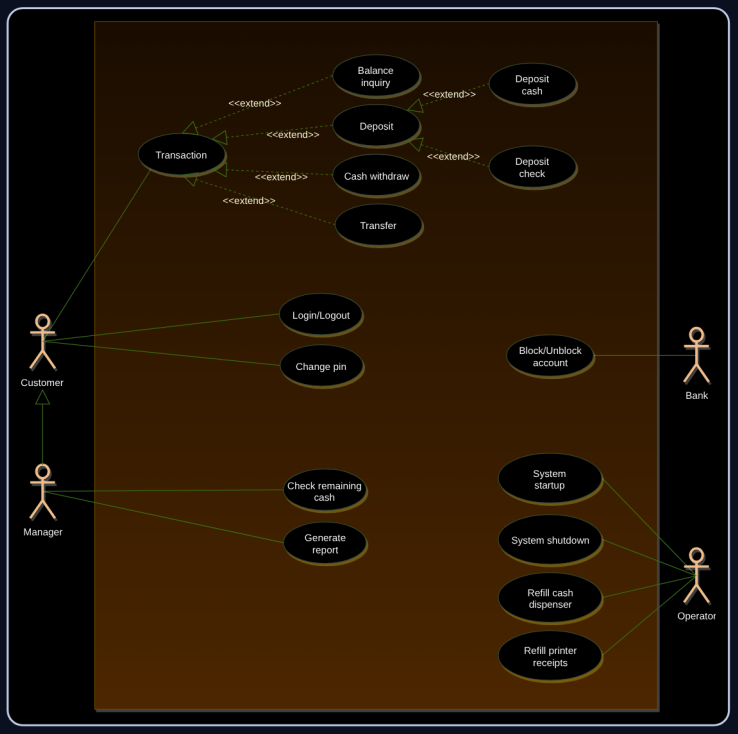


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

The figure above shows three actors. Only the Customer's use cases go through PIN authentication; the Operator works with a physical key and the Bank Manager reads data held at the bank.

Customer authentication is the entry point of every session:

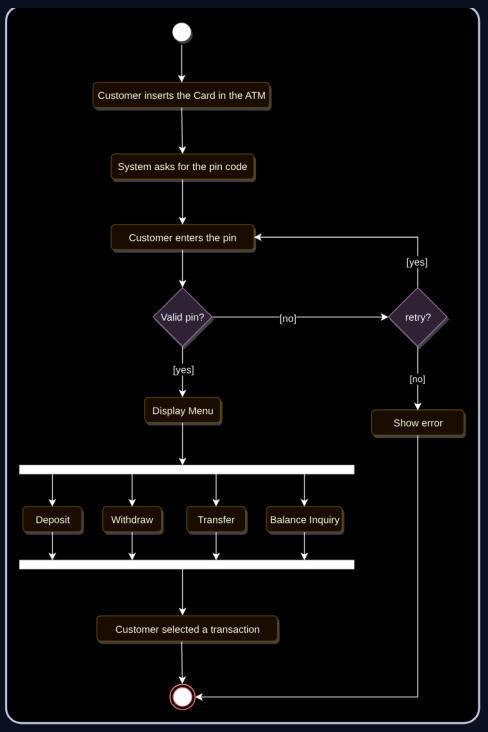


Figure — Activity diagram: insert card, the system asks for the PIN, the customer enters it; an invalid PIN shows an error and offers a retry; a valid PIN displays the menu (balance inquiry, deposit, withdraw, transfer) (course diagram)

Withdraw:

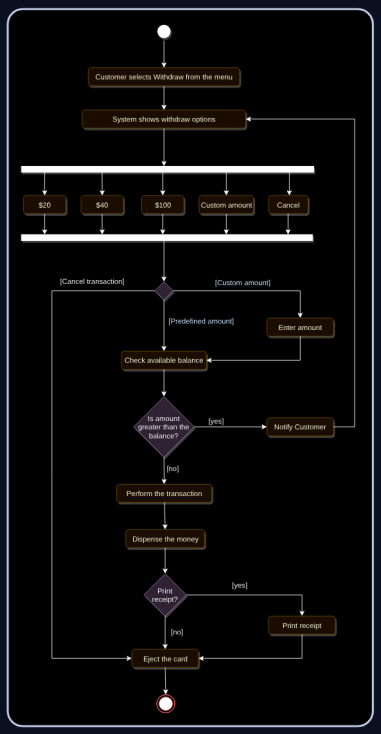


Figure — Activity diagram: choose Withdraw, pick a predefined amount ($20, $40, $100) or enter a custom one, check the available balance (notify the customer if the amount is greater), perform the transaction, dispense, optional receipt, eject the card (course diagram)

Deposit check:

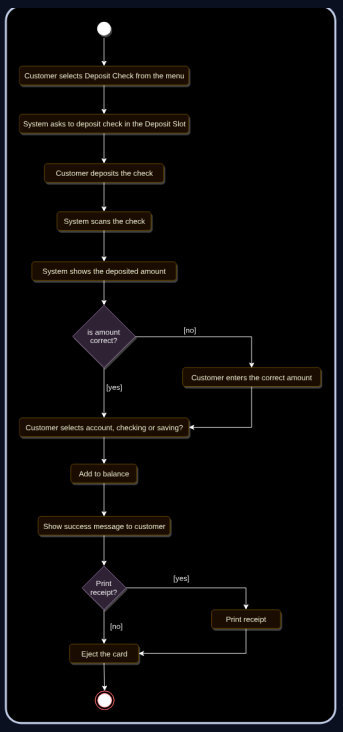


Figure — Activity diagram: choose Deposit Check, pick checking or savings, put the check in the deposit slot, the system scans it and shows the amount; if wrong the customer enters the correct amount; the amount is added, a success message and optional receipt follow, and the card is ejected (course diagram)

Transfer:

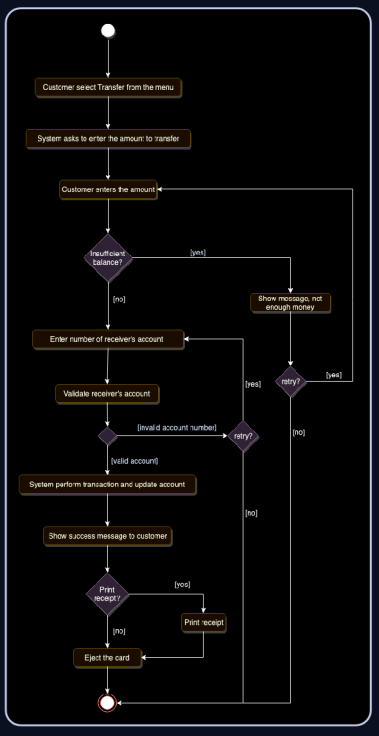


Figure — Activity diagram: enter the amount (retry if balance is insufficient), enter and validate the receiver's account (retry if invalid), perform the transfer, show success, optional receipt, eject card (course diagram)

Note the two retry loops in the transfer diagram ("insufficient balance", "invalid account number"). If the customer declines to retry, the diagram ends without "Eject the card"; in a real design every exit path must return the card — the card-reader rule in stage 2 below.

Core entities

Class	Responsibility
`Card` (record)	Card number and holder name. (The course also has a `Customer` with name and email owning exactly one card; we fold the holder's name into the card.)
`Account` (abstract)	Id, **total** and **available** balance, a lock; the withdrawal-rule hook.
`CheckingAccount`, `SavingsAccount`	Savings overrides the rule with a **daily withdrawal cap** — the only behavioural difference, hence two classes.
`BankService` (interface)	What the ATM needs from the bank: authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	A stand-in bank: PIN hashes (one-way fingerprints of PINs), failed-attempt counts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)` or `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** for $100 → $50 → $20 → $10.
`CashDispenser`	Plans a withdrawal on the chain (dry run: a trial that changes nothing), then dispenses; can jam (fault injection: a deliberately triggered failure, for testing).
`Transaction` (sealed) + 5 records	**Command**: each transaction has one `execute(ctx)` holding its rules.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated → Idle, plus OutOfService.
`Atm`	Has an id (the course adds a location); owns its parts, holds the session, forwards buttons to the state, keeps the journal.
`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot` (cash / check)	The course's hardware classes, folded into `Atm`: the `card` field is the reader's slot, `screen(...)` the screen, method arguments the keypad; a receipt would format a journal line.

Class diagram

The course builds the ATM in four stages:

Stage	What it adds	What works after it
1	`Customer`, `Card`, `Account` (checking, savings)	Verify a PIN and read a balance
2	Card reader, dispenser, keypad, screen, printer, deposit slots	Work out and count out a set of notes
3	`Transaction` and its five kinds	Move money, with each rule written in exactly one class
4	`Bank` and `ATM`	Run a full session for a single customer at a time

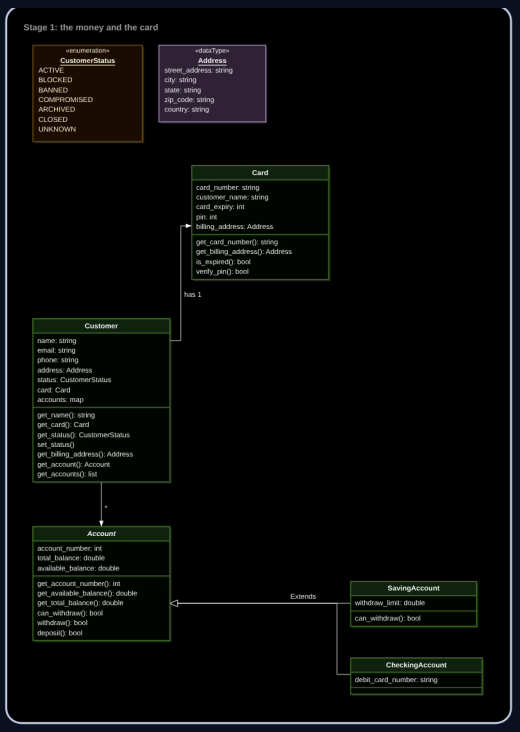


Figure — Stage 1: enumerations, Customer, Card and the abstract Account with Checking and Saving subclasses (course diagram)

Stage 1 — money and card. An account keeps **two balances**. The *total balance* is everything credited to it; the *available balance* is the part the customer may spend right now. A check deposit is the moment they diverge: the check raises the total at once, but the money becomes available only after the check clears. `Account` is abstract with `CheckingAccount` and `SavingsAccount` beneath it; savings overrides a single method to impose its withdrawal cap, and because that is a difference in **behaviour** (not just data) inheritance is justified here instead of a type field. `Customer` (name, email; each customer has **one** card) is linked to its card and accounts by plain associations, since none of them is deleted along with another.

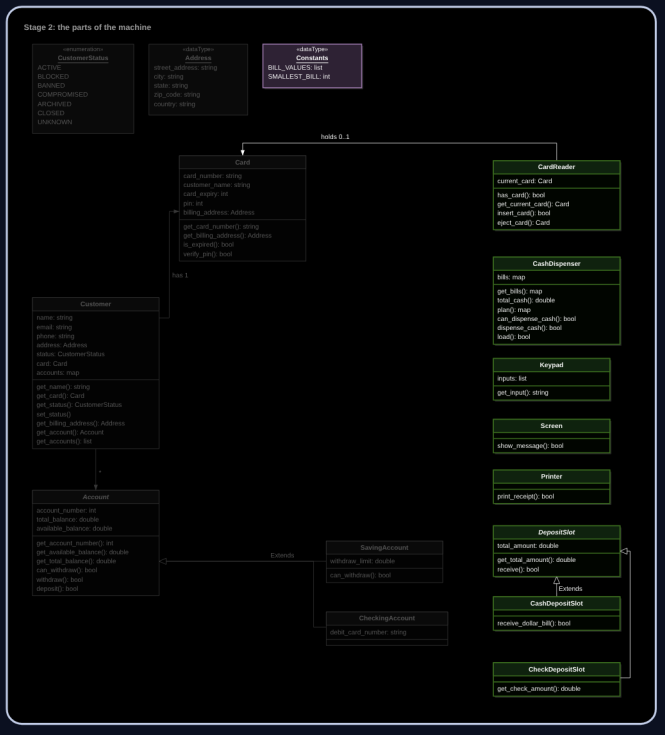


Figure — Stage 2: CardReader, CashDispenser, Keypad, Screen, Printer and the abstract DepositSlot with cash and check slots (course diagram)

Stage 2 — the parts. Six of the seven hardware parts become one class each (the network link is assumed, not modelled), and none knows what a transaction is. Whether an amount (say $85) can be paid from the notes **currently loaded** is the **dispenser's** decision, not the withdrawal's: it tries the largest

note first and reports failure when the exact amount is impossible. The card reader **keeps the card for the whole session** and releases it when the session ends, so every failed sign-in path must eject (or retain) the card; its link to `Card` is the only arrow leaving this stage. `DepositSlot` is abstract (cash slot, check slot) — the physical opening, not the deposit *transaction* of stage 3.

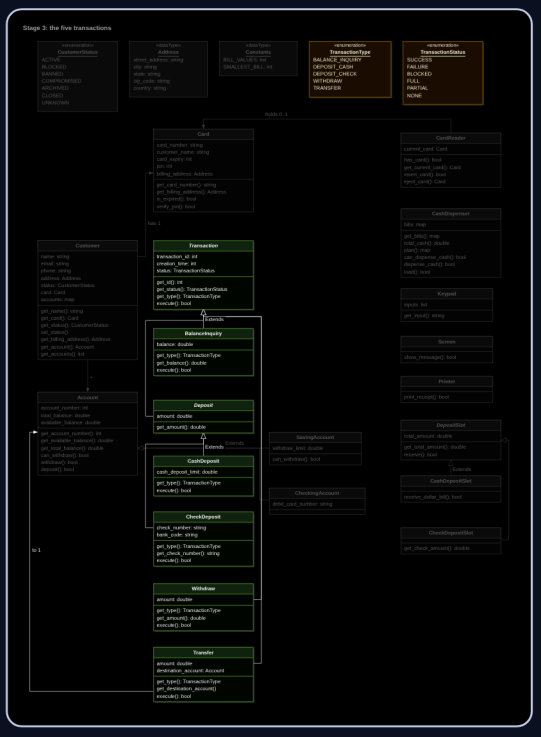


Figure — Stage 3: the abstract Transaction with BalanceInquiry, Deposit (CashDeposit, CheckDeposit), Withdraw and Transfer (course diagram)

Stage 3 — the five transactions. Each transaction type is a class with one `execute` method, so each rule has exactly one home. `Deposit` is intermediate: cash and check deposits share an amount but differ in when the money becomes spendable. `Transfer` is the only one with an outgoing arrow (to the destination account). The course's `Withdraw` checks **note denominations**, then the **account**, then the **dispenser**, and debits only once the notes are counted out — if you take the money first and dispensing fails, the customer has paid for cash never received.

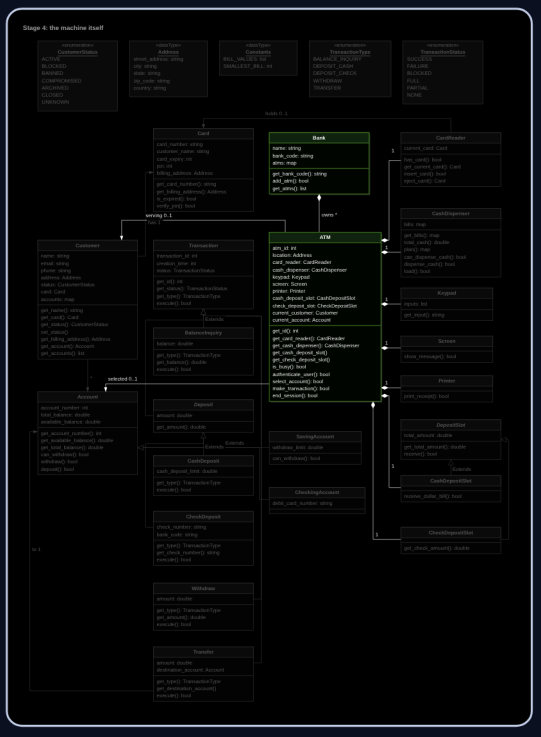


Figure — Stage 4: the ATM owns its parts (composition), refers to the current Customer and Account, and the Bank owns its ATMs (course diagram)

Stage 4 — the machine. The ATM has an **id** (`atmID`) and a **location** and **owns** its parts (composition: scrap the machine and its reader, dispenser and screen go too). Its links to the current `Customer` and `Account` are plain associations that change per person, so "one customer at a time" is enforced by structure, not a comment. The `Bank` owns its ATMs and all account data; the ATM asks the bank to carry out every customer transaction.

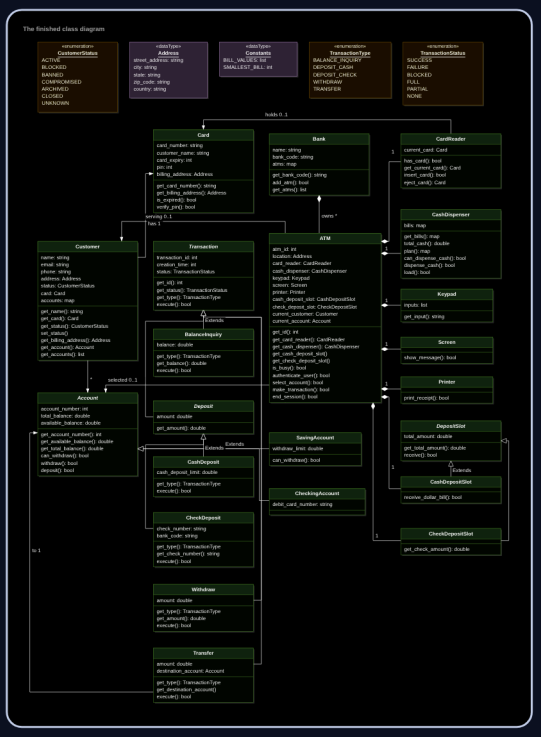
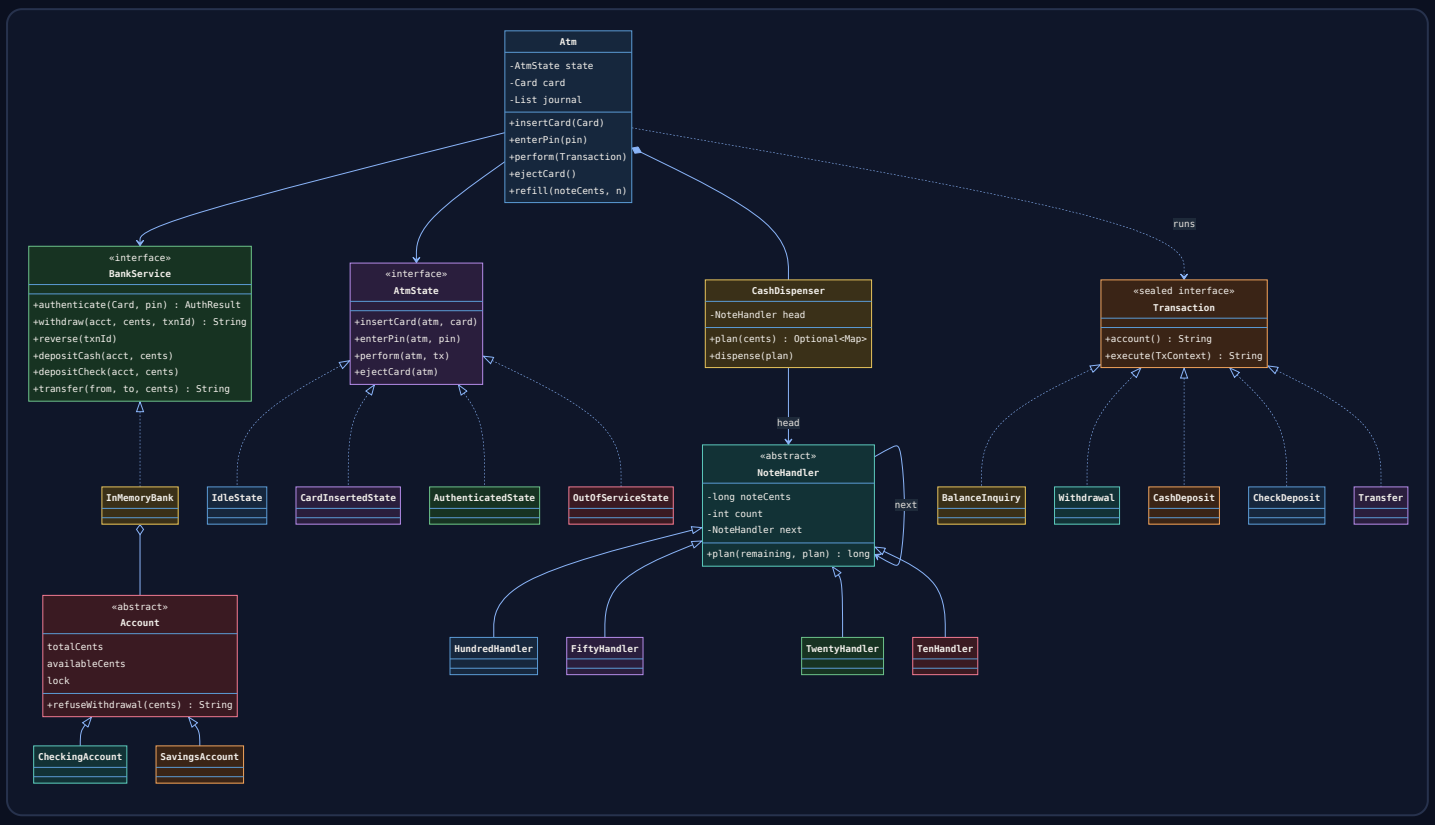


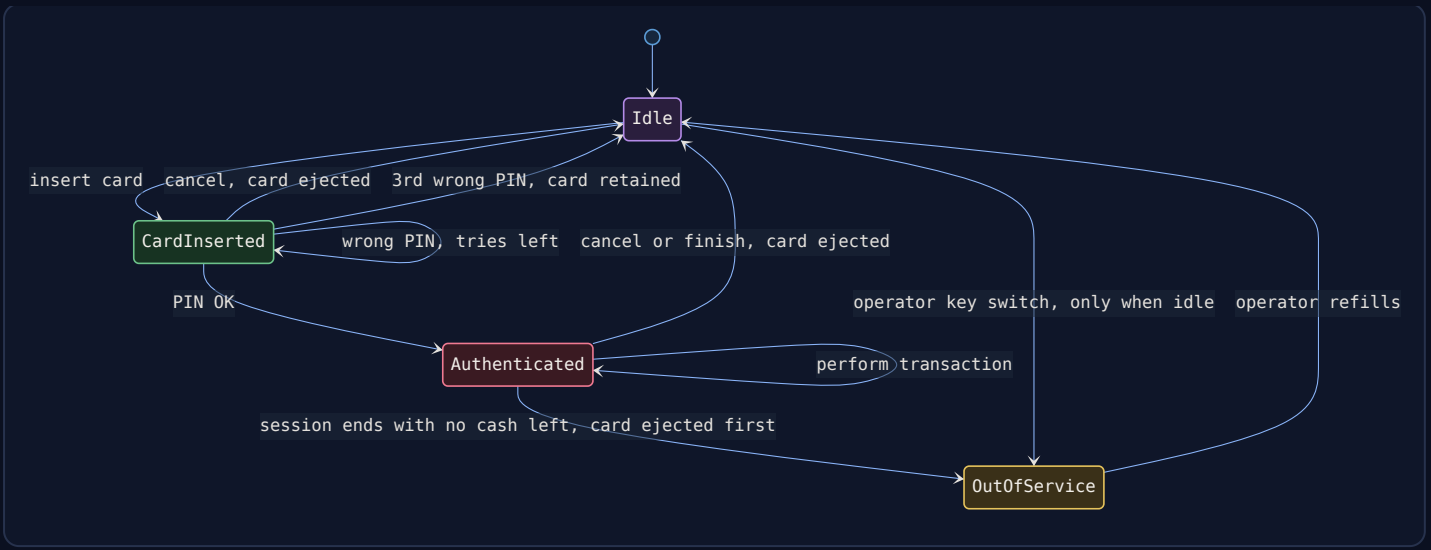
Figure — The finished class diagram of the ATM system (course diagram)

Class diagram of **our Java code**:



Key flows

Session states.



Balance inquiry (the course's sequence diagram):

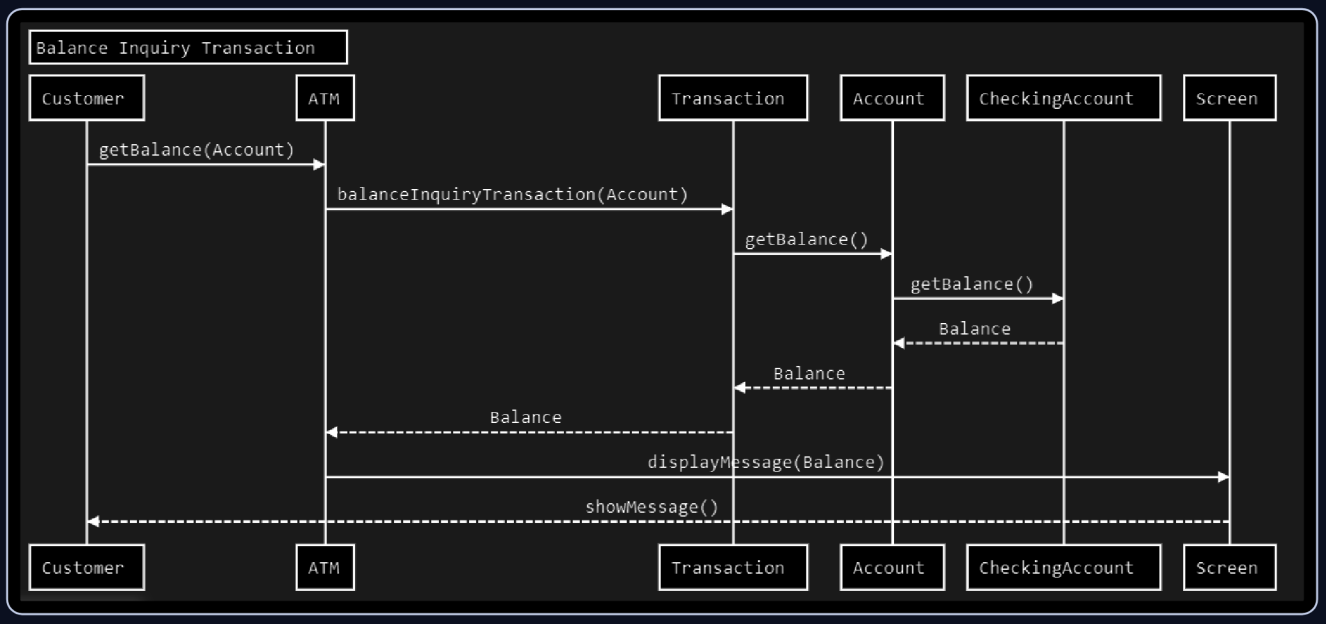
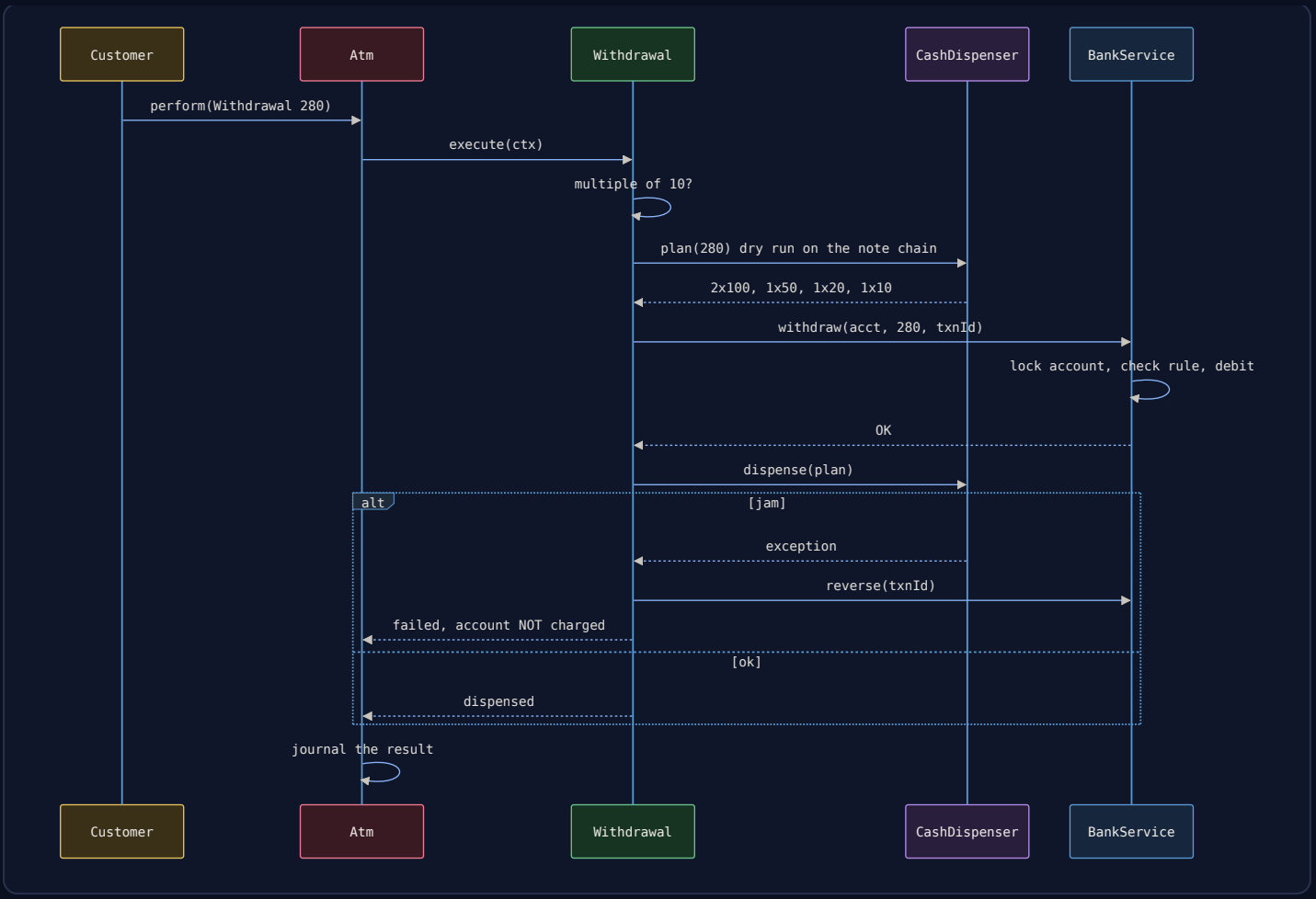


Figure — Sequence diagram of a balance inquiry: Customer → ATM → Transaction → Account → CheckingAccount, and the balance travels back to the Screen (course diagram)

In the sequence above, the ATM never reads the balance itself: it creates a transaction, the transaction asks the account, and the result is shown on the screen.

Withdrawal with compensation (our code):


Customer
Atm
Withdrawal
CashDispenser
BankService
perform(Withdrawal 280)
execute(ctx)
multiple of 10?
plan(280) dry run on the note chain
2x100, 1x50, 1x20, 1x10
withdraw(acct, 280, txnId)
lock account, check rule, debit
OK
dispense(plan)
alt
[jam]
exception
reverse(txnId)
failed, account NOT charged
[ok]
dispensed
journal the result
Customer
Atm
Withdrawal
CashDispenser
BankService

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session flow as state classes	**State**	Insert-card/enter-PIN/perform/eject each mean something different per state; e.g. `perform` in Idle is simply refused	Booleans `cardInserted`, `authenticated` and nested `if`s
Dispenser as a chain of note handlers	**Chain of Responsibility**	Each handler takes what it can and passes the rest on; adding a $5 cassette is one new handler linked into the chain	One method with a hard-coded list of notes
Plan first, then dispense	—	The chain's dry run touches nothing; a failed plan leaves the cassettes intact	Removing notes while searching
Each transaction = one class with `execute`	**Command**	The rules of each transaction live in one place; transactions can be journaled, queued or replayed	A big `switch (type)` in the ATM
`Transaction` and `AuthResult` are **sealed**	Java 17+ sealed types	The compiler knows the full list; `switch` over `AuthResult` is exhaustive	Open hierarchies with `instanceof` chains
ATM talks to a `BankService` interface	Dependency Inversion (depend on an interface, not on a concrete class)	The ATM does not care whether the bank is in-memory, ISO 8583 (the standard card-payment message format) over a network, or a test fake	ATM changes balances directly
Total vs available balance	Domain modelling	Makes the check-hold rule correct, not merely convenient	A single balance
Savings is a subclass	Inheritance for **behaviour**	Its withdrawal rule differs (daily cap), so it overrides one hook	A `type` field and `if (type == SAVINGS)`

Debit before or after dispensing? The course debits **after** the notes are counted out — the safer order for the customer — and admits the weakness: a crash between the two leaves the bank short. Our code does the opposite (debit, then dispense) and adds a **compensating action**: if the dispenser jams, `reverse(txnId)` puts the money back. Both are simplifications of what real systems do: make the pair atomic (a hold/authorisation, then a confirmation after dispensing), and **reconcile** (compare two sets of records afterwards and fix any difference) from the ATM's journal when something goes wrong in between. Say this out loud — it is the "power fails halfway through" follow-up.

Cash deposits. The course assumes all deposits are verified before they count. Many banks make cash that the machine itself has counted available immediately (or up to a limit) and hold checks. Our `depositCash` credits both balances and `depositCheck` credits only the total; changing the policy is a one-line change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`); notes are $100/$50/$20/$10. Essential lines:

```java
// Bank.java - total = everything credited; available = spendable now (a check has not cleared)
abstract class Account {
    final String id;
    final ReentrantLock lock = new ReentrantLock();   // one lock per account, shared by every ATM
    long totalCents, availableCents;
    /** Hook, called with the lock held. null = allowed, else the reason. */
    String refuseWithdrawal(long cents) { return cents > availableCents ? "insufficient funds" : null; }
    void recordWithdrawal(long cents) {}
    void reverseWithdrawal(long cents) {}
}
class CheckingAccount extends Account { /* no extra rule */ }
class SavingsAccount extends Account {              // the one behavioural difference: a daily cap
    @Override String refuseWithdrawal(long cents) {
        if (withdrawnTodayCents + cents > dailyLimitCents) return "savings daily limit " +
Money.fmt(dailyLimitCents);
        return super.refuseWithdrawal(cents);
    }
    @Override void recordWithdrawal(long cents) { withdrawnTodayCents += cents; }
    @Override void reverseWithdrawal(long cents) { withdrawnTodayCents -= cents; }
}

record Card(String number, String holder) {}
sealed interface AuthResult permits AuthResult.Ok, AuthResult.WrongPin, AuthResult.Blocked {
    record Ok(List<String> accountIds) implements AuthResult {}
    record WrongPin(int attemptsLeft) implements AuthResult {}
    record Blocked() implements AuthResult {}
}

interface BankService {                                 // the ATM never touches balances directly
    AuthResult authenticate(Card card, String pin);
    long availableBalance(String accountId);
    long totalBalance(String accountId);
    String withdraw(String accountId, long cents, String txnId);   // null = OK, else reason
    void reverse(String txnId);
    void depositCash(String accountId, long cents);   // credits total AND available
    void depositCheck(String accountId, long cents);  // credits total only: on hold
    void clearCheck(String accountId, long cents);
    String transfer(String from, String to, long cents);
}

final class InMemoryBank implements BankService {
    private static final int MAX_PIN_TRIES = 3;
    @Override public AuthResult authenticate(Card card, String pin) {
        String n = card.number();
        if (failedTries.getOrDefault(n, 0) >= MAX_PIN_TRIES) return new AuthResult.Blocked();
        if (hash(n, pin).equals(pinHashByCard.get(n))) { failedTries.remove(n); return new
AuthResult.Ok(accountsByCard.get(n)); }
        int tries = failedTries.merge(n, 1, Integer::sum);
        return tries >= MAX_PIN_TRIES ? new AuthResult.Blocked() : new AuthResult.WrongPin(MAX_PIN_TRIES - tries);
    }
    @Override public String withdraw(String id, long cents, String txnId) {
        Account a = acc(id);
        a.lock.lock();
        try {                                           // check-then-act under ONE lock: atomic
            String refusal = a.refuseWithdrawal(cents);
            if (refusal != null) return refusal;
            a.availableCents -= cents;
            a.totalCents -= cents;
            a.recordWithdrawal(cents);
            withdrawals.put(txnId, new Object[]{id, cents});   // remembered so it can be reversed
            return null;
        } finally { a.lock.unlock(); }
    }
    // reverse(txnId): compensating action - puts the money back under the same lock
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b, second = first == a ? b : a;   // fixed (id) order
        first.lock.lock();
```

```
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents; a.totalCents -= cents;
                b.availableCents += cents; b.totalCents += cents;
                return null;
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
    }
    /** Demo only: SHA-256(card + ":" + pin). Real banks never see the clear PIN (PIN pad encrypts, an HSM
verifies). */
    private static String hash(String card, String pin) { /* MessageDigest SHA-256 -> hex */ }
}
```

```
// CashDispenser.java - Chain of Responsibility: each handler takes what it can and passes the rest on
abstract class NoteHandler {
    private final long noteCents;
    private int count;
    private NoteHandler next;
    NoteHandler linkTo(NoteHandler next) { this.next = next; return next; }

    /** Dry run: fills 'plan', returns the amount nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }
    void take(Map<Long, Integer> plan) {
        count -= plan.getOrDefault(noteCents, 0);
        if (next != null) next.take(plan);
    }
}
final class HundredHandler extends NoteHandler { HundredHandler(int n) { super(100_00, n); } }
// FiftyHandler, TwentyHandler, TenHandler likewise

final class CashDispenser {
    CashDispenser(int hundreds, int fifties, int twenties, int tens) {
        head = new HundredHandler(hundreds);
        head.linkTo(new FiftyHandler(fifties)).linkTo(new TwentyHandler(twenties)).linkTo(new TenHandler(tens));
    }
    synchronized Optional<Map<Long, Integer>> plan(long cents) {      // touches nothing
        Map<Long, Integer> plan = new LinkedHashMap<>();
        return head.plan(cents, plan) == 0 ? Optional.of(plan) : Optional.empty();
    }
    synchronized void dispense(Map<Long, Integer> plan) {             // may jam: then nothing leaves
        if (jamNext) { jamNext = false; throw new IllegalStateException("note jam"); }   // fault injection
        head.take(plan);
    }
    // refill(noteCents, n), total()
}
```

```java
// Transactions.java - Command: each transaction is one record with one execute(); sealed = full list known
record TxContext(BankService bank, CashDispenser dispenser, String txnId) {}
sealed interface Transaction permits BalanceInquiry, Withdrawal, CashDeposit, CheckDeposit, Transfer {
    String account();
    String execute(TxContext ctx);
}

record Withdrawal(String account, long cents) implements Transaction {
    public String execute(TxContext c) {
        if (cents <= 0 || cents % 10_00 != 0) return "Withdrawal refused: amount must be a multiple of $10";
        var plan = c.dispenser().plan(cents);                                  // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());         // 2. atomic debit at the bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                       // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }
}
// BalanceInquiry, CashDeposit, CheckDeposit ("on hold until the bank clears it"), Transfer: one bank call each
```

```java
// States.java - default methods refuse; each state overrides only the buttons it accepts
interface AtmState {
    default void insertCard(Atm atm, Card card) { atm.screen("Card slot is not ready."); }
    default void enterPin(Atm atm, String pin) { atm.screen("Please insert your card first."); }
    default void perform(Atm atm, Transaction tx) { atm.screen("Please sign in first."); }
    default void ejectCard(Atm atm) { atm.screen("No card to eject."); }
}
final class CardInsertedState implements AtmState {
    @Override public void enterPin(Atm atm, String pin) {
        switch (atm.bank().authenticate(atm.card(), pin)) {                    // exhaustive over the sealed type
            case AuthResult.Ok ok -> {
                atm.startSession(ok.accountIds());
                atm.screen("PIN OK. Accounts: " + ok.accountIds());
                atm.setState(AuthenticatedState.INSTANCE);
            }
            case AuthResult.WrongPin w -> atm.screen("Wrong PIN. Tries left: " + w.attemptsLeft());
            case AuthResult.Blocked b -> {
                atm.screen("Too many wrong PINs. Card retained. Contact your bank.");
                atm.retainCard();
                atm.finishSession();
            }
        }
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
}
final class AuthenticatedState implements AtmState {
    @Override public void perform(Atm atm, Transaction tx) {
        if (!atm.sessionAccounts().contains(tx.account())) {                   // only this card's accounts
            atm.screen("Account " + tx.account() + " is not linked to this card.");
            return;
        }
        atm.run(tx);
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
}
// IdleState.insertCard -> CardInserted; OutOfServiceState.insertCard -> "Out of service. Card returned."
```

```java
// Atm.java - owns its parts, one session at a time, forwards buttons to its state, keeps the journal
final class Atm {
    private final List<String> journal = new ArrayList<>();   // operator / bank-manager audit log
    private AtmState state = IdleState.INSTANCE;
    private Card card;                                        // the card reader's slot

    synchronized void insertCard(Card c) { state.insertCard(this, c); }       // enterPin, perform, ejectCard
alike

    synchronized boolean setOutOfService() {                  // key switch: never while serving a customer
        if (state != IdleState.INSTANCE && state != OutOfServiceState.INSTANCE) {
            journal("operator: switch-off refused, session in progress");
            return false;
        }
        state = OutOfServiceState.INSTANCE;
        return true;
    }
    synchronized void refill(long noteCents, int n) {
        dispenser.refill(noteCents, n);
        if (state == OutOfServiceState.INSTANCE) state = IdleState.INSTANCE;
    }
    void run(Transaction tx) {
        String txnId = id + "-" + TXN_SEQ.getAndIncrement();
        String result = tx.execute(new TxContext(bank, dispenser, txnId));
        screen(result);
        journal(txnId + " " + tx.getClass().getSimpleName() + ": " + result);
    }
    void finishSession() {                                    // after every eject/retain: card is already out
        if (dispenser.total() == 0) { state = OutOfServiceState.INSTANCE; journal("cash empty: out of service"); }
        else state = IdleState.INSTANCE;
    }
    // journal also records "card in/out ****1111" (masked) and "card RETAINED"
}
// Main.java: Alice (CHK-1 $1000, SAV-1 $2000 with a $100 daily cap), Bob (CHK-2 $50); a jam injected;
// two ATMs race $300 each on a $500 JOINT account; 2,000 opposite A<->B transfers on 8 threads.
```

The demo PIN hash is SHA-256, a standard one-way function producing a fixed-length fingerprint that cannot practically be reversed. (The course compares the PIN inside `Card` and flags that as a simplification. Real banks never see the clear PIN: the PIN pad encrypts it into a PIN block and a hardware security module (HSM) verifies it; a plain hash of a 4-digit PIN is also easy to brute-force — try every value until one matches.) Transfers lock both accounts in **id order**, so opposite transfers (A→B, B→A) can't deadlock (Chapter 13). The bank refuses zero or negative transfers and deposits.

Output:

```
Cash loaded: $430.00

== Alice: wrong PIN, then right PIN ==
   [ATM-7 screen] Please sign in first.
   [ATM-7 screen] Hello Alice. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   [ATM-7 screen] Balance CHK-1: available $1000.00, total $1000.00
   [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   [ATM-7 screen] Withdrawal refused: amount must be a multiple of $10
   [ATM-7 screen] Withdrawal refused: savings daily limit $100.00
   [ATM-7 screen] Check $500.00 received: on hold until the bank clears it
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Transferred $150.00 to CHK-2
   [ATM-7 screen] Account CHK-2 is not linked to this card.
   (dispenser jams on the next withdrawal)
   [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Card ejected. Goodbye.

== Bank clears the check overnight ==
   CHK-1 available now $1220.00

== Bob forgets his PIN three times ==
   [ATM-7 screen] Hello Bob. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] Wrong PIN. Tries left: 1
   [ATM-7 screen] Too many wrong PINs. Card retained. Contact your bank.
   [ATM-9 screen] Hello Bob. Enter PIN.
   [ATM-9 screen] Too many wrong PINs. Card retained. Contact your bank.

== Two ATMs, one account, same instant: $300 each from $500 ==
   cash handed out $300.00, JOINT left $200.00

== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
   successful=2000, A+B=$2000.00

== ATM-9 runs out of cash; the operator refills it ==
   [ATM-9 screen] Hello Alice. Enter PIN.
   [ATM-9 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   operator key switch mid-session accepted? false
   [ATM-9 screen] Dispensed $180.00 as 1x$100 + 1x$50 + 1x$20 + 1x$10
   state after the last notes: AUTHENTICATED (card still inside)
   [ATM-9 screen] Card ejected. Goodbye.
   state after the session: OUT_OF_SERVICE
   [ATM-9 screen] Out of service. Card returned.
   state after refill: IDLE, cash $200.00

== Journal of ATM-7 (for the operator and bank manager) ==
   card in ****1111
   ATM-7-1 BalanceInquiry: Balance CHK-1: available $1000.00, total $1000.00
   ATM-7-2 Withdrawal: Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   ATM-7-3 Withdrawal: Withdrawal refused: amount must be a multiple of $10
   ATM-7-4 Withdrawal: Withdrawal refused: savings daily limit $100.00
   ATM-7-5 CheckDeposit: Check $500.00 received: on hold until the bank clears it
   ATM-7-6 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   ATM-7-7 Transfer: Transferred $150.00 to CHK-2
   ATM-7-8 Withdrawal: Withdrawal failed (note jam), account NOT charged
   ATM-7-9 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   card out ****1111
   card in ****2222
   card RETAINED ****2222
```

Read the output: $280 = 2×$100 + $50 + $20 + $10; $85 is refused; the savings cap refuses $140; after the $500 check *total* grows but *available* waits for clearing; the jam leaves the balance unchanged; Bob's card is blocked at the bank, so another ATM refuses even the right PIN; two ATMs racing on a $500 joint account (one account shared by two people) hand out only $300; 2,000 opposite transfers finish with no deadlock and the total conserved; ATM-9 refuses the key switch mid-session, pays its last $180, ejects the card, and only then goes out of service until refilled.

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs withdraw from one account at once	Overdraft (check-then-act race)	Check and debit under the **account lock** at the bank
A→B and B→A transfers at once	**Deadlock** (each holds one lock, waits for the other)	Lock in a global order (account id)
Dispenser jams after the debit	Customer charged, no cash	Compensating `reverse(txnId)`; journal entry for reconciliation
Amount not makeable from loaded notes	Partial dispense	Dry-run `plan` first; refuse before touching the bank
Greedy chain misses a combination	e.g. $60 with one $50 and three $20 but no $10: greedy takes $50, then cannot make $10, though 3×$20 works	Acceptable for standard cassettes; mention a DP planner (as in 14.2) if asked
Wrong PIN many times	Brute force	3 tries, then block at the bank (so another ATM also refuses) and retain the card
Customer walks away mid-session	Next person uses the open session	Session timeout that ejects/retains the card (extension)
Transaction on an account not linked to the card	Unauthorised access	`AuthenticatedState` checks `sessionAccounts`
Cash runs out	Customer's card trapped, or the next customer gets refused mid-session	The session finishes normally (card ejected), then `finishSession` moves the ATM to `OutOfService`; an operator refill returns it to Idle
Operator turns the key mid-session	The machine shuts down while serving	`setOutOfService` is refused unless the ATM is idle (requirement: never shut down while serving)

Extensions & follow-up questions

1. **"What if the machine cannot make the amount?"** — The dispenser's plan fails and the withdrawal is refused before the bank is called.
2. **"What if the power fails halfway through a withdrawal?"** — Use a two-phase approach: authorise (hold) at the bank, dispense, then confirm; on restart the ATM reads its journal and either confirms or reverses. The bank reconciles against the journal.
3. **"Add a daily withdrawal limit for all accounts."** — Move the cap into `Account` or into a policy object the bank consults; the ATM does not change.
4. **"Receipts and PIN change?"** — Two more `Transaction` records (`PinChange`, and a `Printer` part that formats the journal entry).
5. **"Other banks' cards?"** — `BankService` gets a routing implementation that forwards to a card network (such as Visa or Mastercard, which passes requests between banks) for foreign cards (a Proxy — a stand-in object — or an Adapter — a translator between two interfaces — in front of the network).
6. **"Cardless / mobile withdrawals?"** — A new state path: `enterOneTimeCode` instead of `insertCard` → `enterPin`. The transactions do not change.
7. **"Bank manager reports?"** — Aggregate the journal (total deposits, total withdrawals, cash left = `dispenser.total()`).

Real-world example

Real ATMs (NCR, Diebold Nixdorf and others) control their devices through the **CEN/XFS** standard, which gives the card reader, PIN pad, dispenser and printer each a standard software interface — the same "parts behind interfaces" idea as our design. Messages between ATM networks and banks commonly use the **ISO 8583** message format, and PIN blocks are defined by **ISO 9564** and verified inside HSMs. Cash sits in

cassettes of one denomination each, and the dispenser works out a note mix from what the cassettes hold, like our handler chain. After a dispense error, ATMs routinely reverse the transaction, and banks reconcile disputed withdrawals from the machine's electronic journal.

In the interview

Plan for 45 minutes: 5 min parts, actors and transactions → 5 min total vs available balance → 10 min state diagram (Idle → CardInserted → Authenticated) → 15 min code: `Withdrawal.execute` (plan → debit → dispense → compensate) and the dispenser chain → 10 min concurrency (account lock, ordered transfer locks) and follow-ups.

30-second answer: "The ATM has parts behind interfaces, a session modelled with the State pattern, and five transactions as Command objects with one `execute` each. It never touches balances: it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans the notes first through a Chain of Responsibility over the cassettes, debits the account atomically at the bank, dispenses, and reverses the debit if the hardware fails. The bank locks per account, and transfers lock in id order to avoid deadlock."

Follow-ups with answers

- *"Where is the PIN checked?"* — At the bank (in reality in an HSM, from an encrypted PIN block). The ATM never stores or compares it.
- *"How do you stop two ATMs overdrawing a joint account?"* — The check and the debit happen together under the account's lock at the bank (or in one database transaction).
- *"Why Chain of Responsibility for notes?"* — Each denomination handler decides how much it can cover and passes the rest on; adding or removing a cassette changes the chain, not the algorithm.

TL;DR

- **State** for the session, **Command** for the five transactions, **Chain of Responsibility** for the note cassettes, `BankService` interface for the bank.
- **Total vs available** balance makes check deposits correct.
- Withdrawal = **plan notes** → **atomic debit at the bank** → **dispense** → **reverse on failure**, all journaled.
- Concurrency: **per-account lock**, transfers lock in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System



Figure — An elevator bank in a multi-floor building: several cars, hall buttons on each floor and displays (course diagram)

An **elevator** (lift) moves people up and down a building. A real building rarely has just one, and its cars are run as a **group** by a shared controller. That group control is what turns this from a small coding task into a design question. Two decisions carry the interview: **which car should answer a call from a floor** (dispatching) and **in what order a single car should visit its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
How many floors and cars?	Many floors (0 … N-1), several cars serving all floors.
Which buttons?	Hall panel on each floor with UP and DOWN (no UP on the top floor, no DOWN on the ground floor); a car panel with one button per floor plus door-open and door-close.
Displays?	Inside each car: floor, direction, load. On each floor: one display per car showing where it is.
Scheduling rule?	A car serves every stop ahead in its direction before turning (the elevator algorithm; we use **LOOK**).
Dispatching rule?	Nearest suitable car; swappable.
Capacity?	Each car has a maximum load; an overloaded car does not move and says so.
Maintenance and emergencies?	A car can be taken out of service; fire alarm recalls all cars to the lobby.
Real time or simulated?	Simulated: one `step()` = one action. This makes the design testable without a clock.

Requirements

Functional (the course's twelve, plus emergency)

1. The building has **multiple floors** and **more than one elevator** serving all of them.
2. Each floor has a **hall panel** with UP and DOWN buttons (top floor: no UP; ground floor: no DOWN).
3. Each car has an **inside panel** with a button per floor, plus **door open** and **door close**.
4. Each car has a **display** showing its floor, direction and the weight it carries.
5. Each floor has a **display for every elevator** so a waiting rider can see where each car is.
6. When a rider presses a hall button, the system **picks one elevator** and sends it.
7. When a rider presses a floor button inside a car, the car **adds that floor** to its stops.
8. A car **serves the stops ahead of it** in its direction before turning around.
9. A car **does not move while its doors are open**.
10. Each car has a **maximum load**; an overloaded car does not move and **says so** on its display.
11. A car with no stops left **waits on the floor where it finished**.
12. An elevator can be **taken out of service** for maintenance; the system must not send riders to it.
13. *(Extension the course leaves out; we add it)* **Fire recall**: all cars go to the lobby, open their doors and ignore car calls.

Non-functional

- **Efficiency:** low average wait and ride time; no car runs to the end of the shaft for nothing.
- **Fairness / no starvation** (starvation = a request that waits forever because others keep being chosen first): every accepted call is eventually served.
- **Safety:** doors, load limit and emergency modes always override scheduling.
- **Thread safety:** riders press buttons concurrently while a controller thread moves the cars.
- **Testability:** deterministic stepping (the same inputs always give the same results).

Use cases

Actors: Rider (calls a car, rides, chooses a destination), **Maintenance engineer** (takes a car out of service and back), **System** (chooses the car, moves cars, opens/closes doors, updates displays).

Top use cases: call an elevator · choose a destination · dispatch a car · move one floor · open and close the doors · report an overload · take a car out of service.

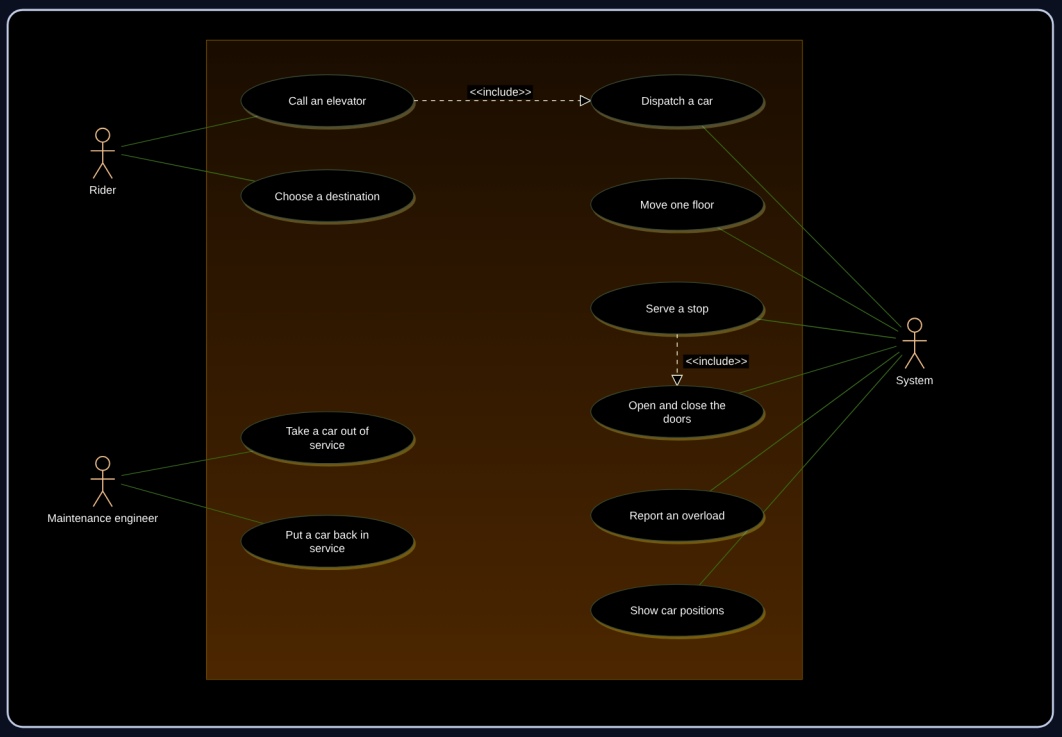


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

In the figure above, "dispatch a car" and "move one floor" belong to the System — the rider only triggers them.

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	A floor **plus** the direction the car will travel when it leaves — two riders on floor 5 (one up, one down) are two stops.
`StopList`	The accepted stops in **two sorted sets** (`up` , `down`); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	One car: floor, direction, mode, load, door, stop list. `step()` does exactly one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What one step did (moved, arrived, doors closed, overload, nothing).
`DispatchStrategy`	*Which car* answers a hall call. `NearestCarStrategy` is one rule.
`Dispatcher`	Holds the cars and one strategy; queues calls no car can take now.
`Building`	Owns hall-button lights, the cars and the dispatcher; ticks every car; clears hall lights on arrival; fire alarm.

The course also models the physical parts one-to-one — `Button` (`HallButton` , `ElevatorButton` , `DoorButton`), `HallPanel` , `ElevatorPanel` , `Display` (`ElevatorDisplay` , `FloorDisplay`) and `Floor` . Our code keeps their behaviour (lights, displays, door) in fewer classes; either level of detail is fine in 45 minutes if you say which you chose.

Class diagram

The course builds the design in five stages:

Stage	What it adds	What works after it
1	`Button` family and `Door`	Buttons light and reset; a door knows if it is open
2	`HallPanel`, `ElevatorPanel`, `Display` family, `Floor`	A floor accepts a call and shows each car's position
3	`Stop` and `StopList`	Given a floor and a direction, compute the next stop
4	`Elevator`	A car travels to all the stops it has accepted, one step at a time
5	`DispatchStrategy`, `Dispatcher`, `Building`	A call on floor 7 is handed to the most suitable car

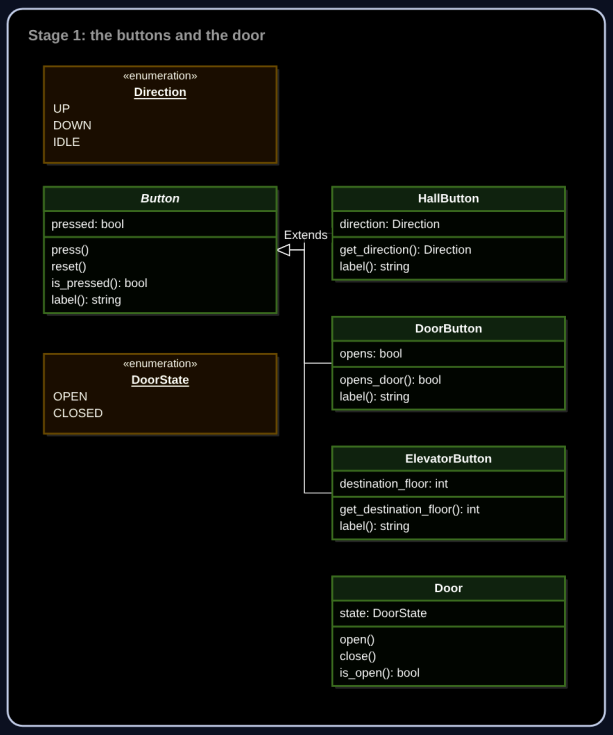


Figure — Stage 1: the Direction enum, the abstract Button with HallButton, ElevatorButton and DoorButton, and the Door (course diagram)

Stage 1. Requirements 2, 3 and 9. All buttons share one behaviour (light up on press, go dark once handled) and differ only in the value they carry: a hall button stores a **direction**, a car button a **floor number**, a door button **open or close**. That gives one abstract `Button` parent with three subclasses

instead of three unrelated classes. `Door` has no owner yet; the car will own it in stage 4. Nothing can move at this point.

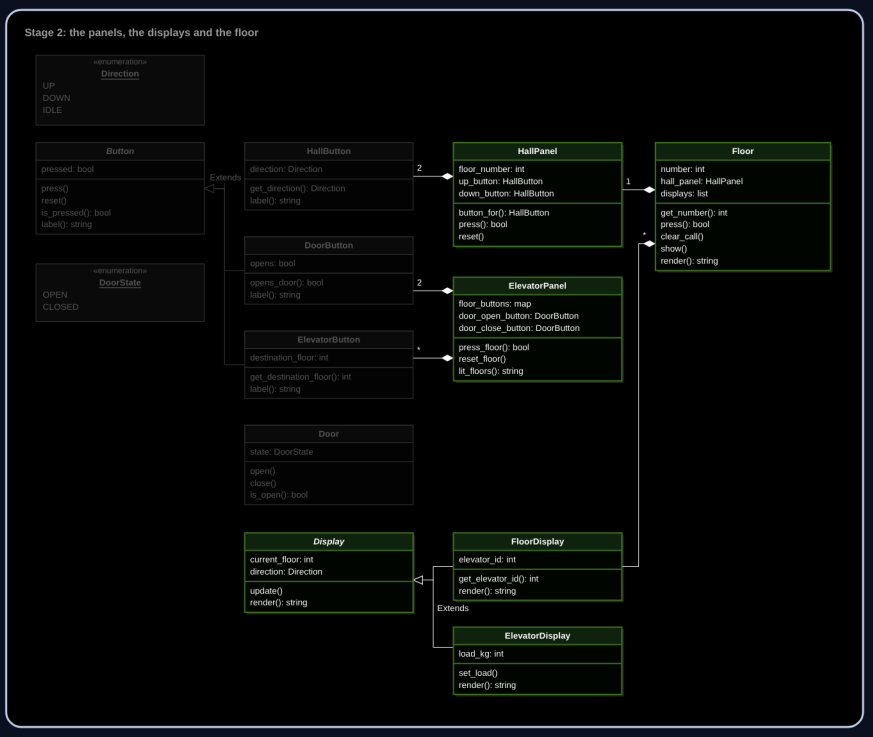


Figure — Stage 2: HallPanel, ElevatorPanel, the abstract Display with ElevatorDisplay and FloorDisplay, and Floor, all joined by composition (course diagram)

Stage 2. Requirements 2, 3, 4 and 5. Every new link is **composition**: a hall panel owns its buttons, a floor owns its hall panel and displays. A hall panel has **at most two** buttons, each optional (top and bottom floors lack one). The car panel creates one button per floor from the floor list it receives, so adding a floor needs no edit here. `Display` is abstract: both subclasses show a floor and a direction; `ElevatorDisplay` adds the load, the wall-mounted `FloorDisplay` adds which car it describes — either can render wherever a `Display` is expected (the substitution test, Chapter 4). A floor can now register a call and show where cars are, but nothing moves yet.



Figure — Stage 3: the Stop data type and the StopList with its up and down sets (course diagram)

Stage 3 — the most important stage. Requirement 8: *knowing a car's floor and direction, which floor should it stop at next?* Store stops in **two sets** (to serve going up, going down), not one list: with two

people waiting on floor 5, one going up and one down, a plain list records "5" once, so the car stops twice for nothing or picks up the wrong person. `Stop` is a small data type (a floor plus the direction the car will be moving there). The next stop returned is always reachable **without reversing**; when nothing lies ahead, the car heads to the **farthest** waiting request, sweeping everyone up on the return trip. `Elevator` **moves**, `StopList` **chooses**.

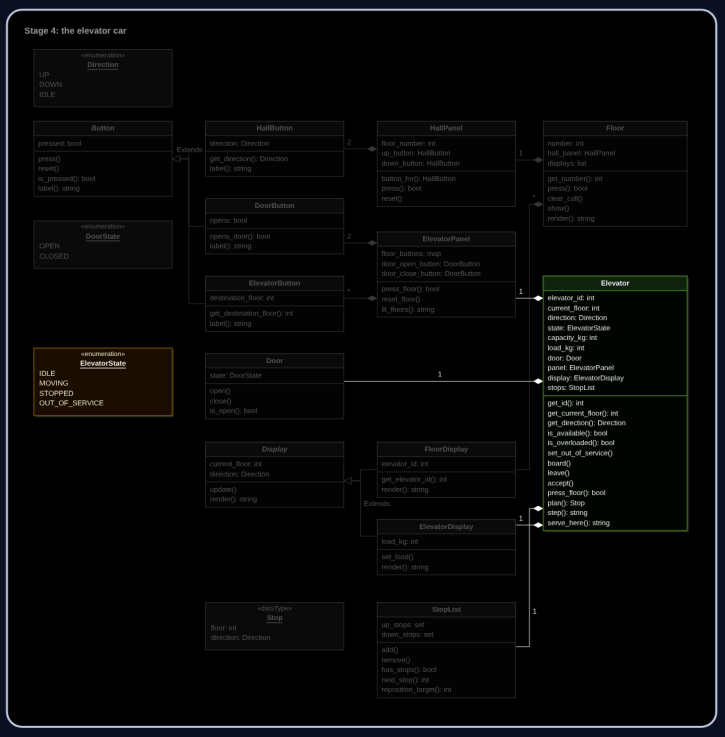


Figure — Stage 4: the Elevator owns its door, panel, display and stop list (course diagram)

Stage 4. Requirements 7, 9, 10 and 11. The car owns its door, panel, display and stop list (composition). One `step` does exactly **one** of four things — close the doors, refuse to move because of overload, move one floor, or stop and open — so a test can drive the car step by step with **no clock**. Only `step` changes the position, and an out-of-service car simply does nothing on `step` (our `NOTHING` event), so it drops out of the rounds without special cases. The car never touches a `Floor` or another car, leaving the choice of car to the dispatcher.

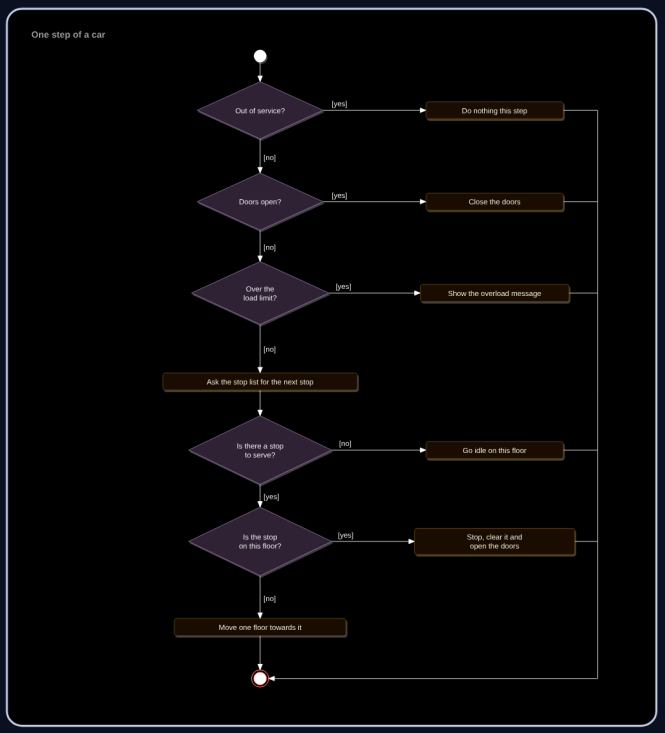


Figure — Activity diagram "One step of a car": out of service → nothing; doors open → overload check or close; otherwise ask the stop list, go idle, stop and open, or move one floor (course diagram)

The activity diagram above is our `Elevator.step()` drawn as a picture: every path ends in exactly one outcome, and only one outcome moves the car.

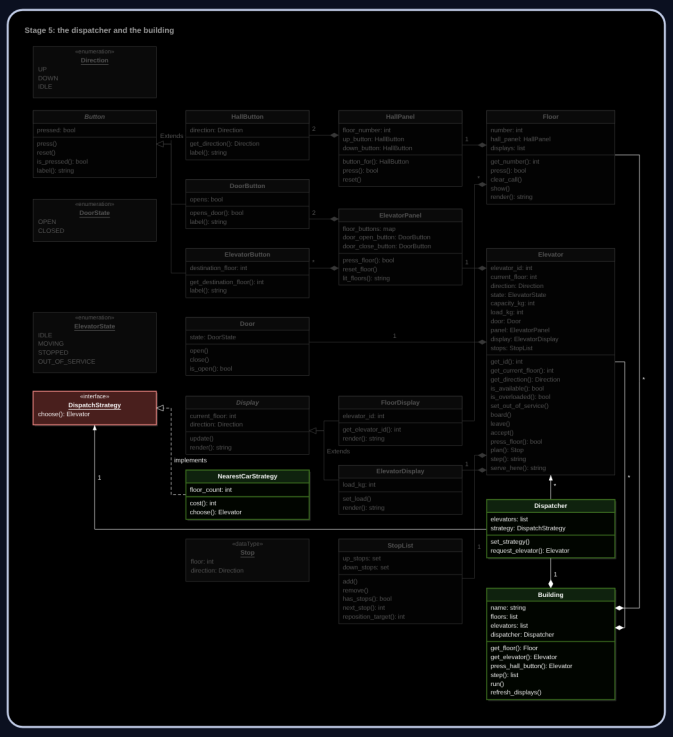


Figure — Stage 5: DispatchStrategy interface with NearestCarStrategy, the Dispatcher, and the Building that owns floors, cars and the dispatcher (course diagram)

Stage 5. Requirements 1, 6 and 12. The rule that picks a car lives **behind an interface** (`DispatchStrategy`); the dispatcher depends only on it, and `NearestCarStrategy` implements it. Morning rush in an office and a quiet hotel night want different rules; swapping touches one class. The building owns its floors, cars and dispatcher, and is the only class that knows both cars and floors — so switching off the hall-button light when a car opens its doors is its job.

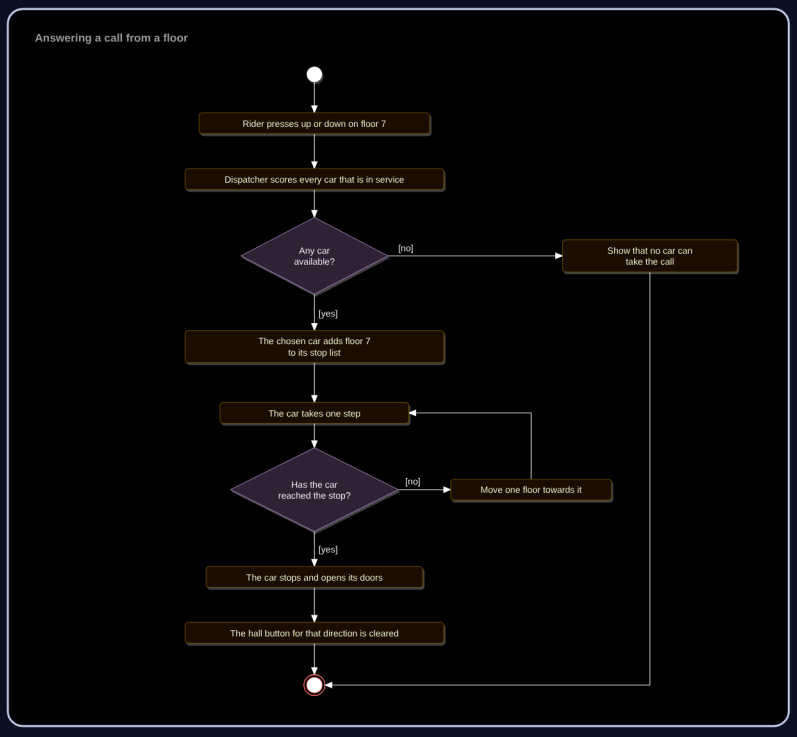


Figure — Activity diagram "Answering a call from a floor": rider presses up/down on floor 7, the dispatcher scores every in-service car, the chosen car adds floor 7, steps until it reaches the stop, opens its doors, and the hall button is cleared (course diagram)

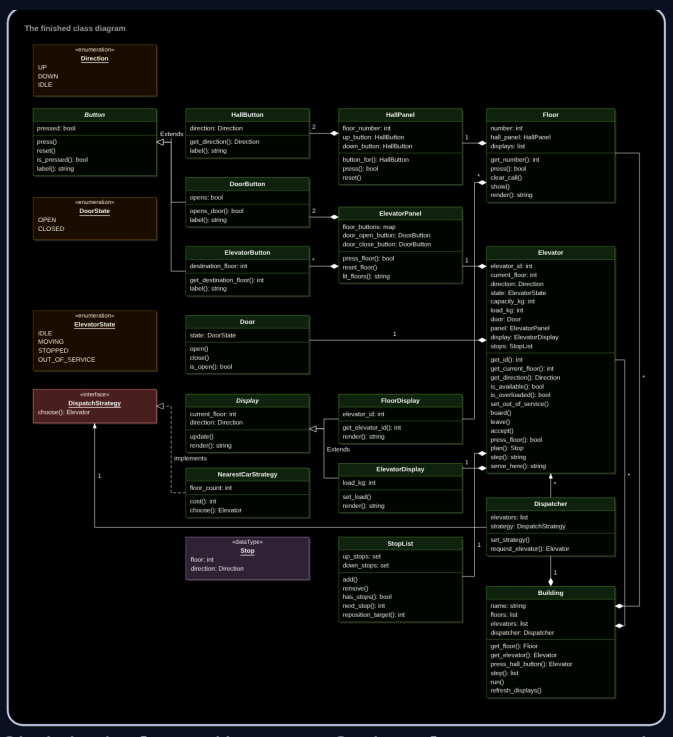
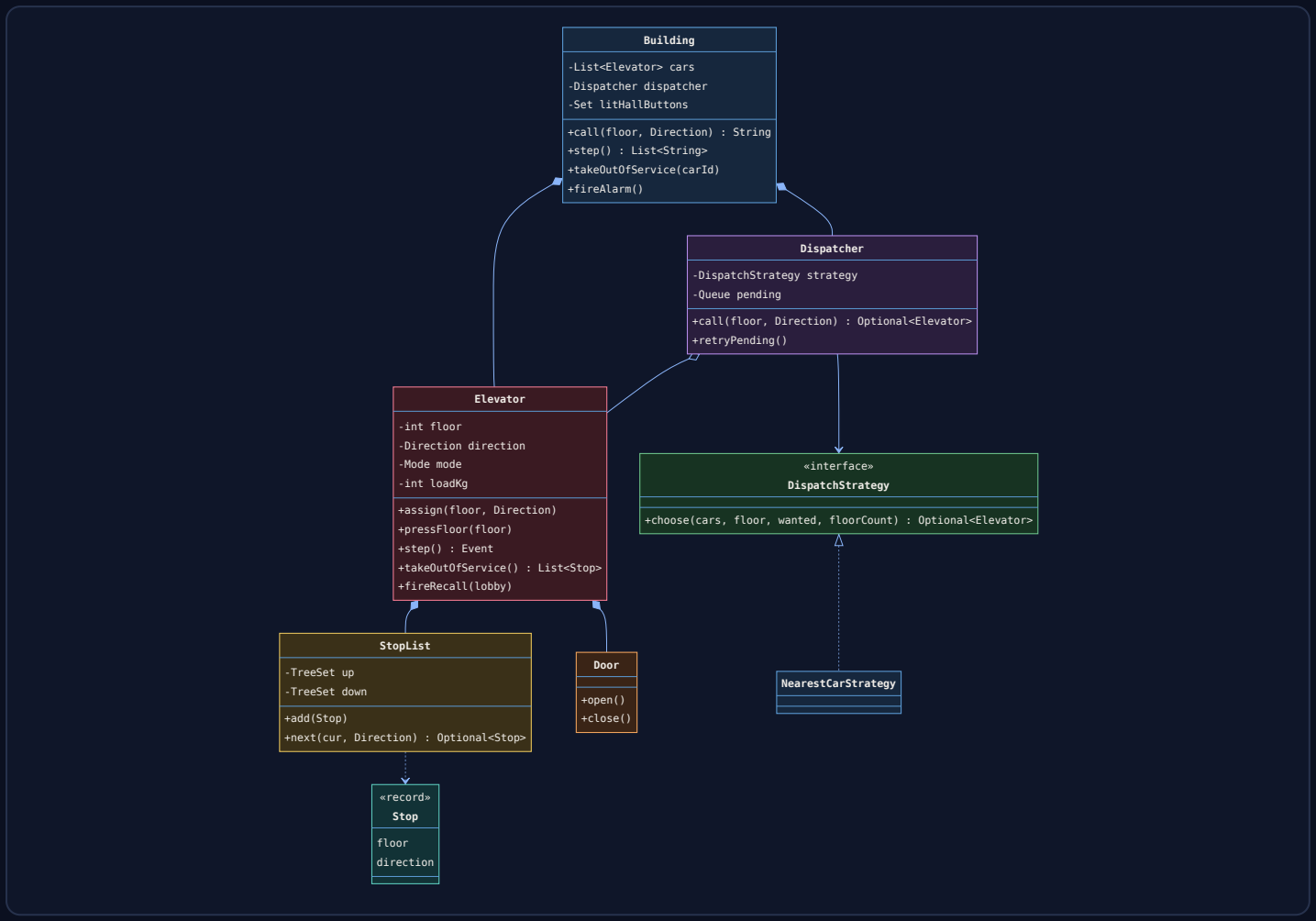


Figure — The finished class diagram of the elevator system (course diagram)

Class diagram of **our Java code**:



Key flows

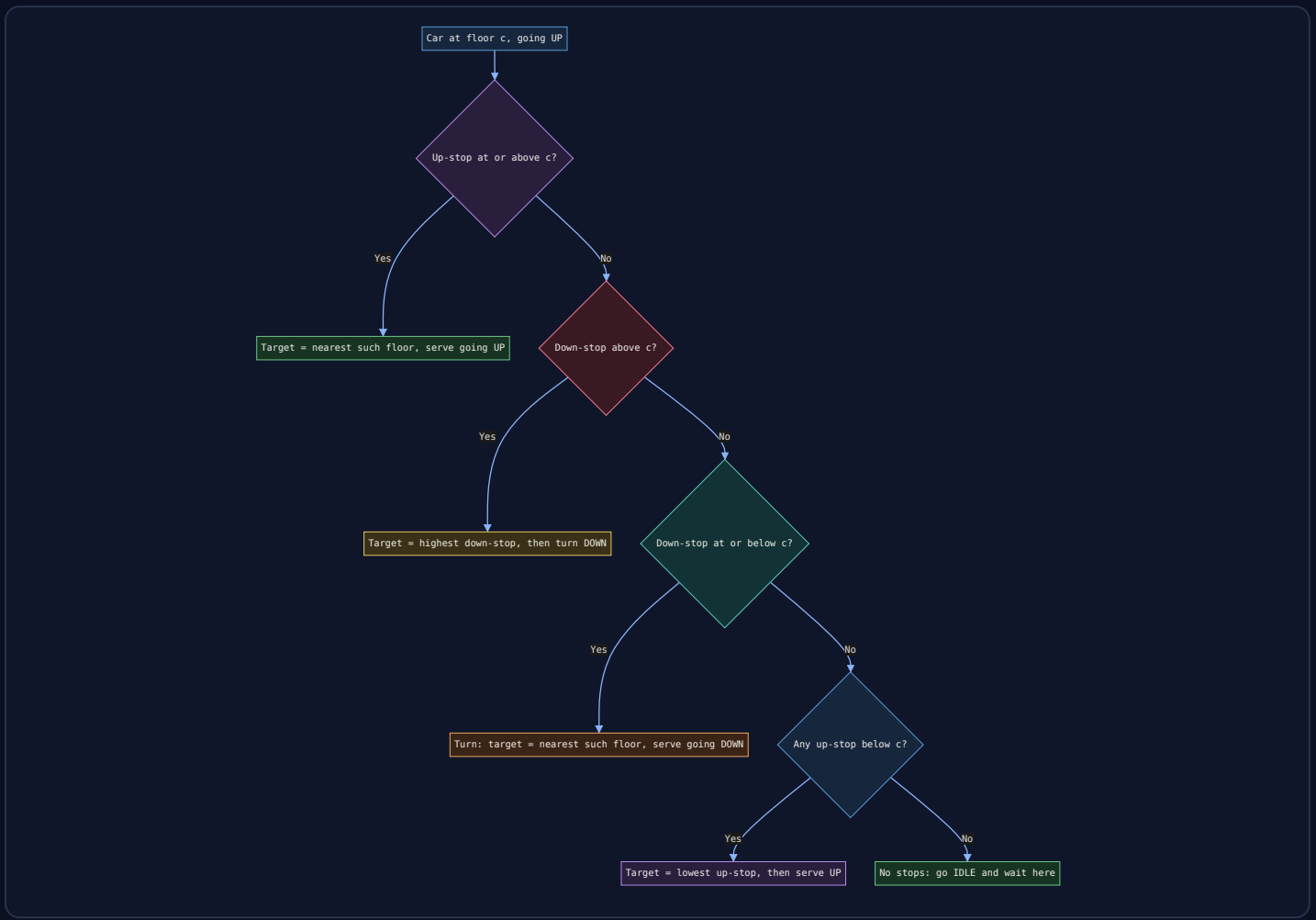
Scheduling algorithms compared. "Scheduling" means choosing the order in which one car serves its stops. These algorithms come from disk-arm scheduling in operating systems (the order in which a hard

disk's read head visits requested tracks), which has the same shape (a head moving along a line of requests).

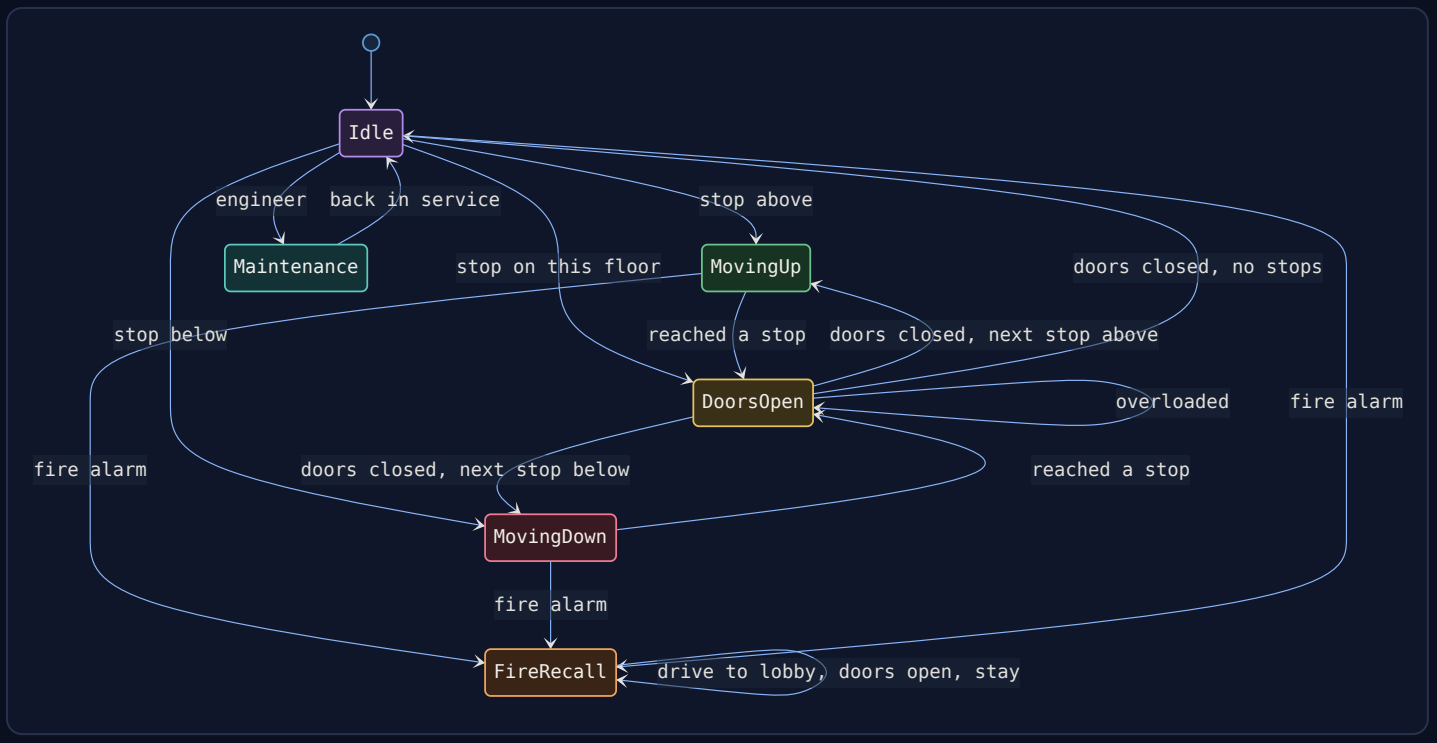
Algorithm	Rule	Good	Bad
FCFS (first come, first served)	Serve calls in arrival order	Simple, fair in order	Zig-zags up and down; very inefficient
SSTF (shortest seek time first)	Always go to the nearest stop	Short average trips	**Starvation**: a far floor may wait forever while nearby calls keep coming
SCAN ("elevator algorithm")	Sweep in one direction to the **end of the shaft**, then reverse	No starvation, predictable	Wastes travel going to the top/bottom floor even when nobody is there
LOOK	Sweep in one direction only **as far as the last request**, then reverse	No starvation, no wasted travel	Slightly more bookkeeping
C-SCAN / C-LOOK	Serve in one direction only; jump back to the start	More uniform waits	Empty return trips

Our `StopList.next` is **LOOK** with direction-aware stops:

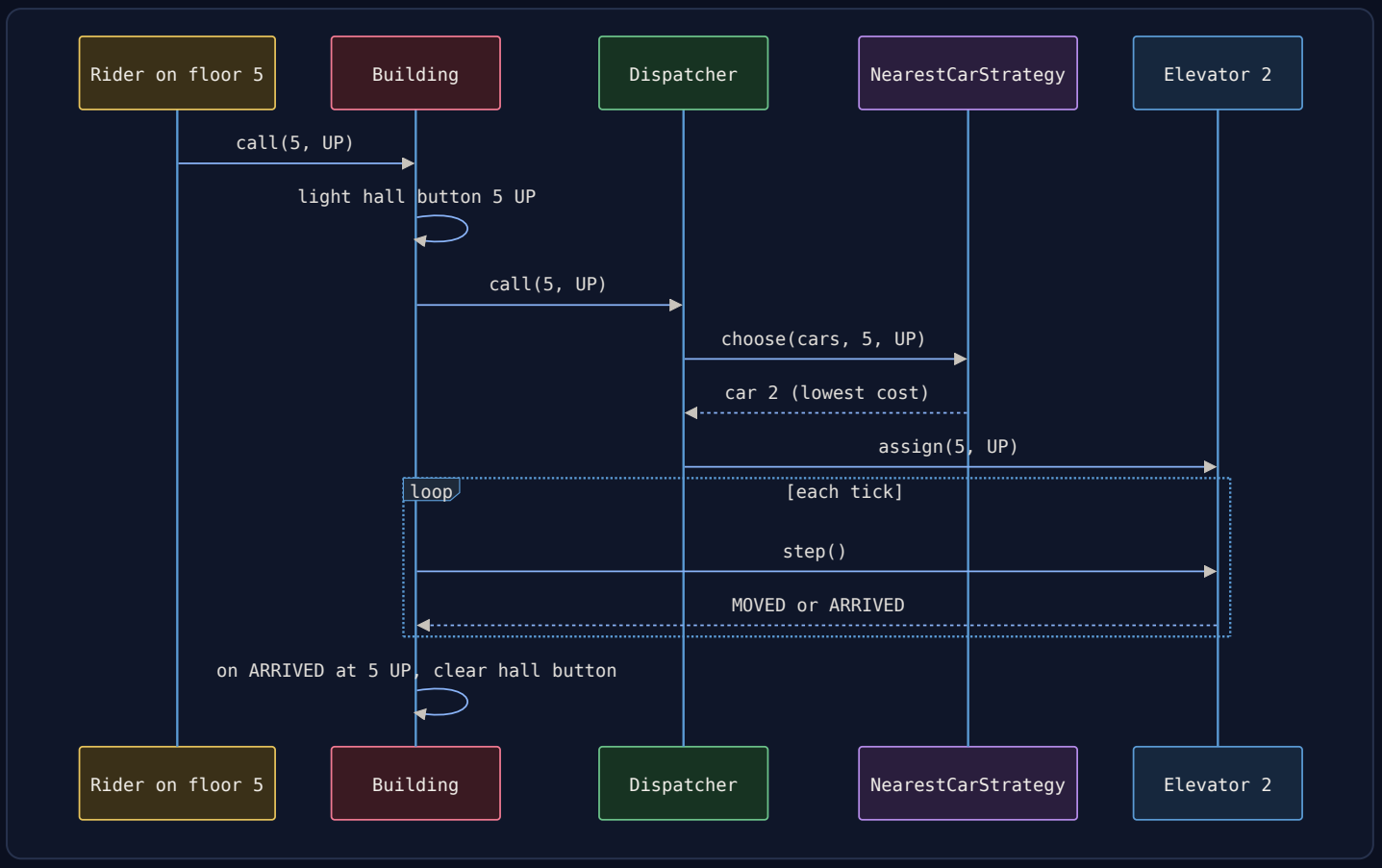
1. Going **UP**: the nearest up-stop at or above the car (`up.ceiling(cur)`).
2. None? If someone *above* wants to go **down**, go up to the **highest** such floor, then turn (collect the rest on the way down).
3. Otherwise turn: the nearest down-stop at or below the car (`down.floor(cur)`).
4. Otherwise the only work left is up-stops **below** the car: go down to the **lowest** one, then serve upward.
5. Going **DOWN** is the mirror image. An **IDLE** car picks the side of the nearest stop.



Car states.



A hall call.



Dispatch cost (`NearestCarStrategy` , from the course): an **idle** car costs its distance; a car **already coming this way in the same direction** costs its distance; any other car pays distance **+ two building heights**, so it wins only when nothing better is free. Cars out of service are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Choosing a car is a `DispatchStrategy`	**Strategy**	Different buildings and times of day want different rules; the course admits one implementation normally doesn't justify an interface, but here the *requirement* (swap the rule) does	`if (officeMode) … else …` in the dispatcher
Sweep rule in `StopList`, not in `Elevator`	Single Responsibility	The car drives; the stop list decides. Without this split, `Elevator` would do two jobs	Next-stop logic inside `step()`
Two sets of stops (up/down)	Data modelling	Distinguishes "5 going up" from "5 going down"	One list of floors
`step()` = one action	Deterministic simulation	Every scenario is a list of calls and an expected list of events; no clock, no flakiness (tests that pass or fail at random)	A thread per car with `sleep`
Modes as an enum inside the car	Lightweight State	Only three modes and a few checks; full State classes would add more code than they save here. If modes grew (fire phase II, independent service, VIP), switch to the State pattern	State classes from day one
Hall calls re-dispatched when a car leaves service	Robustness	A call accepted by a car must not be lost when that car is taken out	Silently clearing the car's stops
Queue calls when no car is available	—	"No car can take the call right now" is not the same as "rejected"	Dropping the call

The course names three weak points worth saying: (1) one step per call means **no threads** — a real controller runs each car on its own thread and needs a lock around the stop list (our methods are `synchronized` for that); (2) the dispatcher **decides once** and never re-evaluates, though real systems do (why a busy building sometimes changes your car number); (3) `Elevator` nearly does two jobs, hence the sweep rule in `StopList`. The course names only **one** pattern here (Strategy): `Button` and `Display` are ordinary abstract bases and `StopList` is Single Responsibility. Its scope leaves out emergency calls, fire service mode, express cars and door sensors that reopen for a hand (we add fire recall; the rest are extensions below).

Java implementation

Package `lld.ch14.elevator`. Essential lines:

```java
// StopList.java - LOOK over TWO sorted sets; a TreeSet's ceiling/floor/first/last answer
// "nearest stop at or above/below" in O(log n)
enum Direction { UP, DOWN, IDLE }
record Stop(int floor, Direction direction) {}        // floor + direction the car leaves in

final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();
    void add(Stop s) { (s.direction() == Direction.DOWN ? down : up).add(s.floor()); }
    void remove(Stop s) { (s.direction() == Direction.DOWN ? down : up).remove(s.floor()); }
    // isEmpty(), clear(), drain() (removes and returns every stop - used when a car leaves service)

    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);   // idle: the side of the nearest stop
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }
    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                                    // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then
turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                                  // turn around now
        return new Stop(up.first(), Direction.UP);                                          // down to lowest
up-call
    }
    private Stop goingDown(int cur) {                                                       // mirror image
        Integer f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);
        if (!up.isEmpty() && up.first() < cur) return new Stop(up.first(), Direction.UP);
        f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);
        return new Stop(down.last(), Direction.DOWN);
    }
}
```

```java
// Elevator.java - it drives, its StopList decides; one step() = one action; synchronized because
// riders press buttons from other threads while the controller thread calls step()
enum Mode { NORMAL, MAINTENANCE, FIRE_RECALL }
record Event(Kind kind, int car, int floor, Direction direction, String note) {
    enum Kind { MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING }
}

final class Elevator {
    synchronized void assign(int f, Direction wanted) { stops.add(new Stop(f, wanted)); }   // from the dispatcher
    synchronized void pressFloor(int f) {                    // from the panel inside the car
        if (mode != Mode.NORMAL) return;                     // car buttons are dead during fire recall
        if (f == floor && door.isOpen()) return;
        Direction d = f > floor ? Direction.UP : f < floor ? Direction.DOWN
                : (direction == Direction.IDLE ? Direction.UP : direction);
        stops.add(new Stop(f, d));
    }

    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();      // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing();   // parked at lobby, doors open
            if (loadKg > capacityKg)
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) { direction = Direction.IDLE; return nothing(); }
        Stop target = next.get();
        if (target.floor() == floor) {                       // arrived
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }

    synchronized List<Stop> takeOutOfService() {             // returns accepted stops for re-dispatch
        mode = Mode.MAINTENANCE;
        direction = Direction.IDLE;
        door.open();                                         // parked with doors open so riders can leave
        return stops.drain();
    }
    synchronized void fireRecall(int lobby) {                // drop every stop, go to the lobby, stay there
        if (mode == Mode.MAINTENANCE) return;
        mode = Mode.FIRE_RECALL;
        stops.clear();
        stops.add(new Stop(lobby, Direction.DOWN));
        direction = Direction.IDLE;
    }
    // board(kg)/alight(kg), returnToService(), available() = mode == NORMAL, display()
}
```

```java
// Dispatch.java
interface DispatchStrategy {                                  // office at 9am vs hotel at midnight
    Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount);
}

final class NearestCarStrategy implements DispatchStrategy {
    @Override public Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount) {
        return cars.stream().filter(Elevator::available)      // out-of-service cars skipped
                .min(Comparator.comparingInt((Elevator c) -> cost(c, floor, wanted, floorCount))
                        .thenComparingInt(Elevator::id));
    }
    private int cost(Elevator c, int floor, Direction wanted, int n) {
        int distance = Math.abs(c.floor() - floor);
        Direction d = c.direction();
        if (d == Direction.IDLE) return distance;
        boolean onTheWay = (d == Direction.UP && c.floor() <= floor) || (d == Direction.DOWN && c.floor() >=
floor);
        if (onTheWay && d == wanted) return distance;
        return distance + 2 * n;                              // penalty: two building heights
    }
}

final class Dispatcher {                                      // calls no car can take wait in a queue
    private volatile DispatchStrategy strategy;
    private final Queue<Stop> pending = new ArrayDeque<>();
    synchronized Optional<Elevator> call(int floor, Direction wanted) {
        Optional<Elevator> car = strategy.choose(cars, floor, wanted, floorCount);
        car.ifPresentOrElse(c -> c.assign(floor, wanted), () -> pending.add(new Stop(floor, wanted)));
        return car;
    }
    // retryPending(): called every tick, re-asks the strategy for each queued call
}
```

```java
// Building.java - owns hall buttons, cars and dispatcher; the only class that knows cars AND floors
final class Building {
    private final Set<Stop> litHallButtons = ConcurrentHashMap.newKeySet();

    String call(int floor, Direction d) {                     // top floor has no UP, ground floor no DOWN
        if ((floor == floors - 1 && d == Direction.UP) || (floor == 0 && d == Direction.DOWN))
            throw new IllegalArgumentException("no " + d + " button on floor " + floor);
        if (!litHallButtons.add(new Stop(floor, d))) return "hall %d %s already lit".formatted(floor, d);
        return dispatcher.call(floor, d).map(c -> "hall %d %s -> car%d".formatted(floor, d, c.id()))
                .orElse("hall %d %s -> queued (no car available)".formatted(floor, d));
    }
    List<String> step() {                                     // one tick: retry queued calls, every car steps once
        tick++;
        dispatcher.retryPending();
        for (Elevator c : cars) {
            Event e = c.step();
            if (e.kind() == Event.Kind.ARRIVED) litHallButtons.remove(new Stop(e.floor(), e.direction()));
            // ... collect "car1->2" / "car1@1 doors open" lines
        }
    }
    List<String> takeOutOfService(int carId) {                // accepted hall calls go back to the dispatcher
        for (Stop s : car(carId).takeOutOfService())
            if (litHallButtons.contains(s)) dispatcher.call(s.floor(), s.direction());   // re-dispatch or queue
    }
    void fireAlarm() { litHallButtons.clear(); cars.forEach(c -> c.fireRecall(0)); }
}
// Main.java: part 1 runs LOOK on one StopList; parts 2-5 a two-car building (floors 0-9, 680 kg);
// part 6: 8 threads released by a CountDownLatch press hall buttons on a three-car building.
```

Output:

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)

== 2. Building: floors 0-9, car1 at 0, car2 at 8 ==
hall 5 UP -> car2
hall 1 UP -> car1
  t1: car1->1, car2->7
  t2: car1@1 doors open, car2->6
rider enters car1 at floor 1, presses 7
  t3: car1@1 doors close, car2->5
  t4: car1->2, car2@5 doors open
rider enters car2 at floor 5, presses 9
hall 3 UP -> car1
  t5: car1->3, car2@5 doors close
  t6: car1@3 doors open, car2->6
  t7: car1@3 doors close, car2->7
  t8: car1->4, car2->8
  t9: car1->5, car2->9
  t10: car1->6, car2@9 doors open
  t11: car1->7, car2@9 doors close
  t12: car1@7 doors open
  t13: car1@7 doors close
display: car1@7 IDLE 80kg | car2@9 IDLE 75kg

== 3. Overload: at floor 9, 10 more people squeeze into car2 ==
hall 9 DOWN -> car2
  t14: car2@9 doors open
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
  t16: car2@9 OVERLOAD 775kg > 680kg, doors stay open
two people step out
  t17: car2@9 doors close
  t18: car2->8
  t19: car2->7
  t20: car2->6
  t21: car2->5
  t22: car2->4
  t23: car2->3
  t24: car2->2
  t25: car2->1
  t26: car2->0
  t27: car2@0 doors open
  t28: car2@0 doors close

== 4. Maintenance: calls move to another car ==
hall 4 DOWN -> car1
re-dispatch hall 4 DOWN -> car2
re-dispatch hall 4 DOWN -> queued
hall 2 UP -> queued (no car available)
engineer returns car1 to service; queued calls are dispatched on the next tick
  t29: car1@7 doors close
  t30: car1->6
  t31: car1->5
  t32: car1->4
  t33: car1@4 doors open
  t34: car1@4 doors close
  t35: car1->3
  t36: car1->2
  t37: car1@2 doors open
  t38: car1@2 doors close

== 5. Fire alarm: every car to the lobby, doors open, car buttons dead ==
  t39: car2@0 doors close
  t40: car2->1
  t41: car2->2
  t42: car1->1, car2->1
  t43: car1->0, car2->0
  t44: car1@0 doors open, car2@0 doors open
display: car1@0 IDLE [open] 80kg FIRE_RECALL | car2@0 IDLE [open] 0kg FIRE_RECALL
```

```
== 6. 8 riders press hall buttons at the same instant (new building) ==
all 8 calls served: true (ticks=11 < 200)
```

Read the output: part 1 — the car serves 6 and 9 going up, turns at 9 (the highest request, not the top of the shaft), then 2 and 1: LOOK, not SCAN. Part 2 — the hall call on 3 goes to car 1, below 3 and already going up. Part 3 — the overloaded car keeps its doors open until two people step out. Part 4 — car 1's call moves to car 2, is queued when no car is available, and is served once car 1 returns. Part 5 — car 2 reverses mid-trip and both cars park at the lobby with doors open.

Concurrency & edge cases

Situation	Handling
Riders press buttons while the controller steps cars	`Elevator` methods are `synchronized`; the dispatcher is `synchronized`; hall lights are a concurrent set
Same hall button pressed twice	`litHallButtons.add` returns false: "already lit", no second dispatch
Up and down requests on the same floor	Two different `Stop`s in two sets
Car call for the floor the car is on, doors open	Ignored (doors already open)
Overload	Doors stay open, display shows OVERLOAD, car does not move
Car taken out of service with accepted hall calls	Calls re-dispatched or queued; never lost
No car available	Call queued; retried every tick
Fire alarm	All stops dropped, lobby added, car buttons ignored, doors stay open at the lobby
Starvation	LOOK serves every accepted stop within one round trip (one sweep each way)
Invalid button (UP on the top floor)	`IllegalArgumentException` — that button does not exist

Extensions & follow-up questions

1. **"How do you choose which car?"** — `DispatchStrategy`; show `NearestCarStrategy` and mention alternatives: zoning (car 1 serves floors 0–10), **up-peak** mode (send idle cars to the lobby at 9 a.m.), and energy-saving rules.
2. **"What if two riders press at the same instant?"** — The dispatcher is synchronized, so calls are dispatched one at a time; each car's stop list is guarded by the car's lock. (The course's design leaves this lock out and says so.)
3. **"How would you test this?"** — Because one call = one step, every test is: a list of button presses, N steps, and an expected list of events. No sleeps, no flakiness.
4. **"Destination dispatch?"** — Riders enter their destination in the lobby keypad; the system groups riders with nearby destinations into the same car. Implement as another `DispatchStrategy` that receives (from, to) instead of (floor, direction).
5. **"Re-evaluate assignments?"** — Each tick, for unserved hall calls, ask the strategy again and move the call if a much better car is free now.
6. **"Real time?"** — A `ScheduledExecutorService` (a Java thread pool that runs tasks on a timer) calls `building.step()` every second (or per car on its own thread, with the car lock guarding the stop list).
7. **"Emergency stop button inside the car?"** — A new mode that halts movement immediately and raises an alarm; with more modes, refactor modes into State classes.
8. **"Express cars?"** — A car with a set of floors it may serve (e.g. lobby plus 30–40); the dispatch strategy skips cars whose served set does not include the call floor.

9. **"Door sensor that reopens for a hand?"** — A `DoorSensor` input; while it reports an obstruction, `step()` reopens the doors instead of closing them (same slot as the overload check).

Real-world example

Modern high-rise buildings use **destination dispatch** systems (for example Schindler's PORT, Otis CompassPlus and KONE's destination control): riders choose their floor in the lobby and the system tells them which car to take, which cuts stops per trip — extension 4. Controllers switch between traffic modes such as **up-peak** in the morning. The fire behaviour in this design follows the idea of **Phase I emergency recall** in elevator safety codes such as ASME A17.1: on a fire alarm, cars return to the designated landing, open their doors and are removed from normal service.

In the interview

Plan for 45 minutes: 5 min requirements and scope (how many cars, which modes) → 5 min entities → **15 min on the stop list and LOOK** (two sets, the four cases) → 10 min car `step()` and dispatcher strategy → 10 min concurrency, testing and follow-ups.

> **30-second answer:** "Each car keeps its stops in two sorted sets, up-stops and down-stops, and uses LOOK: keep serving stops ahead in the current direction, turn at the last request, never run to the end of the shaft for nothing. A car's `step()` does one action — close doors, refuse to move if overloaded, move one floor, or stop and open — which makes the whole system testable without a clock. A dispatcher assigns hall calls through a `DispatchStrategy`, nearest suitable car by default, and queues calls when no car is available. Doors, load, maintenance and fire recall always override scheduling."

Follow-ups with answers

- *"Why LOOK and not SSTF?"* — SSTF can starve a distant floor; LOOK serves every accepted stop within one round trip (one sweep up, one sweep down).
- *"Why two sets and not one?"* — A floor can be requested going up and going down; one set cannot tell which rider to serve on which sweep.
- *"Where do you put the lock?"* — Around each car's stop list and state (the car's monitor — the built-in lock every Java object carries, taken by `synchronized`), and around the dispatcher's assignment. No global lock across cars.

TL;DR

- **Two sorted sets** (up/down) of stops + **LOOK** = efficient, starvation-free scheduling.
- **`step()` = one action**; doors, load, maintenance and fire recall override scheduling.
- **Strategy** for dispatch (nearest car by default), with a pending queue and re-dispatch when a car leaves service.
- Thread safety: per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which approaches of an intersection get green, for how long, and in what order. An **approach** is one road entering the intersection (north, south, east, west). A **phase** is a set of approaches that may be green together because their paths do not cross (for example north and south going straight). The controller cycles through phases, inserting **yellow** (warning) and **all-red** (clearance) intervals between them.

This question tests a **timed state machine** with a **hard safety rule** — *two crossing flows must never be green at the same time* — plus an **emergency override** that must interrupt the normal cycle *safely* (never by cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
How many roads?	A four-way intersection (N, S, E, W); phases are configurable.
Which signals?	Red, yellow, green; flashing red in fault mode.
Fixed or adaptive timing?	Configurable: fixed durations, or adaptive based on queue sensors.
Pedestrian signals, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Yes: an approaching ambulance or fire truck gets green as soon as safely possible, held until it passes.
Many intersections?	Each intersection has its own controller; a central system can coordinate them (extension).
Real time?	A timer ticks once per second; the logic is tick-driven so it can also be simulated.

Requirements

Functional

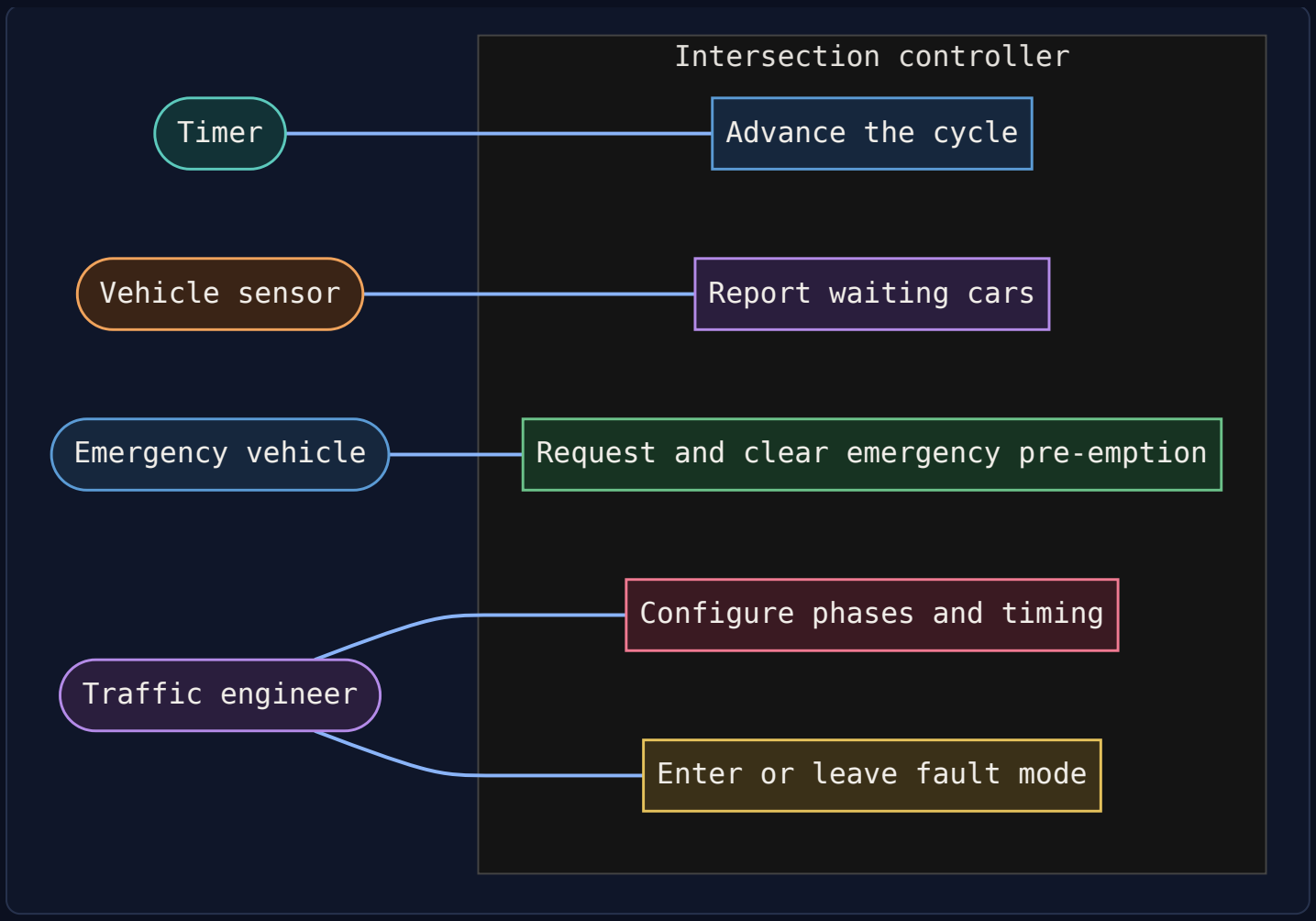
1. Control traffic at an intersection with **multiple roads**.
2. Support **red, yellow and green** signals.
3. Signal **durations are configurable** and can **adapt to traffic** conditions.
4. Transitions are **smooth and safe**: green → yellow → all-red → next green.
5. Handle **emergencies**: an approaching ambulance or fire truck gets priority.
6. Be **scalable and extensible** (more phases, more intersections, new features).
7. On a controller fault, fall back to a **safe mode** (all flashing red = treat as an all-way stop).

Non-functional

- **Safety invariant** (an invariant is a rule that must hold at every moment)**:** never conflicting greens, not even for an instant during a switch.
- **Deterministic timing:** a tick = one second; testable with a simulated clock.
- **Thread safety:** a timer thread ticks while sensor and emergency inputs arrive from other threads.
- **Observability** (the system's behaviour can be watched from outside)**:** displays, logs and monitors can subscribe to light changes.

Use cases

Actors: Timer (ticks every second), **Vehicle sensors** (report queue lengths), **Emergency vehicle / dispatch** (request and clear pre-emption — interrupting the normal cycle to serve a priority request), **Traffic engineer** (configures phases and timing), **Drivers** (see the lights).

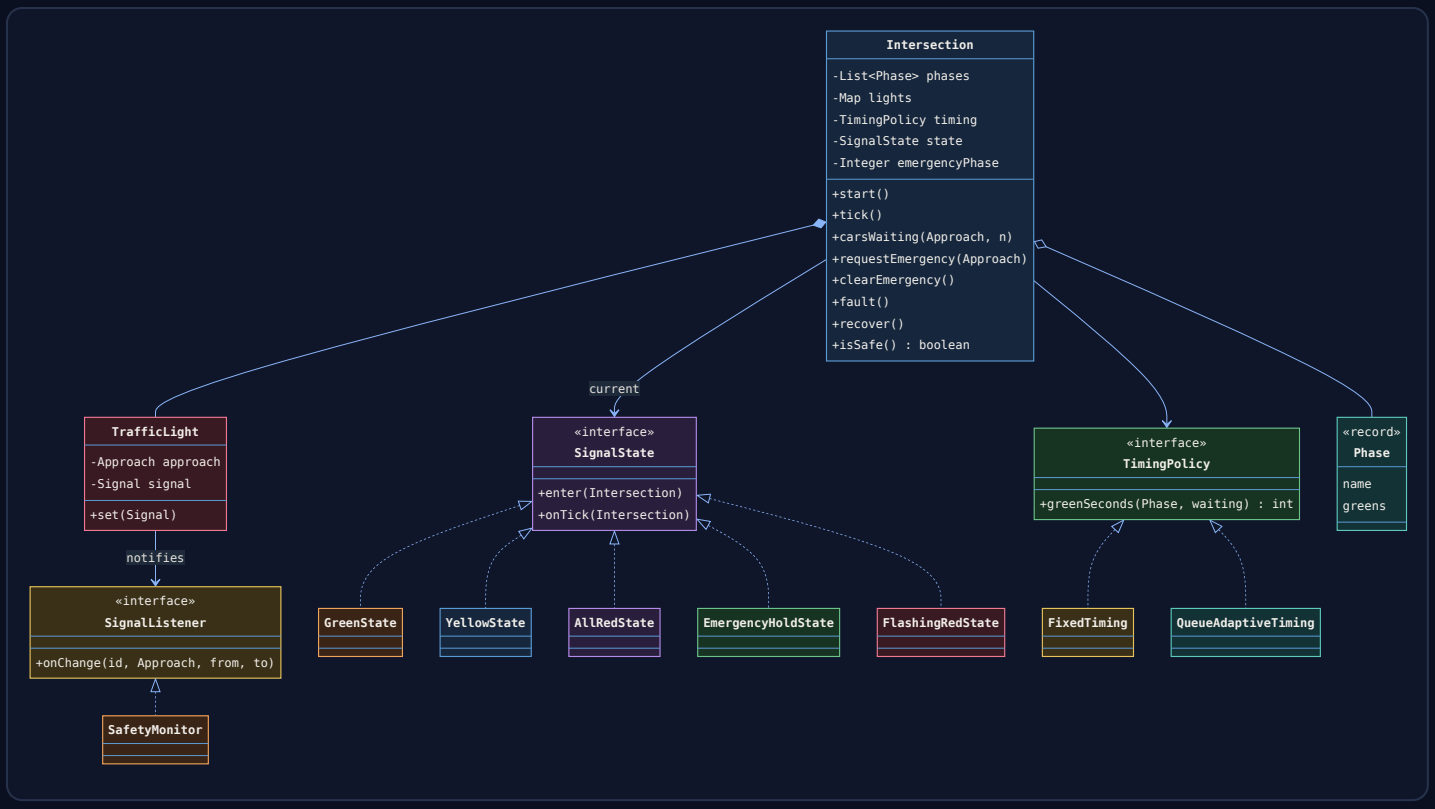


Core entities

Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	A name and the set of approaches that may be green together.
`TrafficLight`	One signal head for one approach; notifies listeners on every change.
`SignalListener`	**Observer** interface for light changes.
`SafetyMonitor`	A listener that checks the safety invariant after every change to green.
`TimingPolicy`	**Strategy** for green duration: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: `GreenState`, `YellowState`, `AllRedState`, `EmergencyHoldState`, `FlashingRedState`.
`Intersection`	The State context (the object that holds the current state and hands it each call): phases, lights, timing, sensors, emergency flag; `tick()`.

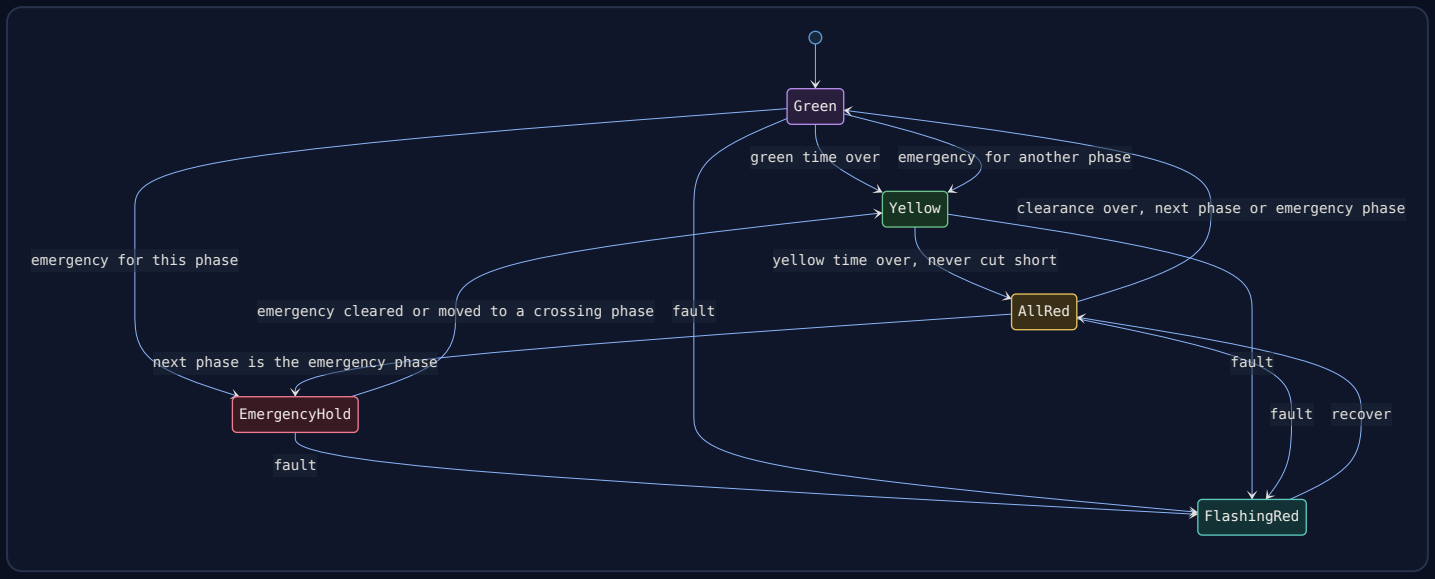
Mapping to the common problem statement. The widely used version (e.g. the open-source *awesome-low-level-design* collection) asks for an intersection with multiple roads, red/yellow/green signals, configurable durations that adjust to traffic, smooth transitions, emergency-vehicle handling, and an extensible design. Its `Signal` enum → our `Signal`; `Road` (id, name, its traffic light) → our `Approach` plus the `TrafficLight` stored for it; `TrafficLight` (signal, durations, notifies observers) → our `TrafficLight` (signal + listeners), with durations moved into `TimingPolicy` and the yellow/all-red settings; `TrafficController` (a Singleton managing all roads) → our `Intersection`, deliberately **not** a Singleton. What the reference leaves implicit, and we make explicit: the phase model (which roads may be green together), the all-red clearance interval, and the safety check.

Class diagram

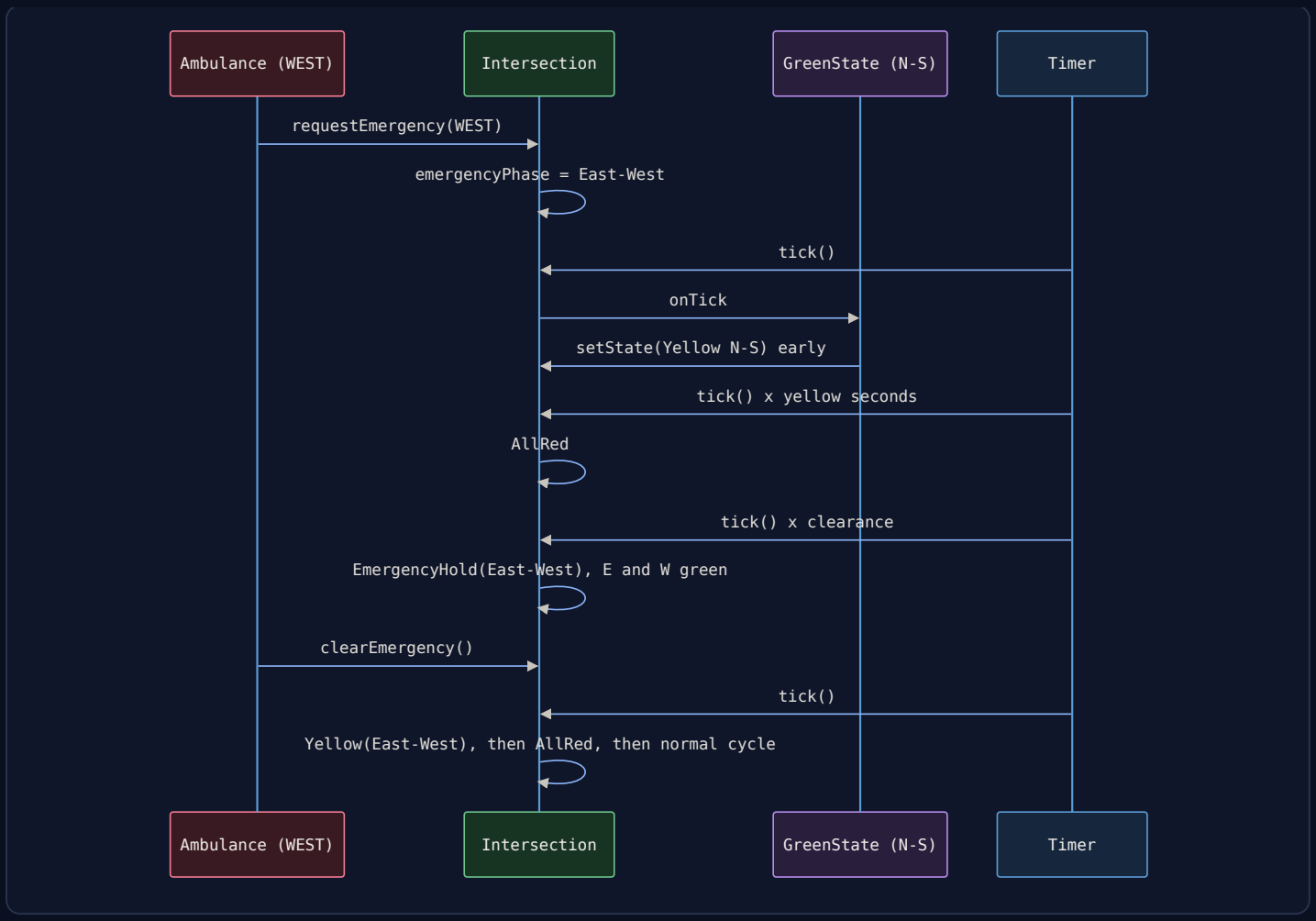


Key flows

The controller's state machine.



Emergency pre-emption.



One full fixed cycle (green 5 s, yellow 2 s, all-red 1 s): `N-S green (5) → N-S yellow (2) → all red (1) → E-W green (5) → E-W yellow (2) → all red (1) → …` — a 16-second cycle.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval is a state class with `enter` and `onTick`	**State**	What a tick means depends on the interval; the transitions (and the "yellow is never cut short" rule) live in one small class each	One `switch` over an enum with counters for every case
States **carry data** (remaining seconds) and are created per transition	State with per-instance data	Unlike the vending machine's stateless states, these hold a countdown, so they cannot be shared	Shared singletons with counters on the context
Green duration comes from a `TimingPolicy`	**Strategy**	Fixed, adaptive (queue sensors), time-of-day plans all swap at run time	Durations hard-coded in `GreenState`
Light changes are published to listeners	**Observer**	Displays, logs, a central monitoring system and the safety monitor subscribe independently	The intersection calling each consumer directly
`showPhaseGreen` turns others **red first**, then turns the phase green	Safety by construction	There is never an instant with conflicting greens	Setting lights in arbitrary order
A separate `SafetyMonitor` checks the invariant	Defence in depth (several independent safety layers)	Mirrors real hardware conflict monitors that force flash if a bug ever causes conflicting greens	Trusting the controller logic alone
Emergency is a **flag** the states read, not a direct jump	Safety	A request only *shortens* green; yellow and all-red still run in full	Jumping straight to the emergency green (a crash risk)
The awesome-lld reference makes the controller a **Singleton**	Rejected	A city has many intersections, each with its own controller; the demo runs two. A per-city `TrafficControlCenter` can own them	`TrafficController.getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`. Essential lines:

```java
// Model.java
enum Signal { RED, YELLOW, GREEN, FLASHING_RED }
enum Approach { NORTH, SOUTH, EAST, WEST }
record Phase(String name, Set<Approach> greens) {}            // approaches that may be green together

interface SignalListener { void onChange(String intersection, Approach approach, Signal from, Signal to); }

final class TrafficLight {                                     // one head facing one approach
    private Signal signal = Signal.RED;
    void set(Signal to) {
        if (to == signal) return;
        Signal from = signal;
        signal = to;
        listeners.forEach(l -> l.onChange(intersection, approach, from, to));
    }
}

interface TimingPolicy { int greenSeconds(Phase phase, Map<Approach, Integer> waitingCars); }
record FixedTiming(int green) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) { return green; }
}
record QueueAdaptiveTiming(int min, int max, int secondsPerCar) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) {    // more cars = longer, clamped
        int cars = p.greens().stream().mapToInt(a -> waiting.getOrDefault(a, 0)).sum();
        return Math.max(min, Math.min(max, cars * secondsPerCar));
    }
}

final class SafetyMonitor implements SignalListener {          // the one rule that must never break
    @Override public void onChange(String id, Approach a, Signal from, Signal to) {
        if (to != Signal.GREEN || watched[0] == null) return;
        if (!watched[0].isSafe()) violations.add(id + ": " + a + " went green while a crossing flow was moving");
    }
}
```

```java
// States.java - each state knows what one second means for it and which state is next
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    private final int phase;
    private int remaining;                                  // per-instance countdown: not shareable
    @Override public void enter(Intersection x) {
        if (Integer.valueOf(phase).equals(x.emergencyPhase())) { x.setState(new EmergencyHoldState(phase));
return; }
        remaining = x.greenSecondsFor(phase);               // from the TimingPolicy
        x.showPhaseGreen(phase);
    }
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                   // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }
}

final class YellowState implements SignalState {
    @Override public void enter(Intersection x) { remaining = x.yellowSeconds(); x.showPhase(phase,
Signal.YELLOW); }
    @Override public void onTick(Intersection x) {          // never reads the emergency flag:
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));   // yellow is never cut short
    }
}

final class AllRedState implements SignalState {            // clearance: the junction empties
    @Override public void enter(Intersection x) { remaining = x.allRedSeconds(); x.showAllRed(); }
    @Override public void onTick(Intersection x) {
        if (--remaining <= 0) {
            Integer em = x.emergencyPhase();
            x.setState(new GreenState(em != null ? em : next));   // emergency phase first, else the next one
        }
    }
}

final class EmergencyHoldState implements SignalState {     // hold the phase green until cleared
    @Override public void enter(Intersection x) { x.showPhaseGreen(phase); }
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em == null || em != phase) x.setState(new YellowState(phase));   // cleared or crossing request: leave
safely
    }
}

final class FlashingRedState implements SignalState {       // fault: treat as a four-way stop
    @Override public void enter(Intersection x) { x.showAll(Signal.FLASHING_RED); }
    @Override public void onTick(Intersection x) {}
}
```

```java
// Intersection.java - the State context; tick() once per second from a timer thread, other inputs from
// other threads; every public method synchronized, so a transition is never observed half-done
final class Intersection {
    private SignalState state;
    private Integer emergencyPhase;                              // null = no override

    synchronized void start() { setState(new GreenState(0)); }
    synchronized void tick() { clock++; state.onTick(this); }
    synchronized void carsWaiting(Approach a, int n) { waitingCars.put(a, n); }
    synchronized void requestEmergency(Approach from) {
        for (int i = 0; i < phases.size(); i++) if (phases.get(i).greens().contains(from)) emergencyPhase = i;
    }
    synchronized void clearEmergency() { emergencyPhase = null; }
    synchronized void fault() { setState(new FlashingRedState()); }
    synchronized void recover() { emergencyPhase = null; setState(new AllRedState(0)); }

    void setState(SignalState s) { state = s; s.enter(this); }   // the new state sets lights and loads its timer

    void showPhaseGreen(int p) {                                 // red first, then green: never two crossing greens
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }
    synchronized boolean isSafe() {                              // every green/yellow approach belongs to ONE phase
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }
}
// Main.java: phases North-South and East-West; FixedTiming(5), yellow 2, all-red 1; part 5 drives a second
// intersection with a ScheduledExecutorService (1 ms stands for 1 s) while a virtual thread (a lightweight
// Java 21 thread, cheap to create by the thousand) fires and clears 20 emergency requests.
```

Output:

```
== 1. Fixed cycle: green 5s, yellow 2s, all-red 1s (printed when lights change) ==
t=0   N:G S:G E:R W:R  GREEN(0)
t=5   N:Y S:Y E:R W:R  YELLOW(0)
t=7   N:R S:R E:R W:R  ALL_RED
t=8   N:R S:R E:G W:G  GREEN(1)
t=13  N:R S:R E:Y W:Y  YELLOW(1)
t=15  N:R S:R E:R W:R  ALL_RED
t=16  N:G S:G E:R W:R  GREEN(0)

== 2. Adaptive timing: 12 cars waiting East-West, 2 North-South ==
t=21  N:Y S:Y E:R W:R  YELLOW(0)
t=23  N:R S:R E:R W:R  ALL_RED
t=24  N:R S:R E:G W:G  GREEN(1)
t=34  N:R S:R E:Y W:Y  YELLOW(1)
t=36  N:R S:R E:R W:R  ALL_RED

== 3. Ambulance from the WEST while North-South is green ==
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=45  ambulance has passed: clear
t=46  N:R S:R E:Y W:Y  YELLOW(1)
t=48  N:R S:R E:R W:R  ALL_RED
t=49  N:G S:G E:R W:R  GREEN(0)

== 4. Controller fault -> flashing red, then recovery ==
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
t=51  N:G S:G E:R W:R  GREEN(0)

== 5. Real timer thread (1 ms = 1 s) + emergency from another thread ==
ticks run: true, safety violations: 0

light changes seen by observer: 48, safety violations: 0
```

Read the output: fixed cycle 5 s green, 2 s yellow, 1 s all-red; with 12 cars waiting east-west the adaptive policy gives E-W its 10-second maximum; the ambulance request at t=38 ends N-S green at the next tick, but yellow (2 s) and all-red (1 s) still run before E-W goes green and is **held**; after the clear the cycle resumes; a fault flashes every head red; no safety violation.

Concurrency & edge cases

Situation	Handling
Emergency request arrives between two ticks	Stored in a field under the intersection's lock; the next tick acts on it
Emergency arrives during yellow	Yellow runs in full; `AllRedState` then picks the emergency phase
Emergency for the phase that is already green	`GreenState.onTick` switches to `EmergencyHold` — no needless yellow
Two emergency vehicles from crossing directions	The latest request wins: if it arrives during a hold, `EmergencyHoldState` leaves through yellow and all-red and the new phase is then held. A real system queues requests by priority and arrival and serves them in turn (extension)
Emergency never cleared (lost message)	Add a maximum hold time after which the controller resumes the cycle
Timer thread and sensor thread overlap	All public methods are `synchronized` ; listeners are in a `CopyOnWriteArrayList` (a thread-safe list that copies itself on every change)
Adaptive timing starves a quiet road	Minimum green time in `QueueAdaptiveTiming` ; every phase still gets its turn in the cycle
Controller bug causes conflicting greens	`SafetyMonitor` detects it; real controllers then force flashing red
Timer drift (small delays adding up so ticks slowly fall behind)	`scheduleAtFixedRate` keeps a fixed rate; for real deployments use the controller's hardware clock

Extensions & follow-up questions

1. **"Pedestrian crossings?"** — Add pedestrian signal heads per phase (WALK during the phase's green, flashing DON'T WALK before yellow) and a push-button input that requests the phase.
2. **"Left-turn arrows?"** — More phases (e.g. N-S protected left, then N-S through); phases are data, so the state machine does not change.
3. **"Coordinate many intersections (green wave)?"** — A `TrafficControlCenter` sets each intersection's cycle length and **offset** so that a platoon (group travelling together) of cars driving at the speed limit meets greens in sequence.
4. **"Emergency corridor?"** — The centre pre-empts the next intersections along the ambulance's route, each through `requestEmergency` .
5. **"Night mode?"** — A time-of-day `TimingPolicy` , or a flashing-yellow state for the main road with flashing red for the side road.
6. **"How to test?"** — The tick-driven design is deterministic: call `tick()` N times and assert the snapshot, as `Main` does.

Real-world example

Adaptive systems such as **SCATS** (developed in Sydney) and **SCOOT** (developed in the UK) adjust green times and cycle lengths from road sensors across whole city networks — the `QueueAdaptiveTiming` strategy scaled up. **Emergency vehicle pre-emption** systems (for example infrared or GPS-based emitters on fire trucks) request green for the vehicle's approach; the controller still times out the current yellow and red clearance before serving it, exactly like our `EmergencyHoldState` . In North American cabinets, a **conflict monitor / malfunction management unit** watches the outputs and forces the intersection into flash if it ever sees conflicting greens — our `SafetyMonitor` observer.

In the interview

Plan for 40 minutes: 5 min requirements (phases, timing, emergency, fault) → 5 min entities → 10 min draw the state machine (green → yellow → all-red → next; emergency and fault edges) → 15 min code `GreenState` / `YellowState` / `AllRedState` and `tick()` → 5 min concurrency and follow-ups.

30-second answer: "An intersection has phases — sets of approaches that can be green together — and a controller that is a timed State machine: Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` once a second; each state counts down and names the next one. Green duration comes from a swappable timing Strategy, fixed or queue-adaptive. An emergency request only shortens the current green: yellow and all-red always run in full, then the emergency phase is held green until cleared. Lights publish changes to observers, including a safety monitor that checks no crossing flows are ever green together."

Follow-ups with answers

- *"Why not jump straight to the emergency green?"* — Cars already in the intersection need the yellow and all-red intervals to clear it; skipping them causes crashes.
- *"Why is the controller not a Singleton?"* — There is one per intersection, and a city has thousands.
- *"How do you make the timing adaptive?"* — Sensors report queue lengths; a `TimingPolicy` turns them into green seconds within a min/max.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.
- **Strategy** for timing (fixed/adaptive), **Observer** for lights, emergency as a flag the states read.
- Tick-driven and synchronized: deterministic tests, safe with a timer thread plus emergency threads.

14.7 Design a Shipping Locker System (Amazon Locker)

Also asked as: "Design Amazon Locker", "Design an Amazon Hub Locker system", "Design a parcel/pickup locker". It is the same problem.

A **parcel locker** (also called a *pickup locker* or *shipping locker*) is a wall of numbered steel doors in a shop, station or apartment lobby. A courier puts a parcel behind one door; the customer later types a **pickup code** on the station's screen and that one door opens. Amazon Hub Locker, InPost's Paczkomat network in Poland and similar networks run by couriers and online shops in many countries all work this way. Lockers save the courier a failed doorstep delivery and let the customer collect at any hour.

This question looks like the parking lot (§14.1) — hand out a limited set of spaces of different sizes — but adds three things interviewers like to probe: **a lifecycle with deadlines** (a reservation that times out, a parcel that expires if nobody collects it), **a secret** (the pickup code, which must be hard to guess and never shown to the courier), and **a network** of stations (if the nearest one is full, suggest another).

Problem & clarifying questions

Question	Assumed answer
What sizes do doors come in?	A few fixed sizes (small, medium, large). A parcel may go into a bigger door if no door of its own size is free.
When is a door chosen — at the station, or before?	Before: the carrier's system **reserves** a door when the parcel is sorted for delivery, so the courier never arrives to a full station. The hold lasts 2 hours; if nothing is deposited, the door is released.
How does the customer open the door?	A 6-digit **one-time code** sent by SMS/app when the parcel is deposited. The courier never sees it.
How long can a parcel stay?	3 days after deposit. After that it **expires** and the carrier takes it back to the sender on the next visit.
What stops someone guessing codes?	Codes are random and single-use; after 5 wrong codes in a row the keypad locks for 15 minutes.
Returns (customer drops a parcel off)?	Yes. The customer starts a return online; a door is held for 2 days and a **drop-off code** is sent. The code opens the empty door once, and the courier collects the parcel on the next visit.
Many stations?	Yes, a city-wide network. If the chosen station is full, offer the nearest station with space.
Broken doors?	A technician can take a free door **out of service**.
Payment at the locker (cash on delivery)?	Out of scope (extension).

Requirements

Functional

1. A station has numbered compartments of fixed sizes.
2. **Reserve** the best-fitting free compartment for a parcel (all-or-nothing); a reservation expires if the parcel is not deposited in time.
3. **Deposit**: the parcel goes in, a one-time pickup code is generated and **sent to the customer only**.
4. **Pickup**: a correct code opens exactly one door, frees the compartment and stops working.
5. **Expiry**: an uncollected parcel expires after the pickup window; the customer is told; the carrier **collects expired parcels** for return.
6. **Brute-force protection**: lock the keypad after too many wrong codes.
7. **Network search**: nearest station within a radius that has a free door this parcel fits.
8. Compartments can be taken out of service.
9. **Customer return**: reserve a door for a return, send a single-use drop-off code, open the empty door on that code, and let the courier collect dropped-off returns. An unused drop-off hold is cancelled after its window.

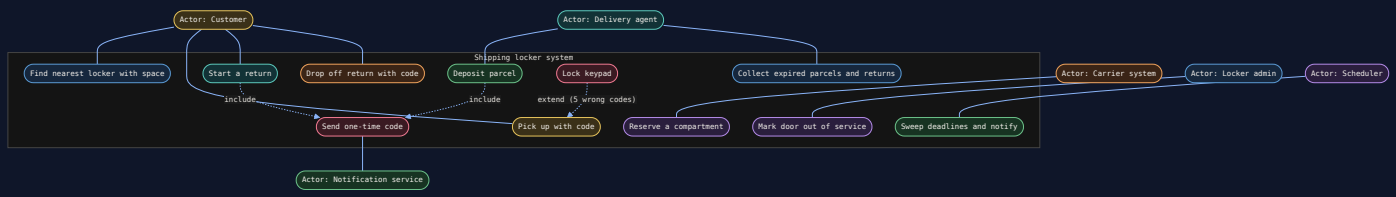
Non-functional

- **Never give one door to two parcels**, even when many carriers reserve at the same moment.
- **Security**: codes are unpredictable, single-use, stored only as a hash (the one-way fingerprint described in the ATM section).
- **Correct deadlines** without depending on a background timer being on time.
- **Testable**: time and randomness are injected (a fake clock and a seeded random generator in the demo — one started from a fixed seed, so it produces the same numbers every run).

Use cases

Actors: Customer (picks up, drops off returns), **Delivery agent** (the courier: deposits parcels, collects expired parcels and returns), **Carrier system** (the shipper's software that books doors), **Locker admin** (a technician who takes doors out of service), **Scheduler** (a component that runs a job at set times; here

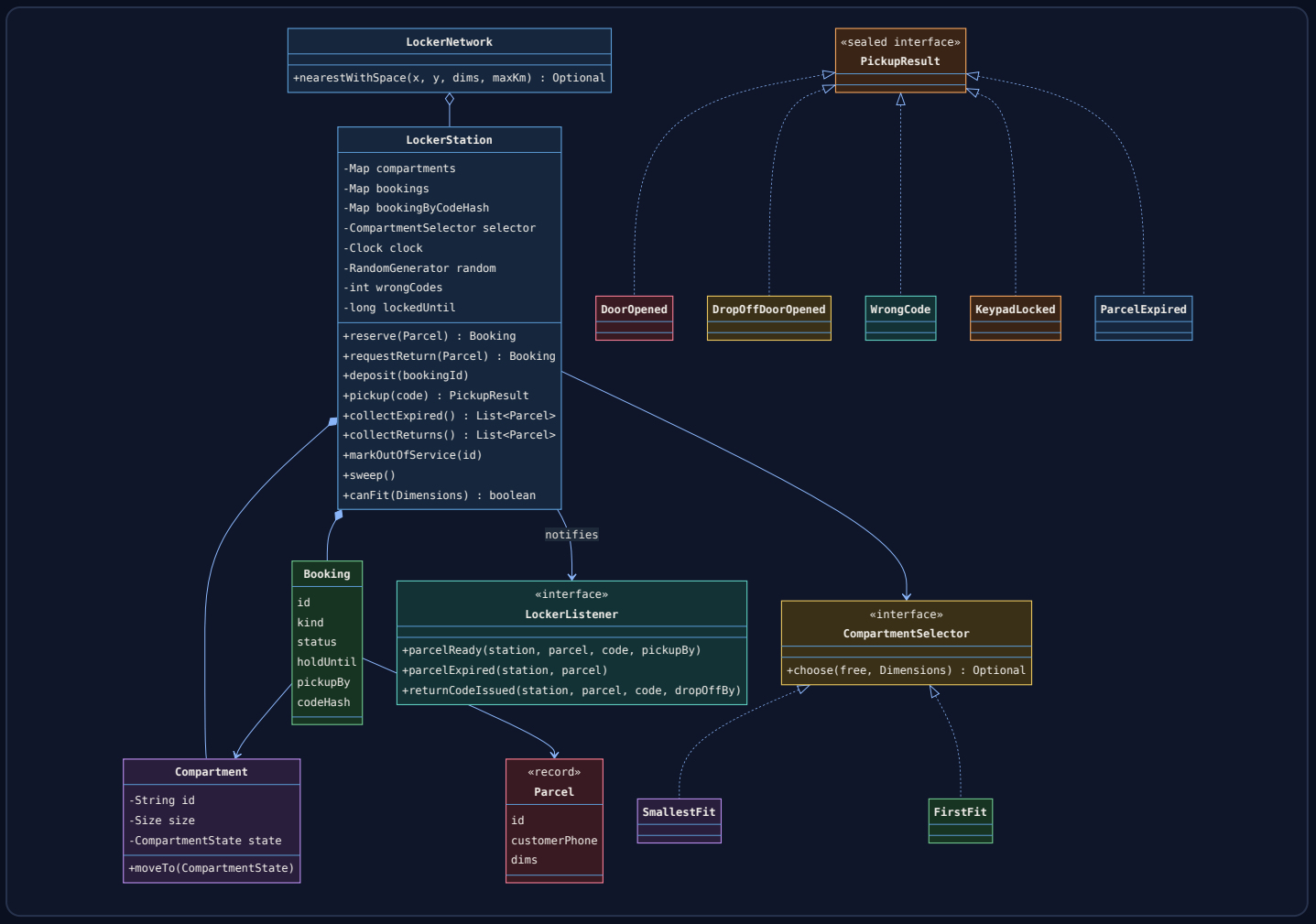
the periodic sweep) and **Notification service** (SMS/app). Notation (system box, include/extend arrows) as in §3.10.



Core entities

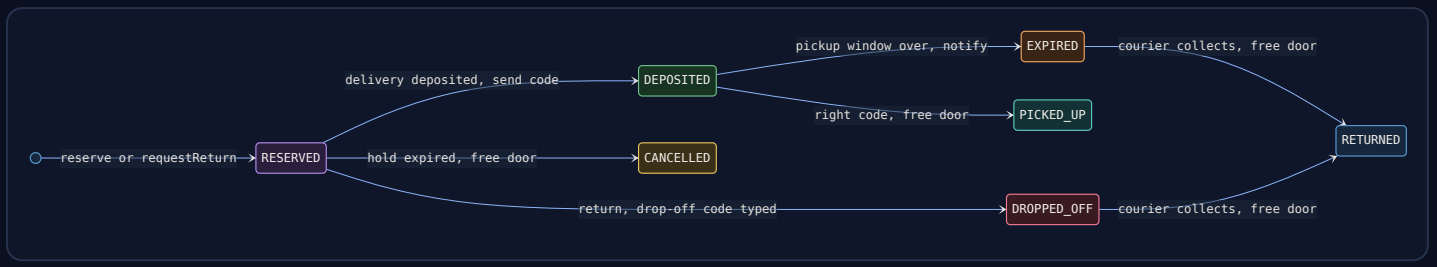
Class	Responsibility
`Size` (enum)	SMALL / MEDIUM / LARGE with inner dimensions; `fits(Dimensions)` checks a box against a door after sorting both sides.
`Dimensions` (record)	A parcel's three sides in cm (validated positive).
`Parcel` (record)	Parcel id, customer phone, dimensions.
`Compartment`	One door: id, size, `CompartmentState` (FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE) with the legal moves inside `moveTo`.
`Booking` + `BookingKind` + `BookingStatus`	One parcel's stay in one door, either a DELIVERY or a CUSTOMER_RETURN (statuses in the state diagram under Key flows). Holds the deadlines and the code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `DropOffDoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired` — every keypad outcome is a type, not a magic string (a bare text value whose meaning is known only by convention).
`CompartmentSelector`	**Strategy**: `SmallestFit` (default), `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with the code), parcel expired, return drop-off code issued.
`LockerStation`	The station: doors, bookings, code index, lockout counter, clock, random generator; every state change under one lock.
`LockerNetwork`	All stations; nearest station with space.

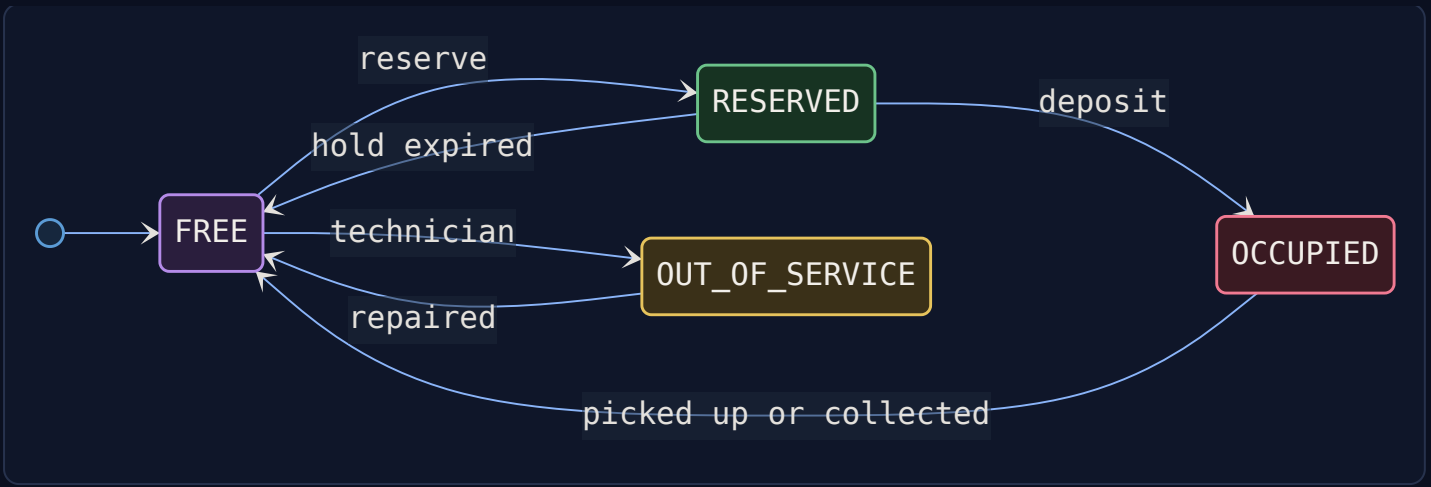
Class diagram



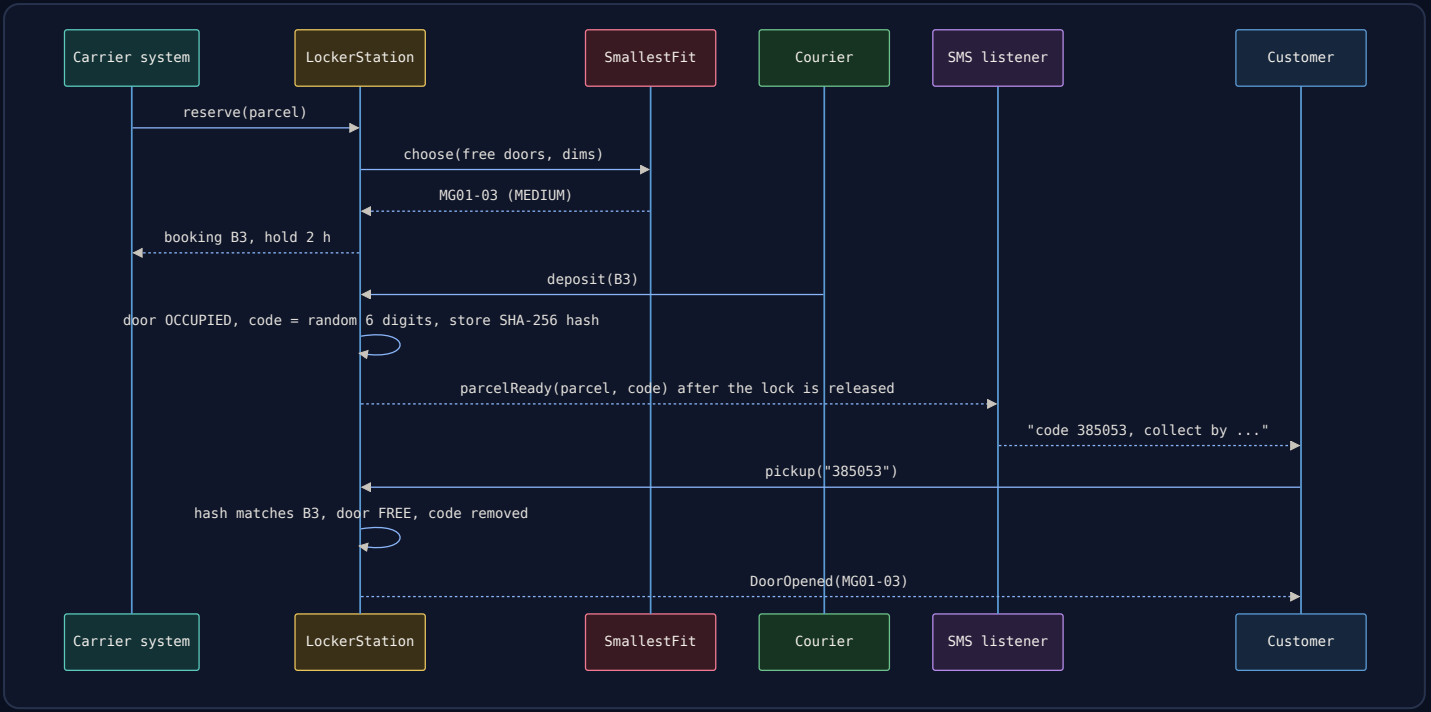
Key flows

The two state machines — first the booking, then the door. A compartment only knows whether it is free; a booking knows the parcel's story. Keeping them separate means a door can be reused while the old booking stays in the history. A customer return uses the same booking machine with one extra status, DROPPED_OFF.



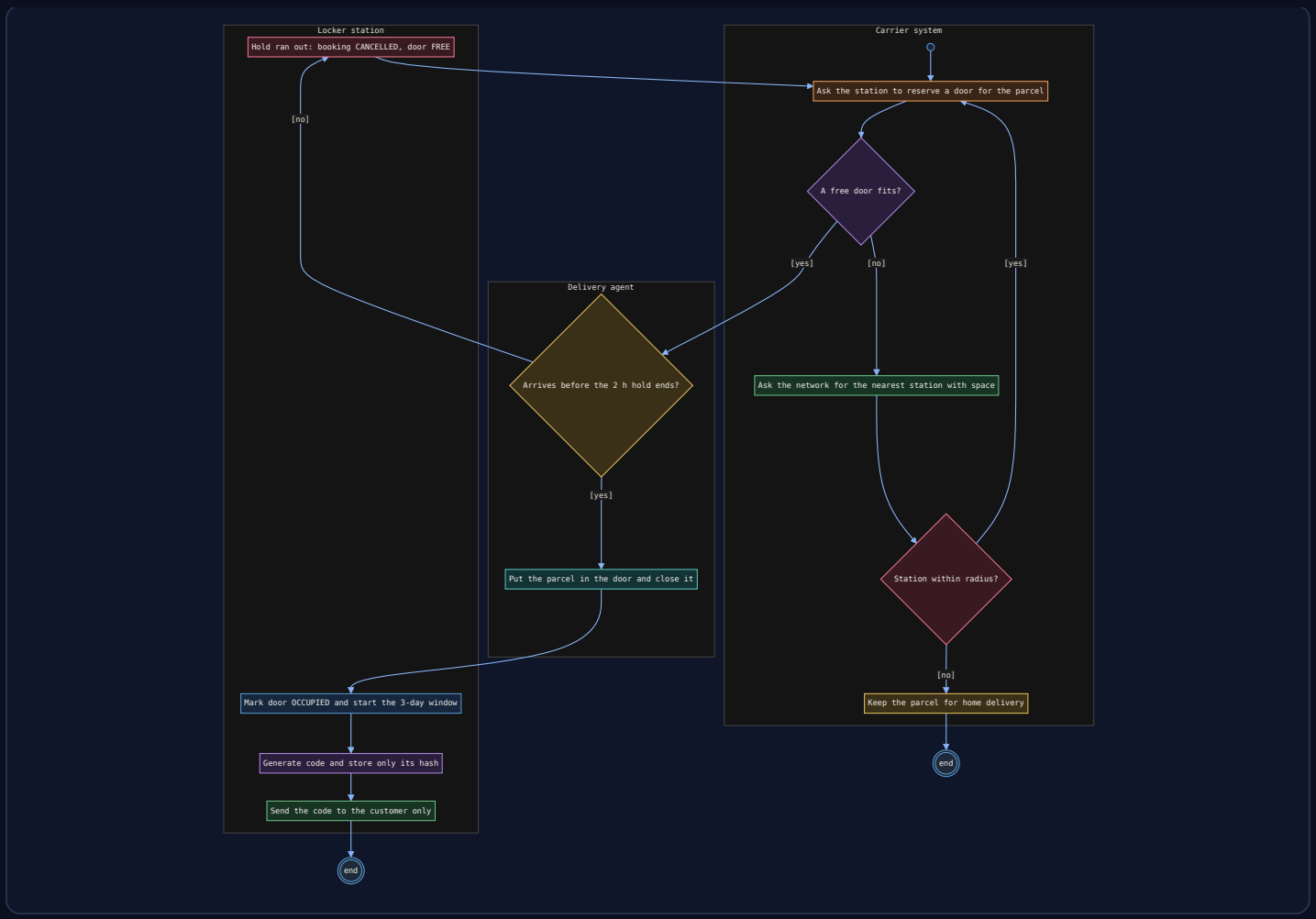


Delivery and pickup.

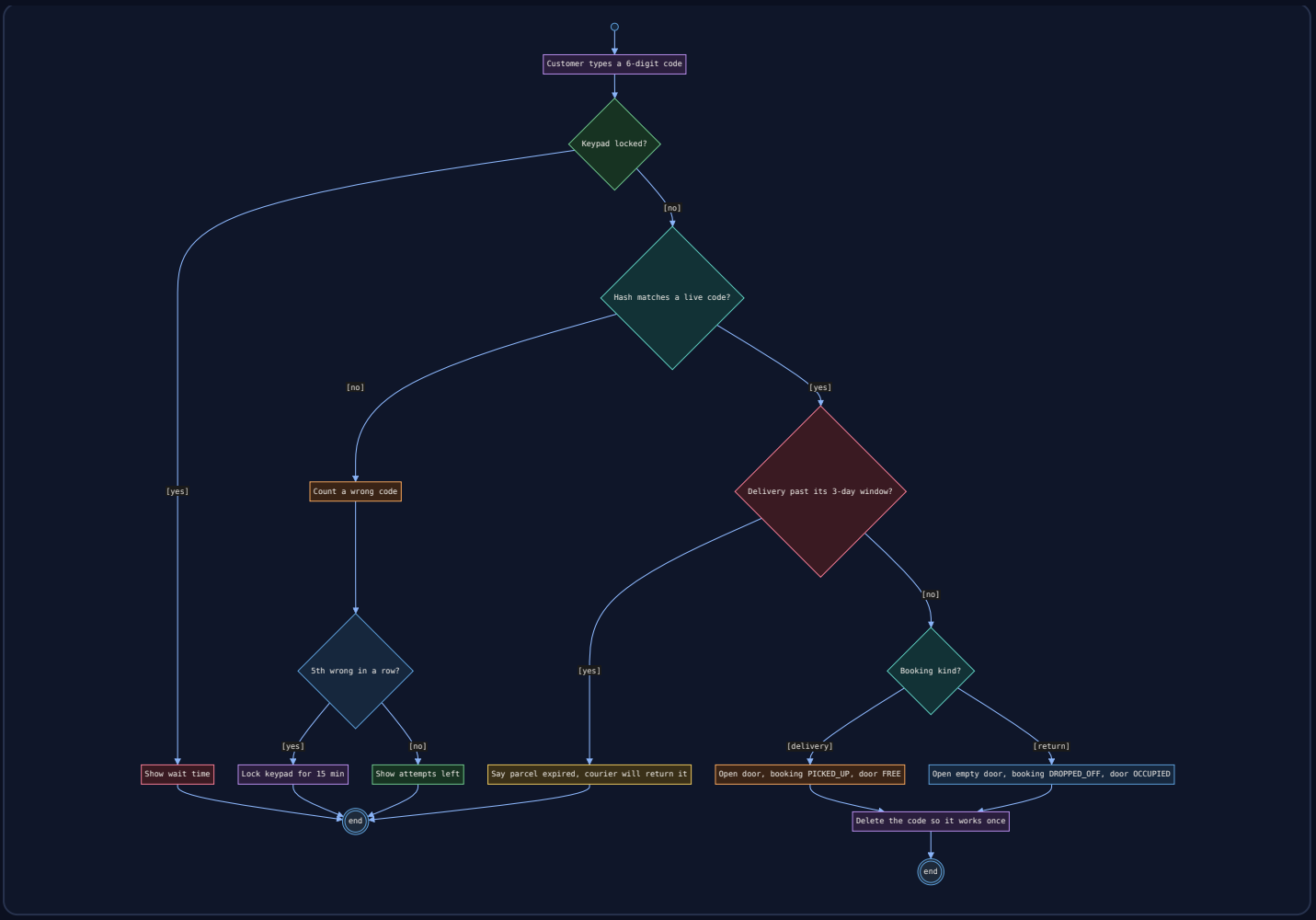


The sequence diagram shows the happy path. The two activity diagrams below (§3.13) add the branches: a full station or a late courier, and every outcome of typing a code.

Activity — deliver a parcel.



Activity — type a code at the keypad (pickup or return drop-off). One keypad serves both: the code's hash finds the booking, and the booking's kind decides whether the door opens to take a parcel out or to put a return in.



Expired deliveries and dropped-off returns leave the station the same way: on the delivery agent's next visit, `collectExpired()` and `collectReturns()` open those doors, mark the bookings RETURNED and free the doors.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice is a `CompartmentSelector`	**Strategy**	"Smallest door that fits" keeps large doors for large parcels; a station may prefer "lowest door first" (fast for the courier) or "eye-level first" (accessibility). Each is a small class.	An `if (mode == ...)` inside `reserve`
Separate `Compartment` state and `Booking` status	Two small state machines	A door is reused many times; the booking keeps each parcel's history and deadlines	One status field on the door that mixes "free" with "expired parcel inside"
Transitions checked inside `moveTo` (a `switch` over the current state)	Enum state machine	An illegal move (e.g. taking an occupied door out of service) throws at once	Setting the field anywhere
Keypad outcomes as a **sealed** `PickupResult`	Sealed result types	The UI must handle every outcome; the compiler checks the `switch`	Returning `null`, `-1` or strings
Codes: random, single-use, unique per station, stored as a salted (mixed with a random value before hashing) **SHA-256** hash	Security by design	A database leak does not reveal live codes; the courier never sees them	Storing the plain code, or deriving it from the parcel id
Notify through `LockerListener` **after** releasing the lock	**Observer**	SMS, app push and audit log are independent; a slow SMS gateway (the service that actually sends text messages) must not block the keypad	Calling the SMS gateway inside `deposit` while holding the lock
Lazy expiry (deadlines checked whenever the data is next touched) at the start of every operation, plus a `sweep()` for notifications	Deadline checking on read	Correct even if the scheduler is late or down; the scheduler only sends messages	A reaper thread (a background thread that cleans up expired items) as the *only* thing that frees doors
`Clock` and `RandomGenerator` are injected	Dependency injection	Deterministic demo and tests; production passes `Clock.systemUTC()` and a `SecureRandom` (Java's cryptographically strong random generator)	`Instant.now()` and `new Random()` inside the class
One lock per station (`synchronized`)	Monitor	A station has tens of doors and each call is microseconds, so one lock is simple and never the bottleneck (the one part that limits overall speed); stations never share a lock	Per-door CAS like the parking lot — more code for no gain at this size

Java implementation

Package `lld.ch14.locker` (imports omitted).

```java
// Model.java
/** Inner size of a compartment door, in centimetres (height x width x depth). */
enum Size {
    SMALL(10, 40, 60), MEDIUM(20, 40, 60), LARGE(40, 40, 60);

    private final int[] sorted;

    Size(int h, int w, int d) { sorted = new int[]{h, w, d}; Arrays.sort(sorted); }

    /** A box fits if, after sorting both sides, each parcel side is no longer than the matching compartment side.
*/
    boolean fits(Dimensions p) {
        int[] q = p.sorted();
        return q[0] <= sorted[0] && q[1] <= sorted[1] && q[2] <= sorted[2];
    }
}

/** Parcel box size in centimetres. */
record Dimensions(int a, int b, int c) {
    Dimensions {
        if (a <= 0 || b <= 0 || c <= 0) throw new LockerException("Dimensions must be positive");
    }
    int[] sorted() { int[] s = {a, b, c}; Arrays.sort(s); return s; }
    @Override public String toString() { return a + "x" + b + "x" + c + "cm"; }
}

record Parcel(String id, String customerPhone, Dimensions dims) { }

enum CompartmentState { FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE }

/** One door in a locker station. Its state is changed only by the station, under the station's lock. */
final class Compartment {
    private final String id;
    private final Size size;
    private CompartmentState state = CompartmentState.FREE;

    Compartment(String id, Size size) { this.id = id; this.size = size; }

    String id() { return id; }
    Size size() { return size; }
    CompartmentState state() { return state; }
    boolean isFree() { return state == CompartmentState.FREE; }

    void moveTo(CompartmentState next) {
        boolean ok = switch (state) {
            case FREE -> next == CompartmentState.RESERVED || next == CompartmentState.OUT_OF_SERVICE;
            case RESERVED -> next == CompartmentState.OCCUPIED || next == CompartmentState.FREE;
            case OCCUPIED -> next == CompartmentState.FREE;
            case OUT_OF_SERVICE -> next == CompartmentState.FREE;
        };
        if (!ok) throw new LockerException("Compartment " + id + ": illegal move " + state + " -> " + next);
        state = next;
    }

    @Override public String toString() { return id + "(" + size + ")"; }
}

/** Why a door is booked: a delivery to the customer, or a parcel the customer sends back. */
enum BookingKind { DELIVERY, CUSTOMER_RETURN }

/** Lifecycle of one parcel's stay in a locker. */
enum BookingStatus {
    RESERVED,      // door held; delivery: parcel not yet inside / return: drop-off code sent
    DEPOSITED,     // delivery parcel inside, pickup code sent
    PICKED_UP,     // customer opened the door with the code (final)
    EXPIRED,       // not collected in time; waiting for the carrier to take it back
    DROPPED_OFF,   // customer put a return inside; waiting for the carrier
    RETURNED,      // carrier removed the parcel: an expired delivery or a customer return (final)
    CANCELLED      // hold ran out before the deposit or the drop-off (final)
}
```

```
/** One parcel's use of one compartment. */
final class Booking {
    final String id;
    final Parcel parcel;
    final Compartment compartment;
    final BookingKind kind;
    BookingStatus status = BookingStatus.RESERVED;
    long holdUntil;          // epoch seconds: deposit (or customer drop-off) must happen before this
    long pickupBy;           // epoch seconds: set at deposit
    String codeHash;         // only a hash of the pickup (or drop-off) code is stored

    Booking(String id, Parcel parcel, Compartment compartment, BookingKind kind, long holdUntil) {
        this.id = id; this.parcel = parcel; this.compartment = compartment; this.kind = kind; this.holdUntil =
holdUntil;
    }
}

/** What the customer's keypad entry produced. */
sealed interface PickupResult permits DoorOpened, DropOffDoorOpened, WrongCode, KeypadLocked, ParcelExpired { }
record DoorOpened(String compartmentId, String parcelId) implements PickupResult { }
record DropOffDoorOpened(String compartmentId, String parcelId) implements PickupResult { }   // empty door for a
return
record WrongCode(int attemptsLeft) implements PickupResult { }
record KeypadLocked(long secondsLeft) implements PickupResult { }
record ParcelExpired(String parcelId) implements PickupResult { }

/** Observer: SMS/app notifications, carrier dashboards, audit logs. */
interface LockerListener {
    void parcelReady(String stationId, Parcel parcel, String pickupCode, long pickupBy);
    default void parcelExpired(String stationId, Parcel parcel) { }
    default void returnCodeIssued(String stationId, Parcel parcel, String dropOffCode, long dropOffBy) { }
}

final class LockerException extends RuntimeException {
    LockerException(String message) { super(message); }
}
```

```
// Selection.java
/** Strategy: which free compartment should this parcel get? */
interface CompartmentSelector {
    Optional<Compartment> choose(List<Compartment> free, Dimensions parcel);
}

/** Smallest door that fits: keeps large doors free for large parcels (the usual default). */
final class SmallestFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::size).thenComparing(Compartment::id));
    }
}

/** Lowest door number that fits, whatever its size: simple and fast, but wastes large doors. */
final class FirstFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::id));
    }
}
```

```java
// LockerStation.java
/**
 * One locker station (a bank of doors at one address). Every state change happens under the
 * station's monitor lock: a station has tens of doors and each operation takes microseconds,
 * so one lock is simple and never the bottleneck.
 */
final class LockerStation {
    static final int MAX_WRONG_CODES = 5;

    private final String id;
    private final double x, y;                                      // location in km on a city grid
    private final Map<String, Compartment> compartments = new LinkedHashMap<>();
    private final Map<String, Booking> bookings = new HashMap<>();
    private final Map<String, String> bookingByCodeHash = new HashMap<>();
    private final List<LockerListener> listeners = new CopyOnWriteArrayList<>();
    private final CompartmentSelector selector;
    private final Clock clock;
    private final RandomGenerator random;
    private final Duration reservationHold, pickupWindow, lockout, dropOffWindow;
    private final List<Parcel> newlyExpired = new ArrayList<>();
    private int nextBooking = 1, wrongCodes = 0;
    private long lockedUntil = 0;

    /** layout = the size of each door, in door-number order (door 01 is the first element). */
    LockerStation(String id, double x, double y, List<Size> layout, CompartmentSelector selector,
                  Clock clock, RandomGenerator random) {
        this.id = id; this.x = x; this.y = y;
        this.selector = selector; this.clock = clock; this.random = random;
        this.reservationHold = Duration.ofHours(2);
        this.pickupWindow = Duration.ofDays(3);
        this.lockout = Duration.ofMinutes(15);
        this.dropOffWindow = Duration.ofDays(2);
        for (int n = 1; n <= layout.size(); n++) {
            String cid = "%s-%02d".formatted(id, n);
            compartments.put(cid, new Compartment(cid, layout.get(n - 1)));
        }
    }

    /** Convenience layout: small doors first, then medium, then large. */
    static List<Size> layout(int small, int medium, int large) {
        List<Size> l = new ArrayList<>();
        for (int i = 0; i < small; i++) l.add(Size.SMALL);
        for (int i = 0; i < medium; i++) l.add(Size.MEDIUM);
        for (int i = 0; i < large; i++) l.add(Size.LARGE);
        return l;
    }

    String id() { return id; }
    double distanceTo(double px, double py) { return Math.hypot(x - px, y - py); }
    void addListener(LockerListener l) { listeners.add(l); }

    /** Carrier books a door before driving to the station. All-or-nothing: a door, or an exception. */
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
        return allocate(parcel, BookingKind.DELIVERY, reservationHold);
    }

    /** Customer starts a return online: an empty door is held for 2 days and a drop-off code goes to the
customer. */
    Booking requestReturn(Parcel parcel) {
        Booking b;
        String code;
        synchronized (this) {
            expireOverdue();
            b = allocate(parcel, BookingKind.CUSTOMER_RETURN, dropOffWindow);
            code = issueCode(b);
        }
        for (LockerListener l : listeners) l.returnCodeIssued(id, b.parcel, code, b.holdUntil);   // outside the
lock
        return b;
```

```
    }

    /** Carrier puts the parcel in and closes the door. The code goes to the customer, never to the carrier. */
    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            expireOverdue();
            b = booking(bookingId);
            if (b.status != BookingStatus.RESERVED) throw new LockerException("Booking " + bookingId + " is " +
b.status);
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            code = issueCode(b);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the lock
    }

    /** Customer types a code on the station keypad: a pickup code, or a drop-off code for a return. */
    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            if (++wrongCodes >= MAX_WRONG_CODES) {                  // slow down code guessing
                lockedUntil = t + lockout.toSeconds();
                wrongCodes = 0;
                return new KeypadLocked(lockout.toSeconds());
            }
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        if (b.kind == BookingKind.CUSTOMER_RETURN) {                // drop-off: open the empty door once
            b.status = BookingStatus.DROPPED_OFF;
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            bookingByCodeHash.remove(b.codeHash);
            return new DropOffDoorOpened(b.compartment.id(), b.parcel.id());
        }
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                       // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }

    /** Carrier's daily visit: open every door holding an expired parcel and take them for return. */
    synchronized List<Parcel> collectExpired() {
        expireOverdue();
        List<Parcel> taken = new ArrayList<>();
        for (Booking b : bookings.values())
            if (b.status == BookingStatus.EXPIRED) {
                b.status = BookingStatus.RETURNED;
                b.compartment.moveTo(CompartmentState.FREE);
                bookingByCodeHash.remove(b.codeHash);
                taken.add(b.parcel);
            }
        return taken;
    }

    /** Same visit: take the parcels customers dropped off for return. */
    synchronized List<Parcel> collectReturns() {
        expireOverdue();
        List<Parcel> taken = new ArrayList<>();
        for (Booking b : bookings.values())
            if (b.status == BookingStatus.DROPPED_OFF) {
                b.status = BookingStatus.RETURNED;
                b.compartment.moveTo(CompartmentState.FREE);
                taken.add(b.parcel);
```

```
            }
        return taken;
    }

    synchronized void markOutOfService(String compartmentId) {
        Compartment c = compartments.get(compartmentId);
        if (c == null) throw new LockerException("Unknown compartment " + compartmentId);
        c.moveTo(CompartmentState.OUT_OF_SERVICE);                // only a FREE door can be taken out
    }

    synchronized Map<Size, Integer> freeBySize() {
        expireOverdue();
        Map<Size, Integer> m = new EnumMap<>(Size.class);
        for (Size s : Size.values()) m.put(s, 0);
        compartments.values().stream().filter(Compartment::isFree).forEach(c -> m.merge(c.size(), 1,
Integer::sum));
        return m;
    }

    synchronized boolean canFit(Dimensions d) {
        expireOverdue();
        return compartments.values().stream().anyMatch(c -> c.isFree() && c.size().fits(d));
    }

    /** Called by a scheduler (e.g. every minute): applies deadlines and tells customers, outside the lock. */
    void sweep() {
        List<Parcel> expired;
        synchronized (this) { expireOverdue(); expired = List.copyOf(newlyExpired); newlyExpired.clear(); }
        for (Parcel p : expired) for (LockerListener l : listeners) l.parcelExpired(id, p);
    }

    synchronized BookingStatus status(String bookingId) { expireOverdue(); return booking(bookingId).status; }

    /** Lazy expiry: every operation first applies the deadlines that have passed, so correctness never waits for
a timer. */
    private void expireOverdue() {
        long t = now();
        for (Booking b : bookings.values()) {
            if (b.status == BookingStatus.RESERVED && t >= b.holdUntil) {
                b.status = BookingStatus.CANCELLED;
                b.compartment.moveTo(CompartmentState.FREE);
                if (b.codeHash != null) bookingByCodeHash.remove(b.codeHash);   // an unused drop-off code dies
too
            } else if (b.status == BookingStatus.DEPOSITED && t >= b.pickupBy) {
                b.status = BookingStatus.EXPIRED;                    // door stays OCCUPIED until the carrier
collects
                newlyExpired.add(b.parcel);                          // notified by sweep(), outside the lock
            }
        }
    }

    private static boolean isActive(Booking b) {
        return b.status == BookingStatus.RESERVED || b.status == BookingStatus.DEPOSITED
                || b.status == BookingStatus.EXPIRED || b.status == BookingStatus.DROPPED_OFF;
    }

    /** Caller holds the lock. All-or-nothing: a door marked RESERVED and a booking, or an exception. */
    private Booking allocate(Parcel parcel, BookingKind kind, Duration hold) {
        if (bookings.values().stream().anyMatch(b -> b.parcel.id().equals(parcel.id()) && isActive(b)))
            throw new LockerException("Parcel " + parcel.id() + " already has a locker");
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " +
id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, kind, now() + hold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    /** Caller holds the lock. A random 6-digit code, unique among live codes here; only its hash is kept. */
```

```
    private String issueCode(Booking b) {
        String code;
        do { code = "%06d".formatted(random.nextInt(1_000_000)); }
        while (bookingByCodeHash.containsKey(hash(code)));
        b.codeHash = hash(code);
        bookingByCodeHash.put(b.codeHash, b.id);
        return code;
    }

    private Booking booking(String bookingId) {
        Booking b = bookings.get(bookingId);
        if (b == null) throw new LockerException("Unknown booking " + bookingId);
        return b;
    }

    private long now() { return clock.instant().getEpochSecond(); }

    /** Store only a salted hash of each code, so a leaked database does not reveal live codes. */
    private String hash(String code) {
        try {
            byte[] h = MessageDigest.getInstance("SHA-256").digest((id + ":" +
code).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(h);
        } catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}
```

```
// LockerNetwork.java
/** All stations in a city. Used at checkout ("pick a locker near you") and when a station is full. */
final class LockerNetwork {
    private final List<LockerStation> stations = new CopyOnWriteArrayList<>();

    void add(LockerStation s) { stations.add(s); }

    /** Nearest station within maxKm that has a free door this parcel fits in right now. */
    Optional<LockerStation> nearestWithSpace(double x, double y, Dimensions parcel, double maxKm) {
        return stations.stream()
                .filter(s -> s.distanceTo(x, y) <= maxKm)
                .sorted(Comparator.comparingDouble(s -> s.distanceTo(x, y)))
                .filter(s -> s.canFit(parcel))
                .findFirst();
    }
}
```

```
// Main.java (excerpt): a movable TestClock (production passes Clock.systemUTC()) and a fake SMS gateway
static final class SmsInbox implements LockerListener {   // remembers the last code sent to each phone
    final Map<String, String> lastCode = new ConcurrentHashMap<>();
    @Override public void parcelReady(String station, Parcel p, String code, long pickupBy) {
        lastCode.put(p.customerPhone(), code);
        System.out.printf("   SMS to %s: parcel %s is in locker %s, code %s, collect by %s%n",
                p.customerPhone(), p.id(), station, code, Instant.ofEpochSecond(pickupBy));
    }
    // parcelExpired and returnCodeIssued print the other two SMS lines
}
// Station MG01 = layout(2, 2, 1) with SmallestFit and new SplittableRandom(7); ten numbered scenarios:
// reserve 4 parcels, deposit, pickup, guessing lockout, hold expiry, 3-day expiry, network search,
// FirstFit vs SmallestFit, 50 threads reserving on a 20-door HUB, and a customer return at RT04.
```

How the code works, part by part

- `Size.fits` sorts the three sides of box and door and compares smallest-to-smallest — handles a parcel on its side without trying all six rotations.
- `reserve` applies overdue deadlines, rejects a parcel that already has a door, asks the selector, marks the door RESERVED and creates a `Booking` with a 2-hour hold. It is `synchronized`, so "find a free door" + "mark it reserved" is one atomic step.
- `deposit` changes state inside `synchronized (this)`, generates a 6-digit code (retrying on the rare clash with another live code at this station), stores only its hash, and calls listeners **after** the

block.

- `pickup` checks the lockout, looks the code up by hash, counts wrong attempts (5 in a row lock the keypad for 15 minutes), refuses an expired parcel, and on success frees the door and deletes the code.
- `expireOverdue` runs at the top of every public method: RESERVED past its hold → CANCELLED (door freed); DEPOSITED past its window → EXPIRED (door stays OCCUPIED until the courier collects). `sweep()`, called by a scheduler every minute, runs the same check and sends "not collected" messages outside the lock.
- `requestReturn` reuses `allocate` (2-day hold instead of 2 hours) and issues a drop-off code at once; in `pickup`, a CUSTOMER_RETURN code opens the empty door → DROPPED_OFF; `collectReturns` frees those doors; a lapsed return hold is cancelled and its code deleted.
- `LockerNetwork.nearestWithSpace` sorts stations by distance, keeps those within the radius, returns the first that can fit the parcel now.

Output:

```
== 1. Carrier reserves doors (smallest door that fits) ==
   P1 3x10x18cm    -> MG01-01(SMALL)
   P2 4x16x24cm    -> MG01-02(SMALL)
   P3 9x12x12cm    -> MG01-03(MEDIUM)
   P4 14x22x34cm   -> MG01-04(MEDIUM)
   free now: {SMALL=0, MEDIUM=0, LARGE=1}  (the mug got a MEDIUM: both SMALL doors were taken)
   error: Parcel P1 already has a locker
   error: No free compartment fits 50x50x70cm at MG01

== 2. Deposit: the code goes to the customer by SMS, never to the carrier ==
   SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00002: parcel P2 is in locker MG01, code 607553, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00003: parcel P3 is in locker MG01, code 385053, collect by 2026-10-04T09:00:00Z
   error: Booking MG01-B1 is DEPOSITED

== 3. Pickup ==
   typo 123456 -> WrongCode[attemptsLeft=4]
   right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
   same code again -> WrongCode[attemptsLeft=4]  (a code works once)

== 4. Someone guesses codes: keypad locks after 5 wrong tries in a row ==
   guess 000001 -> WrongCode[attemptsLeft=3]
   guess 000002 -> WrongCode[attemptsLeft=2]
   guess 000003 -> WrongCode[attemptsLeft=1]
   guess 000004 -> KeypadLocked[secondsLeft=900]
   real owner during lockout -> KeypadLocked[secondsLeft=900]
   15 min later -> DoorOpened[compartmentId=MG01-02, parcelId=P2]

== 5. Reservation hold runs out (carrier never came with the shoes) ==
   booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}

== 6. Mug not collected within 3 days ==
   SMS to +91-98450-00003: parcel P3 was not collected and will be returned
   customer types the code -> ParcelExpired[parcelId=P3]
   carrier collects for return: [P3], free now: {SMALL=2, MEDIUM=2, LARGE=1}

== 7. Station full -> suggest the nearest station with space ==
   customer at (1.0, 0.4), box 8x20x30cm -> MG01
   same box, max 0.2 km -> none within 0.2 km

== 8. FirstFit vs SmallestFit: doors 01 = LARGE, 02 = SMALL ==
   station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
   station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)

== 9. 50 carriers reserve at once on a 20-door station ==
   reserved=20 rejected=30 distinct doors=20

== 10. Customer return: drop-off code opens an empty door, courier collects ==
   SMS to +91-98450-00010: drop return R1 at locker RT04 with code 490825 by 2026-10-06T11:15:00Z
   return RT04-B1 holds RT04-02(MEDIUM), free now: {SMALL=1, MEDIUM=0, LARGE=0}
   customer types the drop-off code -> DropOffDoorOpened[compartmentId=RT04-02, parcelId=R1]
   booking RT04-B1 is DROPPED_OFF
   courier collects returns: [R1], free now: {SMALL=1, MEDIUM=1, LARGE=0}
   SMS to +91-98450-00011: drop return R2 at locker RT04 with code 988592 by 2026-10-06T11:15:00Z
   2 days later, never dropped off: booking RT04-B2 is CANCELLED, old code -> WrongCode[attemptsLeft=4]
```

Reading the output: the mug (9 × 12 × 12 cm) fits a small door but both were taken, so it was upgraded to a medium one; a second reservation for the same parcel and an oversized box are refused. Codes went only to the customers' phones. The keypad counted the earlier typo, so the fourth guess in section 4 hit the limit of five and locked it — even the real owner had to wait 15 minutes. The shoes' reservation lapsed after 2 hours and the door came back. After 3 days the mug expired, the customer's code now says so, and the courier's visit freed the door. `FirstFit` put a tiny parcel in the only large door and then had to refuse a large parcel; `SmallestFit` served both. Fifty concurrent reservations on a 20-door station produced exactly 20 bookings on 20 different doors. In section 10 the kettle return got the medium door, the drop-off code opened it once, and the courier's visit freed it; the lamp was never dropped off, so after 2 days its booking was cancelled and its code stopped working.

Concurrency & edge cases

Situation	Handling
Two carriers reserve the last medium door at once	`reserve` is `synchronized` : choose + mark RESERVED is one step; the loser gets "no free compartment" (or a larger door)
Courier arrives after the hold ran out	`deposit` sees CANCELLED and refuses; the carrier system reserves again (possibly at another station)
Same parcel reserved twice (retry after a timeout)	Rejected while it has an active booking — make the carrier's reserve call idempotent (repeating it has the same effect as calling it once) by parcel id
Two live parcels get the same random code	Generation retries until the code is unique among live codes at this station
Brute-force guessing	10^6 possible codes, 5 wrong tries then a 15-minute lockout per station; codes also expire
Scheduler down for an hour	Lazy expiry still frees doors on the next operation; only the "expired" SMS is late
Slow SMS gateway	Listeners run outside the lock; in production they hand off to a queue (a buffer that a separate worker drains)
`canFit` says yes but `reserve` then fails (someone else took the door)	Expected: `nearestWithSpace` is a hint; the caller reserves and, on failure, tries the next station
Door jammed / broken	Technician marks a **free** door OUT_OF_SERVICE; an occupied door must be emptied first (`moveTo` enforces it)
Power cut or network loss at the station	Real lockers keep a local copy of live code hashes and sync events later; the design already keeps the station self-contained

Extensions & follow-up questions

1. **"Let the customer extend the pickup window?"** — A method that moves `pickupBy` once, only while DEPOSITED; a policy object can decide how many extensions are allowed.
2. **"Cash on delivery?"** — Payment at the station screen before the door opens: a `PaymentStep` inserted into `pickup` (Chain of Responsibility or a small state machine per session).
3. **"Fairness when the station is nearly full?"** — Keep a few large doors for large parcels only (a `ReservedCapacitySelector` that refuses to upgrade small parcels when fewer than k large doors are free).
4. **"Scale to thousands of stations?"** — Stations are independent, so shard by station id (sharding = splitting the data across several servers, each owning a slice; see System Design Fundamentals); the network search uses a geo index (an index for fast "what is near this point?" queries: a geohash turns a map cell into a short string, an R-tree is a tree of nested rectangles) instead of sorting every station.
5. **"How would you store this in a database?"** — `compartment(id, station_id, size, state)` , `booking(id, parcel_id, compartment_id, status, hold_until, pickup_by, code_hash)` ; reserve with `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check that one row changed.

Real-world example

Amazon Hub Locker sends the customer a pickup code by email or app and gives a limited number of days to collect before the parcel goes back — the `DEPOSITED → EXPIRED → RETURNED` path. InPost's Paczkomat lockers in Poland let couriers load many parcels in one visit and customers open a door with a code or a phone app; the network is dense enough that "nearest locker with space" matters. In both, the door is chosen by the system, not the courier, which is our `CompartmentSelector` .

In the interview

Plan for 40 minutes: 5 min questions (sizes, holds, codes, expiry) → 5 min entities → 8 min the two state machines → 15 min code `reserve`, `deposit`, `pickup` with the lock and the listener call outside it → 7 min security, concurrency and follow-ups.

> **30-second answer:** "A station has doors of a few sizes. A carrier reserves one ahead of time; a Strategy picks the smallest free door that fits, and the choose-and-mark step runs under the station's lock, so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash, and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked lazily on every call, so doors come back even if the scheduler is late, and a network object suggests the nearest station with space."

Follow-ups with answers

- *"Why not let the courier choose any empty door?"* — Couriers would use big doors for small parcels; the system also needs to know the door before the courier arrives.
- *"Why store only a hash of the code?"* — Anyone who reads the database or logs could otherwise open doors.
- *"Why a separate Booking instead of putting the parcel on the Compartment?"* — The door is reused; the booking keeps deadlines and history per parcel.

Self-check

1. **Q:** How does `Size.fits` handle a box lying on its side? **A:** It sorts the three sides of both box and door and compares them in order.
2. **Q:** What happens to an expired parcel's door? **A:** It stays OCCUPIED until the courier runs `collectExpired`, which frees it.
3. **Q:** Why is the listener called after the `synchronized` block in `deposit`? **A:** A slow or failing SMS call must not hold the station lock.
4. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every public method.
5. **Q:** Which pattern lets you switch from smallest-fit to first-fit? **A:** Strategy (`CompartmentSelector`).
6. **Q:** Why is one lock per station enough here but not in a parking lot of 5,000 spots with 20 gates? **A:** A station has tens of doors and few concurrent callers; the lock is held for microseconds.

TL;DR

- Doors of fixed sizes; a **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** under the station lock.
- Two small **state machines**: door (FREE/RESERVED/OCCUPIED/OUT_OF_SERVICE) and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED; a customer return goes RESERVED → DROPPED_OFF → RETURNED).
- Codes are **random, single-use, hashed**, sent only to the customer via an **Observer** outside the lock; lockout after 5 wrong tries.
- Deadlines are enforced **lazily** on every call; a sweep only sends messages. Inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Which two problems use the State pattern with *stateless* states, and which uses states *with data*? **A:** Vending machine and ATM share stateless singletons; the traffic signal's states hold countdowns, so a new instance is created per transition.
2. **Q:** Name the check-then-act race in each of parking, coffee and ATM, and its fix. **A:** Two gates take one spot (atomic `pollFirst`/CAS); two orders take the last milk (one lock around check+deduct); two ATMs overdraw one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton, and why? **A:** Parking lot, coffee machine, traffic controller: a Singleton is a hidden global, blocks testing several instances, and real deployments have many (lots, machines, intersections).
4. **Q:** Where does greedy fail and what replaces it? **A:** Change-making with limited coins (vending) and note selection with limited cassettes (ATM); bounded coin-change dynamic programming.
5. **Q:** Which designs are tick-driven, and why is that useful? **A:** Elevator and traffic signal: deterministic, clock-free tests; a real timer just calls `tick()`/`step()`.
6. **Q:** Which case enforces deadlines *lazily*, and why? **A:** The shipping locker: every call first expires overdue holds and parcels, so doors come back even if the scheduler is late; the scheduled sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** in each problem (spot allocation, change-making, ingredient deduction, withdrawal, next stop, next phase) and spend your time there.
- **State** for machines whose buttons mean different things over time; **Strategy** for rules that change (pricing, dispatch, timing); **Observer** for boards, alerts and monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** useful scope, lock in a **fixed order**, and call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, and tick-driven logic over sleeps.
- For anything with a **deadline** (holds, pickup windows), check it on every call and keep secrets (pickup codes) **hashed and single-use**.